

WSNs – Wireless Sensor Networks

Core Source Code

Receiver & Sensor Modules

STM32L071 – LoRa SX1278 – BME280 – E-Ink Display

Table of Contents

#	File	Section
1	main.h	Receiver - Headers
2	stm32l0xx_hal_conf.h	Receiver - Headers
3	stm32l0xx_it.h	Receiver - Headers
4	sx1278.h	Receiver - Headers
5	main.c	Receiver - Sources
6	stm32l0xx_hal_msp.c	Receiver - Sources
7	stm32l0xx_it.c	Receiver - Sources
8	sx1278.c	Receiver - Sources
9	syscalls.c	Receiver - Sources
10	sysmem.c	Receiver - Sources
11	system_stm32l0xx.c	Receiver - Sources
12	bme280.h	Sensor - Headers
13	cmd.h	Sensor - Headers
14	eink_display.h	Sensor - Headers
15	main.h	Sensor - Headers
16	power_manager.h	Sensor - Headers
17	stm32l0xx_hal_conf.h	Sensor - Headers
18	stm32l0xx_it.h	Sensor - Headers
19	sx1278.h	Sensor - Headers
20	bme280.c	Sensor - Sources
21	cmd.c	Sensor - Sources
22	eink_display.c	Sensor - Sources
23	main.c	Sensor - Sources
24	power_manager.c	Sensor - Sources
25	stm32l0xx_hal_msp.c	Sensor - Sources
26	stm32l0xx_it.c	Sensor - Sources
27	sx1278.c	Sensor - Sources
28	syscalls.c	Sensor - Sources
29	sysmem.c	Sensor - Sources
30	system_stm32l0xx.c	Sensor - Sources

Receiver - Headers → [main.h](#)

Path: receiver/Core/Inc/main.h | Lines: 84

```
1  /* USER CODE BEGIN Header */
2  /**
3  ****
4  * @file : main.h
5  * @brief : Header for main.c file.
6  * This file contains the common defines of the application.
7  ****
8  * @attention
9  *
10 * Copyright (c) 2026 STMicroelectronics.
11 * All rights reserved.
12 *
13 * This software is licensed under terms that can be found in the LICENSE file
14 * in the root directory of this software component.
15 * If no LICENSE file comes with this software, it is provided AS-IS.
16 *
17 ****
18 */
19 /* USER CODE END Header */
20
21 /* Define to prevent recursive inclusion -----*/
22 #ifndef __MAIN_H
23 #define __MAIN_H
24
25 #ifdef __cplusplus
26 extern "C" {
27 #endif
28
29 /* Includes -----*/
30 #include <stm32l0xx_hal.h>;
31
32 /* Private includes -----*/
33 /* USER CODE BEGIN Includes */
34
35 /* USER CODE END Includes */
36
37 /* Exported types -----*/
38 /* USER CODE BEGIN ET */
39
40 /* USER CODE END ET */
41
42 /* Exported constants -----*/
43 /* USER CODE BEGIN EC */
44
45 /* USER CODE END EC */
46
47 /* Exported macro -----*/
48 /* USER CODE BEGIN EM */
49
50 /* USER CODE END EM */
51
52 /* Exported functions prototypes -----*/
53 void Error_Handler(void);
54
55 /* USER CODE BEGIN EFP */
56
57 /* USER CODE END EFP */
58
59 /* Private defines -----*/
60 #define LORA_NSS_Pin GPIO_PIN_4
61 #define LORA_NSS_GPIO_Port GPIOA
62 #define LORA_DIO0_Pin GPIO_PIN_4
63 #define LORA_DIO0_GPIO_Port GPIOC
64 #define LORA_RST_Pin GPIO_PIN_5
65 #define LORA_RST_GPIO_Port GPIOC
66 #define EINK_D_C_Pin GPIO_PIN_12
67 #define EINK_D_C_GPIO_Port GPIOB
68 #define EINK_BUSY_Pin GPIO_PIN_14
69 #define EINK_BUSY_GPIO_Port GPIOB
70 #define EINK_CS_Pin GPIO_PIN_6
71 #define EINK_CS_GPIO_Port GPIOC
72 #define EINK_RES_Pin GPIO_PIN_7
73 #define EINK_RES_GPIO_Port GPIOC
```

```

74
75 /* USER CODE BEGIN Private defines */
76
77 /* USER CODE END Private defines */
78
79 #ifdef __cplusplus
80 }
81 #endif
82
83 #endif /* __MAIN_H */
84

```

Receiver - Headers → [stm32l0xx_hal_conf.h](#)

Path: receiver/Core/Inc/stm32l0xx_hal_conf.h | Lines: 330

```

1  /* USER CODE BEGIN Header */
2  /**
3   * *****
4   * @file stm32l0xx_hal_conf.h
5   * @author MCD Application Team
6   * @brief HAL configuration template file.
7   * This file should be copied to the application folder and renamed
8   * to stm32l0xx_hal_conf.h.
9   * *****
10  * @attention
11  *
12  * Copyright (c) 2016 STMicroelectronics.
13  * All rights reserved.
14  *
15  * This software is licensed under terms that can be found in the LICENSE file
16  * in the root directory of this software component.
17  * If no LICENSE file comes with this software, it is provided AS-IS.
18  *
19  * *****
20  */
21  /* USER CODE END Header */
22
23  /* Define to prevent recursive inclusion ----- */
24  #ifndef __STM32L0xx_HAL_CONF_H
25  #define __STM32L0xx_HAL_CONF_H
26
27  #ifdef __cplusplus
28  extern "C" {
29  #endif
30
31  /* Exported types ----- */
32  /* Exported constants ----- */
33
34  /* ##### Module Selection ##### */
35  /**
36  * @brief This is the list of modules to be used in the HAL driver
37  */
38
39  #define HAL_MODULE_ENABLED
40  /*#define HAL_ADC_MODULE_ENABLED */
41  /*#define HAL_Cryp_MODULE_ENABLED */
42  /*#define HAL_COMP_MODULE_ENABLED */
43  #define HAL_I2C_MODULE_ENABLED
44  /*#define HAL_CRC_MODULE_ENABLED */
45  /*#define HAL_Cryp_MODULE_ENABLED */
46  /*#define HAL_DAC_MODULE_ENABLED */
47  /*#define HAL_FIREWALL_MODULE_ENABLED */
48  /*#define HAL_I2S_MODULE_ENABLED */
49  /*#define HAL_IWDG_MODULE_ENABLED */
50  /*#define HAL_LCD_MODULE_ENABLED */
51  /*#define HAL_LPTIM_MODULE_ENABLED */
52  /*#define HAL_RNG_MODULE_ENABLED */
53  #define HAL_RTC_MODULE_ENABLED
54  #define HAL_SPI_MODULE_ENABLED
55  /*#define HAL_TIM_MODULE_ENABLED */
56  /*#define HAL_TSC_MODULE_ENABLED */
57  #define HAL_UART_MODULE_ENABLED
58  /*#define HAL_USART_MODULE_ENABLED */
59  /*#define HAL_IRDA_MODULE_ENABLED */
60  /*#define HAL_SMARTCARD_MODULE_ENABLED */

```

```

61 /**define HAL_SMBUS_MODULE_ENABLED */
62 /**define HAL_WWDG_MODULE_ENABLED */
63 /**define HAL_PCD_MODULE_ENABLED */
64 #define HAL_GPIO_MODULE_ENABLED
65 #define HAL_EXTI_MODULE_ENABLED
66 #define HAL_DMA_MODULE_ENABLED
67 #define HAL_RCC_MODULE_ENABLED
68 #define HAL_FLASH_MODULE_ENABLED
69 #define HAL_PWR_MODULE_ENABLED
70 #define HAL_CORTEX_MODULE_ENABLED
71
72 /* ##### Oscillator Values adaptation ##### */
73 /**
74  * @brief Adjust the value of External High Speed oscillator (HSE) used in your application.
75  * This value is used by the RCC HAL module to compute the system frequency
76  * (when HSE is used as system clock source, directly or through the PLL).
77  */
78 #if !defined (HSE_VALUE)
79 #define HSE_VALUE ((uint32_t)8000000U) /*!&lt; Value of the External oscillator in Hz */
80 #endif /* HSE_VALUE */
81
82 #if !defined (HSE_STARTUP_TIMEOUT)
83 #define HSE_STARTUP_TIMEOUT ((uint32_t)100U) /*!&lt; Time out for HSE start up, in ms */
84 #endif /* HSE_STARTUP_TIMEOUT */
85
86 /**
87  * @brief Internal Multiple Speed oscillator (MSI) default value.
88  * This value is the default MSI range value after Reset.
89  */
90 #if !defined (MSI_VALUE)
91 #define MSI_VALUE ((uint32_t)2097000U) /*!&lt; Value of the Internal oscillator in Hz*/
92 #endif /* MSI_VALUE */
93
94 /**
95  * @brief Internal High Speed oscillator (HSI) value.
96  * This value is used by the RCC HAL module to compute the system frequency
97  * (when HSI is used as system clock source, directly or through the PLL).
98  */
99 #if !defined (HSI_VALUE)
100 #define HSI_VALUE ((uint32_t)16000000U) /*!&lt; Value of the Internal oscillator in Hz*/
101 #endif /* HSI_VALUE */
102
103 /**
104  * @brief Internal High Speed oscillator for USB (HSI48) value.
105  */
106 #if !defined (HSI48_VALUE)
107 #define HSI48_VALUE ((uint32_t)48000000U) /*!&lt; Value of the Internal High Speed oscillator for USB in Hz.
108 The real value may vary depending on the variations
109 in voltage and temperature. */
110 #endif /* HSI48_VALUE */
111
112 /**
113  * @brief Internal Low Speed oscillator (LSI) value.
114  */
115 #if !defined (LSI_VALUE)
116 #define LSI_VALUE ((uint32_t)37000U) /*!&lt; LSI Typical Value in Hz*/
117 #endif /* LSI_VALUE */ /*!&lt; Value of the Internal Low Speed oscillator in Hz
118 The real value may vary depending on the variations
119 in voltage and temperature.*/
120 /**
121  * @brief External Low Speed oscillator (LSE) value.
122  * This value is used by the UART, RTC HAL module to compute the system frequency
123  */
124 #if !defined (LSE_VALUE)
125 #define LSE_VALUE ((uint32_t)32768U) /*!&lt; Value of the External oscillator in Hz*/
126 #endif /* LSE_VALUE */
127
128 #if !defined (LSE_STARTUP_TIMEOUT)
129 #define LSE_STARTUP_TIMEOUT ((uint32_t)5000U) /*!&lt; Time out for LSE start up, in ms */
130 #endif /* LSE_STARTUP_TIMEOUT */
131
132 /* Tip: To avoid modifying this file each time you need to use different HSE,
133 === you can define the HSE value in your toolchain compiler preprocessor. */
134
135 /* ##### System Configuration ##### */
136 /**

```

```

137 * @brief This is the HAL system configuration section
138 */
139 #define VDD_VALUE ((uint32_t)3300U) /*!&lt; Value of VDD in mv */
140 #define TICK_INT_PRIORITY ((uint32_t)3U) /*!&lt; tick interrupt priority */
141 #define USE_RTOS 0U
142 #define PREFETCH_ENABLE 0U
143 #define PREREAD_ENABLE 1U
144 #define BUFFER_CACHE_DISABLE 0U
145
146 /* ##### Assert Selection ##### */
147 /**
148 * @brief Uncomment the line below to expanse the "assert_param" macro in the
149 * HAL drivers code
150 */
151 /* #define USE_FULL_ASSERT 1U */
152
153 /* ##### Register callback feature configuration ##### */
154 /**
155 * @brief Set below the peripheral configuration to "1U" to add the support
156 * of HAL callback registration/deregistration feature for the HAL
157 * driver(s). This allows user application to provide specific callback
158 * functions thanks to HAL_PPP_RegisterCallback() rather than overwriting
159 * the default weak callback functions (see each stm32l0xx_hal_ppp.h file
160 * for possible callback identifiers defined in HAL_PPP_CallbackIDTypeDef
161 * for each PPP peripheral).
162 */
163 #define USE_HAL_ADC_REGISTER_CALLBACKS 0U
164 #define USE_HAL_COMP_REGISTER_CALLBACKS 0U
165 #define USE_HAL_DAC_REGISTER_CALLBACKS 0U
166 #define USE_HAL_I2C_REGISTER_CALLBACKS 0U
167 #define USE_HAL_I2S_REGISTER_CALLBACKS 0U
168 #define USE_HAL_IRDA_REGISTER_CALLBACKS 0U
169 #define USE_HAL_LPTIM_REGISTER_CALLBACKS 0U
170 #define USE_HAL_PCD_REGISTER_CALLBACKS 0U
171 #define USE_HAL_RNG_REGISTER_CALLBACKS 0U
172 #define USE_HAL_RTC_REGISTER_CALLBACKS 0U
173 #define USE_HAL_SMARTCARD_REGISTER_CALLBACKS 0U
174 #define USE_HAL_SMBUS_REGISTER_CALLBACKS 0U
175 #define USE_HAL_SPI_REGISTER_CALLBACKS 0U
176 #define USE_HAL_TIM_REGISTER_CALLBACKS 0U
177 #define USE_HAL_TSC_REGISTER_CALLBACKS 0U
178 #define USE_HAL_UART_REGISTER_CALLBACKS 0U
179 #define USE_HAL_USART_REGISTER_CALLBACKS 0U
180 #define USE_HAL_WWDG_REGISTER_CALLBACKS 0U
181
182 /* Includes -----*/
183 /**
184 * @brief Include module's header file
185 */
186
187 #ifdef HAL_RCC_MODULE_ENABLED
188 #include &quot;stm32l0xx_hal_rcc.h&quot;;
189 #endif /* HAL_RCC_MODULE_ENABLED */
190
191 #ifdef HAL_EXTI_MODULE_ENABLED
192 #include &quot;stm32l0xx_hal_exti.h&quot;;
193 #endif /* HAL_EXTI_MODULE_ENABLED */
194
195 #ifdef HAL_GPIO_MODULE_ENABLED
196 #include &quot;stm32l0xx_hal_gpio.h&quot;;
197 #endif /* HAL_GPIO_MODULE_ENABLED */
198
199 #ifdef HAL_DMA_MODULE_ENABLED
200 #include &quot;stm32l0xx_hal_dma.h&quot;;
201 #endif /* HAL_DMA_MODULE_ENABLED */
202
203 #ifdef HAL_CORTEX_MODULE_ENABLED
204 #include &quot;stm32l0xx_hal_cortex.h&quot;;
205 #endif /* HAL_CORTEX_MODULE_ENABLED */
206
207 #ifdef HAL_ADC_MODULE_ENABLED
208 #include &quot;stm32l0xx_hal_adc.h&quot;;
209 #endif /* HAL_ADC_MODULE_ENABLED */
210
211 #ifdef HAL_COMP_MODULE_ENABLED

```

```

212 #include <stm32l0xx_hal_comp.h>;
213 #endif /* HAL_COMP_MODULE_ENABLED */
214
215 #ifdef HAL_CRC_MODULE_ENABLED
216 #include <stm32l0xx_hal_crc.h>;
217 #endif /* HAL_CRC_MODULE_ENABLED */
218
219 #ifdef HAL_CRYP_MODULE_ENABLED
220 #include <stm32l0xx_hal_cryp.h>;
221 #endif /* HAL_CRYP_MODULE_ENABLED */
222
223 #ifdef HAL_DAC_MODULE_ENABLED
224 #include <stm32l0xx_hal_dac.h>;
225 #endif /* HAL_DAC_MODULE_ENABLED */
226
227 #ifdef HAL_FIREWALL_MODULE_ENABLED
228 #include <stm32l0xx_hal_firewall.h>;
229 #endif /* HAL_FIREWALL_MODULE_ENABLED */
230
231 #ifdef HAL_FLASH_MODULE_ENABLED
232 #include <stm32l0xx_hal_flash.h>;
233 #endif /* HAL_FLASH_MODULE_ENABLED */
234
235 #ifdef HAL_I2C_MODULE_ENABLED
236 #include <stm32l0xx_hal_i2c.h>;
237 #endif /* HAL_I2C_MODULE_ENABLED */
238
239 #ifdef HAL_I2S_MODULE_ENABLED
240 #include <stm32l0xx_hal_i2s.h>;
241 #endif /* HAL_I2S_MODULE_ENABLED */
242
243 #ifdef HAL_IWDG_MODULE_ENABLED
244 #include <stm32l0xx_hal_iwdg.h>;
245 #endif /* HAL_IWDG_MODULE_ENABLED */
246
247 #ifdef HAL_LCD_MODULE_ENABLED
248 #include <stm32l0xx_hal_lcd.h>;
249 #endif /* HAL_LCD_MODULE_ENABLED */
250
251 #ifdef HAL_LPTIM_MODULE_ENABLED
252 #include <stm32l0xx_hal_lptim.h>;
253 #endif /* HAL_LPTIM_MODULE_ENABLED */
254
255 #ifdef HAL_PWR_MODULE_ENABLED
256 #include <stm32l0xx_hal_pwr.h>;
257 #endif /* HAL_PWR_MODULE_ENABLED */
258
259 #ifdef HAL_RNG_MODULE_ENABLED
260 #include <stm32l0xx_hal_rng.h>;
261 #endif /* HAL_RNG_MODULE_ENABLED */
262
263 #ifdef HAL_RTC_MODULE_ENABLED
264 #include <stm32l0xx_hal_rtc.h>;
265
266 #endif /* HAL_RTC_MODULE_ENABLED */
267
268 #ifdef HAL_SPI_MODULE_ENABLED
269 #include <stm32l0xx_hal_spi.h>;
270 #endif /* HAL_SPI_MODULE_ENABLED */
271
272 #ifdef HAL_TIM_MODULE_ENABLED
273 #include <stm32l0xx_hal_tim.h>;
274 #endif /* HAL_TIM_MODULE_ENABLED */
275
276 #ifdef HAL_TSC_MODULE_ENABLED
277 #include <stm32l0xx_hal_tsc.h>;
278 #endif /* HAL_TSC_MODULE_ENABLED */
279
280 #ifdef HAL_UART_MODULE_ENABLED
281 #include <stm32l0xx_hal_uart.h>;
282 #endif /* HAL_UART_MODULE_ENABLED */
283
284 #ifdef HAL_USART_MODULE_ENABLED
285 #include <stm32l0xx_hal_usart.h>;
286 #endif /* HAL_USART_MODULE_ENABLED */

```

```

287
288 #ifndef HAL_IRDA_MODULE_ENABLED
289 #include <stm32l0xx_hal_irda.h>;
290 #endif /* HAL_IRDA_MODULE_ENABLED */
291
292 #ifndef HAL_SMARTCARD_MODULE_ENABLED
293 #include <stm32l0xx_hal_smartcard.h>;
294 #endif /* HAL_SMARTCARD_MODULE_ENABLED */
295
296 #ifndef HAL_SMBUS_MODULE_ENABLED
297 #include <stm32l0xx_hal_smbus.h>;
298 #endif /* HAL_SMBUS_MODULE_ENABLED */
299
300 #ifndef HAL_WWDG_MODULE_ENABLED
301 #include <stm32l0xx_hal_wwdg.h>;
302 #endif /* HAL_WWDG_MODULE_ENABLED */
303
304 #ifndef HAL_PCD_MODULE_ENABLED
305 #include <stm32l0xx_hal_pcd.h>;
306 #endif /* HAL_PCD_MODULE_ENABLED */
307
308 /* Exported macro -----*/
309 #ifndef USE_FULL_ASSERT
310 /**
311 * @brief The assert_param macro is used for function's parameters check.
312 * @param expr: If expr is false, it calls assert_failed function
313 * which reports the name of the source file and the source
314 * line number of the call that failed.
315 * If expr is true, it returns no value.
316 * @retval None
317 */
318 #define assert_param(expr) ((expr) ? (void)0U : assert_failed((uint8_t *)__FILE__, __LINE__))
319 /* Exported functions ----- */
320 void assert_failed(uint8_t* file, uint32_t line);
321 #else
322 #define assert_param(expr) ((void)0U)
323 #endif /* USE_FULL_ASSERT */
324
325 #ifdef __cplusplus
326 }
327 #endif
328
329 #endif /* __STM32L0xx_HAL_CONF_H */
330

```

Receiver - Headers → [stm32l0xx_it.h](#)

Path: receiver/Core/Inc/stm32l0xx_it.h | Lines: 63

```

1 /* USER CODE BEGIN Header */
2 /**
3 *****
4 * @file stm32l0xx_it.h
5 * @brief This file contains the headers of the interrupt handlers.
6 *****
7 * @attention
8 *
9 * Copyright (c) 2026 STMicroelectronics.
10 * All rights reserved.
11 *
12 * This software is licensed under terms that can be found in the LICENSE file
13 * in the root directory of this software component.
14 * If no LICENSE file comes with this software, it is provided AS-IS.
15 *
16 *****
17 */
18 /* USER CODE END Header */
19
20 /* Define to prevent recursive inclusion -----*/
21 #ifndef __STM32L0xx_IT_H
22 #define __STM32L0xx_IT_H
23
24 #ifdef __cplusplus
25 extern "C" {
26 #endif
27

```

```

28 /* Private includes ----- */
29 /* USER CODE BEGIN Includes */
30
31 /* USER CODE END Includes */
32
33 /* Exported types ----- */
34 /* USER CODE BEGIN ET */
35
36 /* USER CODE END ET */
37
38 /* Exported constants ----- */
39 /* USER CODE BEGIN EC */
40
41 /* USER CODE END EC */
42
43 /* Exported macro ----- */
44 /* USER CODE BEGIN EM */
45
46 /* USER CODE END EM */
47
48 /* Exported functions prototypes ----- */
49 void NMI_Handler(void);
50 void HardFault_Handler(void);
51 void SVC_Handler(void);
52 void PendSV_Handler(void);
53 void SysTick_Handler(void);
54 /* USER CODE BEGIN EFP */
55
56 /* USER CODE END EFP */
57
58 #ifdef __cplusplus
59 }
60 #endif
61
62 #endif /* __STM32L0xx_IT_H */
63

```

Receiver - Headers → [sx1278.h](#)

Path: receiver/Core/Inc/sx1278.h | Lines: 76

```

1 #ifndef __SX1278_H
2 #define __SX1278_H
3
4 #include <main.h>;
5 #include <stdint.h>;
6 #include <stdbool.h>;
7
8 #define SX1278_REG_FIFO 0x00
9 #define SX1278_REG_OP_MODE 0x01
10 #define SX1278_REG_FRF_MSB 0x06
11 #define SX1278_REG_FRF_MID 0x07
12 #define SX1278_REG_FRF_LSB 0x08
13 #define SX1278_REG_PA_CONFIG 0x09
14 #define SX1278_REG_PA_RAMP 0x0A
15 #define SX1278_REG_OCP 0x0B
16 #define SX1278_REG_LNA 0x0C
17 #define SX1278_REG_FIFO_ADDR_PTR 0x0D
18 #define SX1278_REG_FIFO_TX_BASE_ADDR 0x0E
19 #define SX1278_REG_FIFO_RX_BASE_ADDR 0x0F
20 #define SX1278_REG_FIFO_RX_CURRENT_ADDR 0x10
21 #define SX1278_REG_IRQ_FLAGS_MASK 0x11
22 #define SX1278_REG_IRQ_FLAGS 0x12
23 #define SX1278_REG_RX_NB_BYTES 0x13
24 #define SX1278_REG_PKT_SNR_VALUE 0x19
25 #define SX1278_REG_PKT_RSSI_VALUE 0x1A
26 #define SX1278_REG_MODEM_CONFIG_1 0x1D
27 #define SX1278_REG_MODEM_CONFIG_2 0x1E
28 #define SX1278_REG_SYMB_TIMEOUT_LSB 0x1F
29 #define SX1278_REG_MODEM_CONFIG_3 0x26
30 #define SX1278_REG_PREAMBLE_MSB 0x20
31 #define SX1278_REG_PREAMBLE_LSB 0x21
32 #define SX1278_REG_SYNC_WORD 0x39
33 #define SX1278_REG_PAYLOAD_LENGTH 0x22
34 #define SX1278_REG_FIFO_RX_BYTE_ADDR 0x25
35 #define SX1278_REG_DIO_MAPPING_1 0x40

```

```

36 #define SX1278_REG_DIO_MAPPING_2 0x41
37 #define SX1278_REG_VERSION 0x42
38 #define SX1278_REG_PA_DAC 0x4D
39
40 #define SX1278_MODE_LONG_RANGE 0x80
41 #define SX1278_MODE_SLEEP 0x00
42 #define SX1278_MODE_STDBY 0x01
43 #define SX1278_MODE_TX 0x03
44 #define SX1278_MODE_RX_CONTINUOUS 0x05
45 #define SX1278_MODE_RX_SINGLE 0x06
46
47 #define SX1278_FXOSC 32000000ULL
48
49 #define SX1278_IRQ_TX_DONE_MASK 0x08
50 #define SX1278_IRQ_RX_DONE_MASK 0x40
51 #define SX1278_IRQ_RX_TIMEOUT_MASK 0x80
52 #define SX1278_IRQ_PAYLOAD_CRC_ERROR_MASK 0x20
53
54 #define SX1278_DIO0_TX_DONE 0x01
55 #define SX1278_DIO0_RX_DONE 0x00
56
57 #define SX1278_MAX_PACKET_SIZE 255
58
59 extern volatile bool sx1278_tx_done;
60 extern volatile bool sx1278_rx_done;
61
62 bool SX1278_Init(void);
63 bool SX1278_SendPacket(uint8_t *data, uint8_t len);
64 void SX1278_HardReset(void);
65 void SX1278_EnterSleep(void);
66 void SX1278_SetStandby(void);
67
68 void SX1278_StartRX(void);
69 bool SX1278_ReadPacket(uint8_t *data, uint8_t *len);
70 int SX1278_GetLastRSSI(void);
71 int SX1278_GetLastSNR(void);
72 uint8_t SX1278_ReadIrqFlags(void);
73 void SX1278_ClearIRQ(void);
74
75 #endif
76

```

Receiver - Sources → [main.c](#)

Path: receiver/Core/Src/main.c | Lines: 537

```

1  /* USER CODE BEGIN Header */
2  /**
3   * *****
4   * @file : main.c
5   * @brief : Main program body
6   * *****
7   * @attention
8   *
9   * Copyright (c) 2026 STMicroelectronics.
10  * All rights reserved.
11  *
12  * This software is licensed under terms that can be found in the LICENSE file
13  * in the root directory of this software component.
14  * If no LICENSE file comes with this software, it is provided AS-IS.
15  *
16  * *****
17  */
18  /* USER CODE END Header */
19  /* Includes ----- */
20  #include "main.h";
21
22  /* Private includes ----- */
23  /* USER CODE BEGIN Includes */
24  #include "sx1278.h";
25  #include "stdio.h";
26  /* USER CODE END Includes */
27
28  /* Private typedef ----- */
29  /* USER CODE BEGIN PTD */
30

```

```

31 /* USER CODE END PTD */
32
33 /* Private define -----*/
34 /* USER CODE BEGIN PD */
35
36 /* USER CODE END PD */
37
38 /* Private macro -----*/
39 /* USER CODE BEGIN PM */
40
41 /* USER CODE END PM */
42
43 /* Private variables -----*/
44 I2C_HandleTypeDef hi2c3;
45
46 RTC_HandleTypeDef hrtc;
47
48 SPI_HandleTypeDef hspi1;
49 SPI_HandleTypeDef hspi2;
50
51 UART_HandleTypeDef huart2;
52
53 /* USER CODE BEGIN PV */
54
55 char msg_buf[128];
56 uint8_t rx_payload[32];
57 uint8_t rx_len;
58
59 /* USER CODE END PV */
60
61 /* Private function prototypes -----*/
62 void SystemClock_Config(void);
63
64 static void MX_GPIO_Init(void);
65
66 static void MX_I2C3_Init(void);
67
68 static void MX_SPI1_Init(void);
69
70 static void MX_SPI2_Init(void);
71
72 static void MX_USART2_UART_Init(void);
73
74 static void MX_RTC_Init(void);
75
76 /* USER CODE BEGIN PFP */
77
78 /* USER CODE END PFP */
79
80 /* Private user code -----*/
81 /* USER CODE BEGIN 0 */
82
83 /* USER CODE END 0 */
84
85 /**
86  * @brief The application entry point.
87  * @retval int
88  */
89 int main(void)
90 {
91 /* USER CODE BEGIN 1 */
92
93 /* USER CODE END 1 */
94
95 /* MCU Configuration-----*/
96
97 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
98 HAL_Init();
99
100 /* USER CODE BEGIN Init */
101
102 /* USER CODE END Init */
103
104 /* Configure the system clock */
105 SystemClock_Config();

```

```

106
107 /* USER CODE BEGIN SysInit */
108
109 /* USER CODE END SysInit */
110
111 /* Initialize all configured peripherals */
112 MX_GPIO_Init();
113 MX_I2C3_Init();
114 MX_SPI1_Init();
115 MX_SPI2_Init();
116 MX_USART2_UART_Init();
117 MX_RTC_Init();
118 /* USER CODE BEGIN 2 */
119
120 if (!SX1278_Init())
121 {
122     const char *fail = "--- LoRa Init FAILED ---\r\n";
123     HAL_UART_Transmit(&huart2, (uint8_t *)fail, 24, 1000);
124     Error_Handler();
125 }
126
127 const char *startup = "--- LoRa Gateway Ready ---\r\n";
128 HAL_UART_Transmit(&huart2, (uint8_t *)startup, 26, 1000);
129
130 /* USER CODE END 2 */
131
132 /* Infinite loop */
133 /* USER CODE BEGIN WHILE */
134 while (1)
135 {
136     /* USER CODE END WHILE */
137
138     /* USER CODE BEGIN 3 */
139
140     SX1278_StartRX();
141
142     rx_len = 0;
143     uint32_t rx_timeout = HAL_GetTick() + 900000;
144
145     while (1)
146     {
147         if (SX1278_ReadPacket(rx_payload, &rx_len)) break;
148         if (HAL_GetTick() > rx_timeout) break;
149         HAL_Delay(200);
150     }
151
152     if (rx_len == 0) continue;
153
154     // Validate checksum and magic byte
155     uint8_t cksum = rx_payload[0] ^ rx_payload[1] ^ rx_payload[2]
156     ^ rx_payload[3] ^ rx_payload[4] ^ rx_payload[5]
157     ^ rx_payload[6];
158     if (cksum != rx_payload[7] || rx_payload[9] != 0xAA)
159     {
160         continue;
161     }
162
163     int16_t temp_raw = (int16_t)(((uint16_t)rx_payload[1] << 8) | rx_payload[2]);
164     int16_t hum_raw = (int16_t)(((uint16_t)rx_payload[3] << 8) | rx_payload[4]);
165     uint16_t press_raw = ((uint16_t)rx_payload[5] << 8) | rx_payload[6];
166
167     int t_i = (int)temp_raw / 10;
168     int t_d = (int)temp_raw % 10;
169     int h_i = (int)hum_raw / 10;
170     int h_d = (int)hum_raw % 10;
171     int p_i = (int)press_raw / 10;
172     int p_d = (int)press_raw % 10;
173
174     int rssi = SX1278_GetLastRSSI();
175     int snr = SX1278_GetLastSNR();
176
177     int len = snprintf(msg_buf, sizeof(msg_buf),
178     "--- Tram Do Moi Truong ---\r\n"
179     "Station ID: %d\r\n"
180     "Nhiet do: %d.%d C\r\n"

```

```

181 "Do am: %d.%d %%\r\n"
182 "Ap suat: %d.%d hPa\r\n"
183 "RSSI: %d dBm / SNR: %d dB\r\n"
184 "\r\n",
185 rx_payload[0],
186 t_i, t_d < 0 ? -t_d : t_d,
187 h_i, h_d < 0 ? -h_d : h_d,
188 p_i, p_d < 0 ? -p_d : p_d,
189 rssi, snr);
190 HAL_UART_Transmit(&huart2, (uint8_t *)msg_buf, (uint16_t)len, 1000);
191 }
192 /* USER CODE END 3 */
193 }
194
195 /**
196  * @brief System Clock Configuration
197  * @retval None
198  */
199 void SystemClock_Config(void)
200 {
201     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
202     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
203     RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};
204
205     /** Configure the main internal regulator output voltage
206     */
207     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
208
209     /** Initializes the RCC Oscillators according to the specified parameters
210     * in the RCC_OscInitTypeDef structure.
211     */
212     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSI | RCC_OSCILLATORTYPE_MSI;
213     RCC_OscInitStruct.LSIState = RCC_LSI_ON;
214     RCC_OscInitStruct.MSIState = RCC_MSI_ON;
215     RCC_OscInitStruct.MSICalibrationValue = 0;
216     RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_5;
217     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
218     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
219     {
220         Error_Handler();
221     }
222
223     /** Initializes the CPU, AHB and APB buses clocks
224     */
225     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
226     | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
227     RCC_ClkInitStruct.SYSClkSource = RCC_SYSCCLKSOURCE_MSI;
228     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCCLK_DIV1;
229     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
230     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
231
232     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
233     {
234         Error_Handler();
235     }
236     PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_USART2 | RCC_PERIPHCLK_I2C3
237     | RCC_PERIPHCLK_RTC;
238     PeriphClkInit.Usart2ClockSelection = RCC_USART2CLKSOURCE_PCLK1;
239     PeriphClkInit.I2c3ClockSelection = RCC_I2C3CLKSOURCE_PCLK1;
240     PeriphClkInit.RTCClockSelection = RCC_RTCCLKSOURCE_LSI;
241     if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
242     {
243         Error_Handler();
244     }
245 }
246
247 /**
248  * @brief I2C3 Initialization Function
249  * @param None
250  * @retval None
251  */
252 static void MX_I2C3_Init(void)
253 {
254     /* USER CODE BEGIN I2C3_Init 0 */
255
256     /* USER CODE END I2C3_Init 0 */

```

```

257
258 /* USER CODE BEGIN I2C3_Init 1 */
259
260 /* USER CODE END I2C3_Init 1 */
261 hi2c3.Instance = I2C3;
262 hi2c3.Init.Timing = 0x00000608;
263 hi2c3.Init.OwnAddress1 = 0;
264 hi2c3.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
265 hi2c3.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
266 hi2c3.Init.OwnAddress2 = 0;
267 hi2c3.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
268 hi2c3.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
269 hi2c3.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
270 if (HAL_I2C_Init(&hi2c3) != HAL_OK)
271 {
272   Error_Handler();
273 }
274
275 /** Configure Analogue filter
276 */
277 if (HAL_I2CEx_ConfigAnalogFilter(&hi2c3, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
278 {
279   Error_Handler();
280 }
281
282 /** Configure Digital filter
283 */
284 if (HAL_I2CEx_ConfigDigitalFilter(&hi2c3, 0) != HAL_OK)
285 {
286   Error_Handler();
287 }
288 /* USER CODE BEGIN I2C3_Init 2 */
289
290 /* USER CODE END I2C3_Init 2 */
291 }
292
293 /**
294 * @brief RTC Initialization Function
295 * @param None
296 * @retval None
297 */
298 static void MX_RTC_Init(void)
299 {
300 /* USER CODE BEGIN RTC_Init 0 */
301
302 /* USER CODE END RTC_Init 0 */
303
304 /* USER CODE BEGIN RTC_Init 1 */
305
306 /* USER CODE END RTC_Init 1 */
307
308 /** Initialize RTC Only
309 */
310 hrtc.Instance = RTC;
311 hrtc.Init.HourFormat = RTC_HOURFORMAT_24;
312 hrtc.Init.AsynchPrediv = 127;
313 hrtc.Init.SynchPrediv = 255;
314 hrtc.Init.OutPut = RTC_OUTPUT_DISABLE;
315 hrtc.Init.OutPutRemap = RTC_OUTPUT_REMAP_NONE;
316 hrtc.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
317 hrtc.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;
318 if (HAL_RTC_Init(&hrtc) != HAL_OK)
319 {
320   Error_Handler();
321 }
322
323 /** Enable the WakeUp
324 */
325 if (HAL_RTCEx_SetWakeUpTimer(&hrtc, 0, RTC_WAKEUPCLOCK_RTCCLK_DIV16) != HAL_OK)
326 {
327   Error_Handler();
328 }
329 /* USER CODE BEGIN RTC_Init 2 */
330
331 /* USER CODE END RTC_Init 2 */

```

```

332 }
333
334 /**
335 * @brief SPI1 Initialization Function
336 * @param None
337 * @retval None
338 */
339 static void MX_SPI1_Init(void)
340 {
341     /* USER CODE BEGIN SPI1_Init 0 */
342
343     /* USER CODE END SPI1_Init 0 */
344
345     /* USER CODE BEGIN SPI1_Init 1 */
346
347     /* USER CODE END SPI1_Init 1 */
348     /* SPI1 parameter configuration*/
349     hspi1.Instance = SPI1;
350     hspi1.Init.Mode = SPI_MODE_MASTER;
351     hspi1.Init.Direction = SPI_DIRECTION_2LINES;
352     hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
353     hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
354     hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;
355     hspi1.Init.NSS = SPI_NSS_SOFT;
356     hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_2;
357     hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
358     hspi1.Init.TIMode = SPI_TIMODE_DISABLE;
359     hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
360     hspi1.Init.CRCPolynomial = 7;
361     if (HAL_SPI_Init(&hspi1) != HAL_OK)
362     {
363         Error_Handler();
364     }
365     /* USER CODE BEGIN SPI1_Init 2 */
366
367     /* USER CODE END SPI1_Init 2 */
368 }
369
370 /**
371 * @brief SPI2 Initialization Function
372 * @param None
373 * @retval None
374 */
375 static void MX_SPI2_Init(void)
376 {
377     /* USER CODE BEGIN SPI2_Init 0 */
378
379     /* USER CODE END SPI2_Init 0 */
380
381     /* USER CODE BEGIN SPI2_Init 1 */
382
383     /* USER CODE END SPI2_Init 1 */
384     /* SPI2 parameter configuration*/
385     hspi2.Instance = SPI2;
386     hspi2.Init.Mode = SPI_MODE_MASTER;
387     hspi2.Init.Direction = SPI_DIRECTION_2LINES;
388     hspi2.Init.DataSize = SPI_DATASIZE_8BIT;
389     hspi2.Init.CLKPolarity = SPI_POLARITY_LOW;
390     hspi2.Init.CLKPhase = SPI_PHASE_1EDGE;
391     hspi2.Init.NSS = SPI_NSS_SOFT;
392     hspi2.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_2;
393     hspi2.Init.FirstBit = SPI_FIRSTBIT_MSB;
394     hspi2.Init.TIMode = SPI_TIMODE_DISABLE;
395     hspi2.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
396     hspi2.Init.CRCPolynomial = 7;
397     if (HAL_SPI_Init(&hspi2) != HAL_OK)
398     {
399         Error_Handler();
400     }
401     /* USER CODE BEGIN SPI2_Init 2 */
402
403     /* USER CODE END SPI2_Init 2 */
404 }
405
406 /**

```

```

407 * @brief USART2 Initialization Function
408 * @param None
409 * @retval None
410 */
411 static void MX_USART2_UART_Init(void)
412 {
413     /* USER CODE BEGIN USART2_Init 0 */
414
415     /* USER CODE END USART2_Init 0 */
416
417     /* USER CODE BEGIN USART2_Init 1 */
418
419     /* USER CODE END USART2_Init 1 */
420     huart2.Instance = USART2;
421     huart2.Init.BaudRate = 9600;
422     huart2.Init.WordLength = UART_WORDLENGTH_8B;
423     huart2.Init.StopBits = UART_STOPBITS_1;
424     huart2.Init.Parity = UART_PARITY_NONE;
425     huart2.Init.Mode = UART_MODE_TX_RX;
426     huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
427     huart2.Init.OverSampling = UART_OVERSAMPLING_16;
428     huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
429     huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
430     if (HAL_UART_Init(&huart2) != HAL_OK)
431     {
432         Error_Handler();
433     }
434     /* USER CODE BEGIN USART2_Init 2 */
435
436     /* USER CODE END USART2_Init 2 */
437 }
438
439 /**
440 * @brief GPIO Initialization Function
441 * @param None
442 * @retval None
443 */
444 static void MX_GPIO_Init(void)
445 {
446     GPIO_InitTypeDef GPIO_InitStruct = {0};
447     /* USER CODE BEGIN MX_GPIO_Init_1 */
448
449     /* USER CODE END MX_GPIO_Init_1 */
450
451     /* GPIO Ports Clock Enable */
452     __HAL_RCC_GPIOC_CLK_ENABLE();
453     __HAL_RCC_GPIOA_CLK_ENABLE();
454     __HAL_RCC_GPIOB_CLK_ENABLE();
455
456     /*Configure GPIO pin Output Level */
457     HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
458
459     /*Configure GPIO pin Output Level */
460     HAL_GPIO_WritePin(GPIOC, LORA_RST_Pin | EINK_CS_Pin | EINK_RES_Pin, GPIO_PIN_SET);
461
462     /*Configure GPIO pin Output Level */
463     HAL_GPIO_WritePin(EINK_D_C_GPIO_Port, EINK_D_C_Pin, GPIO_PIN_SET);
464
465     /*Configure GPIO pin : LORA_NSS_Pin */
466     GPIO_InitStruct.Pin = LORA_NSS_Pin;
467     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
468     GPIO_InitStruct.Pull = GPIO_PULLUP;
469     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
470     HAL_GPIO_Init(LORA_NSS_GPIO_Port, &GPIO_InitStruct);
471
472     /*Configure GPIO pin : LORA_DIO0_Pin */
473     GPIO_InitStruct.Pin = LORA_DIO0_Pin;
474     GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
475     GPIO_InitStruct.Pull = GPIO_PULLDOWN;
476     HAL_GPIO_Init(LORA_DIO0_GPIO_Port, &GPIO_InitStruct);
477
478     /*Configure GPIO pins : LORA_RST_Pin EINK_CS_Pin EINK_RES_Pin */
479     GPIO_InitStruct.Pin = LORA_RST_Pin | EINK_CS_Pin | EINK_RES_Pin;
480     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;

```

```

481 GPIO_InitStruct.Pull = GPIO_PULLUP;
482 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
483 HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
484
485 /*Configure GPIO pin : EINK_D_C_Pin */
486 GPIO_InitStruct.Pin = EINK_D_C_Pin;
487 GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
488 GPIO_InitStruct.Pull = GPIO_PULLUP;
489 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
490 HAL_GPIO_Init(EINK_D_C_GPIO_Port, &GPIO_InitStruct);
491
492 /*Configure GPIO pin : EINK_BUSY_Pin */
493 GPIO_InitStruct.Pin = EINK_BUSY_Pin;
494 GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
495 GPIO_InitStruct.Pull = GPIO_NOPULL;
496 HAL_GPIO_Init(EINK_BUSY_GPIO_Port, &GPIO_InitStruct);
497
498 /* USER CODE BEGIN MX_GPIO_Init_2 */
499
500 /* USER CODE END MX_GPIO_Init_2 */
501 }
502
503 /* USER CODE BEGIN 4 */
504
505 /* USER CODE END 4 */
506
507 /**
508 * @brief This function is executed in case of error occurrence.
509 * @retval None
510 */
511 void Error_Handler(void)
512 {
513 /* USER CODE BEGIN Error_Handler_Debug */
514 /* User can add his own implementation to report the HAL error return state */
515 __disable_irq();
516 while (1)
517 {
518 }
519 /* USER CODE END Error_Handler_Debug */
520 }
521 #ifdef USE_FULL_ASSERT
522 /**
523 * @brief Reports the name of the source file and the source line number
524 * where the assert_param error has occurred.
525 * @param file: pointer to the source file name
526 * @param line: assert_param error line source number
527 * @retval None
528 */
529 void assert_failed(uint8_t *file, uint32_t line)
530 {
531 /* USER CODE BEGIN 6 */
532 /* User can add his own implementation to report the file name and line number,
533 ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
534 /* USER CODE END 6 */
535 }
536 #endif /* USE_FULL_ASSERT */
537

```

Receiver - Sources → [stm32l0xx_hal_msp.c](#)

Path: receiver/Core/Src/stm32l0xx_hal_msp.c | Lines: 370

```

1 /* USER CODE BEGIN Header */
2 /**
3 ****
4 * @file stm32l0xx_hal_msp.c
5 * @brief This file provides code for the MSP Initialization
6 * and de-Initialization codes.
7 ****
8 * @attention
9 *
10 * Copyright (c) 2026 STMicroelectronics.
11 * All rights reserved.
12 *
13 * This software is licensed under terms that can be found in the LICENSE file
14 * in the root directory of this software component.
15 * If no LICENSE file comes with this software, it is provided AS-IS.

```

```

16 *
17 ****
18 */
19 /* USER CODE END Header */
20 /* Includes ----- */
21 #include "main.h";
22 /* USER CODE BEGIN Includes */
23
24 /* USER CODE END Includes */
25
26 /* Private typedef ----- */
27 /* USER CODE BEGIN TD */
28
29 /* USER CODE END TD */
30
31 /* Private define ----- */
32 /* USER CODE BEGIN Define */
33
34 /* USER CODE END Define */
35
36 /* Private macro ----- */
37 /* USER CODE BEGIN Macro */
38
39 /* USER CODE END Macro */
40
41 /* Private variables ----- */
42 /* USER CODE BEGIN PV */
43
44 /* USER CODE END PV */
45
46 /* Private function prototypes ----- */
47 /* USER CODE BEGIN PFP */
48
49 /* USER CODE END PFP */
50
51 /* External functions ----- */
52 /* USER CODE BEGIN ExternalFunctions */
53
54 /* USER CODE END ExternalFunctions */
55
56 /* USER CODE BEGIN 0 */
57
58 /* USER CODE END 0 */
59 /**
60 * Initializes the Global MSP.
61 */
62 void HAL_MspInit(void)
63 {
64
65 /* USER CODE BEGIN MspInit 0 */
66
67 /* USER CODE END MspInit 0 */
68
69 __HAL_RCC_SYSCFG_CLK_ENABLE();
70 __HAL_RCC_PWR_CLK_ENABLE();
71
72 /* System interrupt init*/
73
74 /* USER CODE BEGIN MspInit 1 */
75
76 /* USER CODE END MspInit 1 */
77 }
78
79 /**
80 * @brief I2C MSP Initialization
81 * This function configures the hardware resources used in this example
82 * @param hi2c: I2C handle pointer
83 * @retval None
84 */
85 void HAL_I2C_MspInit(I2C_HandleTypeDef* hi2c)
86 {
87 GPIO_InitTypeDef GPIO_InitStruct = {0};
88 if(hi2c->Instance==I2C3)
89 {
90 /* USER CODE BEGIN I2C3_MspInit 0 */

```

```

91
92 /* USER CODE END I2C3_MspInit 0 */
93
94 __HAL_RCC_GPIOC_CLK_ENABLE();
95 /**I2C3 GPIO Configuration
96 PC0 -----> I2C3_SCL
97 PC1 -----> I2C3_SDA
98 */
99 GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
100 GPIO_InitStruct.Mode = GPIO_MODE_AF_OD;
101 GPIO_InitStruct.Pull = GPIO_NOPULL;
102 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
103 GPIO_InitStruct.Alternate = GPIO_AF7_I2C3;
104 HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
105
106 /* Peripheral clock enable */
107 __HAL_RCC_I2C3_CLK_ENABLE();
108 /* USER CODE BEGIN I2C3_MspInit 1 */
109
110 /* USER CODE END I2C3_MspInit 1 */
111
112 }
113
114 }
115
116 /**
117 * @brief I2C MSP De-Initialization
118 * This function freeze the hardware resources used in this example
119 * @param hi2c: I2C handle pointer
120 * @retval None
121 */
122 void HAL_I2C_MspDeInit(I2C_HandleTypeDef* hi2c)
123 {
124     if(hi2c->Instance==I2C3)
125     {
126         /* USER CODE BEGIN I2C3_MspDeInit 0 */
127
128         /* USER CODE END I2C3_MspDeInit 0 */
129         /* Peripheral clock disable */
130         __HAL_RCC_I2C3_CLK_DISABLE();
131
132         /**I2C3 GPIO Configuration
133 PC0 -----> I2C3_SCL
134 PC1 -----> I2C3_SDA
135 */
136 HAL_GPIO_DeInit(GPIOC, GPIO_PIN_0);
137
138 HAL_GPIO_DeInit(GPIOC, GPIO_PIN_1);
139
140 /* USER CODE BEGIN I2C3_MspDeInit 1 */
141
142 /* USER CODE END I2C3_MspDeInit 1 */
143 }
144
145 }
146
147 /**
148 * @brief RTC MSP Initialization
149 * This function configures the hardware resources used in this example
150 * @param hrtc: RTC handle pointer
151 * @retval None
152 */
153 void HAL_RTC_MspInit(RTC_HandleTypeDef* hrtc)
154 {
155     if(hrtc->Instance==RTC)
156     {
157         /* USER CODE BEGIN RTC_MspInit 0 */
158
159         /* USER CODE END RTC_MspInit 0 */
160         /* Peripheral clock enable */
161         __HAL_RCC_RTC_ENABLE();
162         /* USER CODE BEGIN RTC_MspInit 1 */
163
164         /* USER CODE END RTC_MspInit 1 */
165

```

```

166 }
167
168 }
169
170 /**
171  * @brief RTC MSP De-Initialization
172  * This function freeze the hardware resources used in this example
173  * @param hrtc: RTC handle pointer
174  * @retval None
175  */
176 void HAL_RTC_MspDeInit(RTC_HandleTypeDef* hrtc)
177 {
178     if(hrtc->Instance==RTC)
179     {
180         /* USER CODE BEGIN RTC_MspDeInit 0 */
181
182         /* USER CODE END RTC_MspDeInit 0 */
183         /* Peripheral clock disable */
184         __HAL_RCC_RTC_DISABLE();
185         /* USER CODE BEGIN RTC_MspDeInit 1 */
186
187         /* USER CODE END RTC_MspDeInit 1 */
188     }
189 }
190 }
191
192 /**
193  * @brief SPI MSP Initialization
194  * This function configures the hardware resources used in this example
195  * @param hspi: SPI handle pointer
196  * @retval None
197  */
198 void HAL_SPI_MspInit(SPI_HandleTypeDef* hspi)
199 {
200     GPIO_InitTypeDef GPIO_InitStruct = {0};
201     if(hspi->Instance==SPI1)
202     {
203         /* USER CODE BEGIN SPI1_MspInit 0 */
204
205         /* USER CODE END SPI1_MspInit 0 */
206         /* Peripheral clock enable */
207         __HAL_RCC_SPI1_CLK_ENABLE();
208
209         __HAL_RCC_GPIOA_CLK_ENABLE();
210         /**SPI1 GPIO Configuration
211         PA5 -----> SPI1_SCK
212         PA6 -----> SPI1_MISO
213         PA7 -----> SPI1_MOSI
214         */
215         GPIO_InitStruct.Pin = GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
216         GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
217         GPIO_InitStruct.Pull = GPIO_NOPULL;
218         GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
219         GPIO_InitStruct.Alternate = GPIO_AF0_SPI1;
220         HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
221
222         /* USER CODE BEGIN SPI1_MspInit 1 */
223
224         /* USER CODE END SPI1_MspInit 1 */
225     }
226     else if(hspi->Instance==SPI2)
227     {
228         /* USER CODE BEGIN SPI2_MspInit 0 */
229
230         /* USER CODE END SPI2_MspInit 0 */
231         /* Peripheral clock enable */
232         __HAL_RCC_SPI2_CLK_ENABLE();
233
234         __HAL_RCC_GPIOB_CLK_ENABLE();
235         /**SPI2 GPIO Configuration
236         PB13 -----> SPI2_SCK
237         PB15 -----> SPI2_MOSI
238         */
239         GPIO_InitStruct.Pin = GPIO_PIN_13|GPIO_PIN_15;
240         GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;

```

```

241 GPIO_InitStruct.Pull = GPIO_NOPULL;
242 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
243 GPIO_InitStruct.Alternate = GPIO_AF0_SPI2;
244 HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
245
246 /* USER CODE BEGIN SPI2_MspInit 1 */
247
248 /* USER CODE END SPI2_MspInit 1 */
249 }
250
251 }
252
253 /**
254  * @brief SPI MSP De-Initialization
255  * This function freeze the hardware resources used in this example
256  * @param hspi: SPI handle pointer
257  * @retval None
258  */
259 void HAL_SPI_MspDeInit(SPI_HandleTypeDef* hspi)
260 {
261     if(hspi->Instance==SPI1)
262     {
263         /* USER CODE BEGIN SPI1_MspDeInit 0 */
264
265         /* USER CODE END SPI1_MspDeInit 0 */
266         /* Peripheral clock disable */
267         __HAL_RCC_SPI1_CLK_DISABLE();
268
269         /**SPI1 GPIO Configuration
270         PA5 -----> SPI1_SCK
271         PA6 -----> SPI1_MISO
272         PA7 -----> SPI1_MOSI
273         */
274         HAL_GPIO_DeInit(GPIOA, GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7);
275
276         /* USER CODE BEGIN SPI1_MspDeInit 1 */
277
278         /* USER CODE END SPI1_MspDeInit 1 */
279     }
280     else if(hspi->Instance==SPI2)
281     {
282         /* USER CODE BEGIN SPI2_MspDeInit 0 */
283
284         /* USER CODE END SPI2_MspDeInit 0 */
285         /* Peripheral clock disable */
286         __HAL_RCC_SPI2_CLK_DISABLE();
287
288         /**SPI2 GPIO Configuration
289         PB13 -----> SPI2_SCK
290         PB15 -----> SPI2_MOSI
291         */
292         HAL_GPIO_DeInit(GPIOB, GPIO_PIN_13|GPIO_PIN_15);
293
294         /* USER CODE BEGIN SPI2_MspDeInit 1 */
295
296         /* USER CODE END SPI2_MspDeInit 1 */
297     }
298 }
299 }
300
301 /**
302  * @brief UART MSP Initialization
303  * This function configures the hardware resources used in this example
304  * @param huart: UART handle pointer
305  * @retval None
306  */
307 void HAL_UART_MspInit(UART_HandleTypeDef* huart)
308 {
309     GPIO_InitTypeDef GPIO_InitStruct = {0};
310     if(huart->Instance==USART2)
311     {
312         /* USER CODE BEGIN USART2_MspInit 0 */
313
314         /* USER CODE END USART2_MspInit 0 */
315         /* Peripheral clock enable */
316         __HAL_RCC_USART2_CLK_ENABLE();

```

```

317
318 __HAL_RCC_GPIOA_CLK_ENABLE();
319 /**USART2 GPIO Configuration
320 PA2 -----> USART2_TX
321 PA3 -----> USART2_RX
322 */
323 GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3;
324 GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
325 GPIO_InitStruct.Pull = GPIO_NOPULL;
326 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
327 GPIO_InitStruct.Alternate = GPIO_AF4_USART2;
328 HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
329
330 /* USER CODE BEGIN USART2_MspInit 1 */
331
332 /* USER CODE END USART2_MspInit 1 */
333
334 }
335
336 }
337
338 /**
339 * @brief UART MSP De-Initialization
340 * This function freeze the hardware resources used in this example
341 * @param huart: UART handle pointer
342 * @retval None
343 */
344 void HAL_UART_MspDeInit(UART_HandleTypeDef* huart)
345 {
346 if(huart->Instance==USART2)
347 {
348 /* USER CODE BEGIN USART2_MspDeInit 0 */
349
350 /* USER CODE END USART2_MspDeInit 0 */
351 /* Peripheral clock disable */
352 __HAL_RCC_USART2_CLK_DISABLE();
353
354 /**USART2 GPIO Configuration
355 PA2 -----> USART2_TX
356 PA3 -----> USART2_RX
357 */
358 HAL_GPIO_DeInit(GPIOA, GPIO_PIN_2|GPIO_PIN_3);
359
360 /* USER CODE BEGIN USART2_MspDeInit 1 */
361
362 /* USER CODE END USART2_MspDeInit 1 */
363 }
364
365 }
366
367 /* USER CODE BEGIN 1 */
368
369 /* USER CODE END 1 */
370

```

Receiver - Sources → [stm32l0xx_it.c](#)

Path: receiver/Core/Src/stm32l0xx_it.c | Lines: 151

```

1 /* USER CODE BEGIN Header */
2 /**
3 *****
4 * @file stm32l0xx_it.c
5 * @brief Interrupt Service Routines.
6 *****
7 * @attention
8 *
9 * Copyright (c) 2026 STMicroelectronics.
10 * All rights reserved.
11 *
12 * This software is licensed under terms that can be found in the LICENSE file
13 * in the root directory of this software component.
14 * If no LICENSE file comes with this software, it is provided AS-IS.
15 *
16 *****
17 */

```

```

18 /* USER CODE END Header */
19
20 /* Includes -----*/
21 #include <main.h>;
22 #include <stm32l0xx_it.h>;
23 /* Private includes -----*/
24 /* USER CODE BEGIN Includes */
25 /* USER CODE END Includes */
26
27 /* Private typedef -----*/
28 /* USER CODE BEGIN TD */
29
30 /* USER CODE END TD */
31
32 /* Private define -----*/
33 /* USER CODE BEGIN PD */
34
35 /* USER CODE END PD */
36
37 /* Private macro -----*/
38 /* USER CODE BEGIN PM */
39
40 /* USER CODE END PM */
41
42 /* Private variables -----*/
43 /* USER CODE BEGIN PV */
44
45 /* USER CODE END PV */
46
47 /* Private function prototypes -----*/
48 /* USER CODE BEGIN PFP */
49
50 /* USER CODE END PFP */
51
52 /* Private user code -----*/
53 /* USER CODE BEGIN 0 */
54
55 /* USER CODE END 0 */
56
57 /* External variables -----*/
58
59 /* USER CODE BEGIN EV */
60
61 /* USER CODE END EV */
62
63 /*****
64  * Cortex-M0+ Processor Interruption and Exception Handlers */
65  *****/
66 /**
67  * @brief This function handles Non maskable Interrupt.
68  */
69 void NMI_Handler(void)
70 {
71 /* USER CODE BEGIN NonMaskableInt_IRQn 0 */
72
73 /* USER CODE END NonMaskableInt_IRQn 0 */
74 /* USER CODE BEGIN NonMaskableInt_IRQn 1 */
75 while (1)
76 {
77 }
78 /* USER CODE END NonMaskableInt_IRQn 1 */
79 }
80
81 /**
82  * @brief This function handles Hard fault interrupt.
83  */
84 void HardFault_Handler(void)
85 {
86 /* USER CODE BEGIN HardFault_IRQn 0 */
87
88 /* USER CODE END HardFault_IRQn 0 */
89 while (1)
90 {
91 /* USER CODE BEGIN W1_HardFault_IRQn 0 */
92 /* USER CODE END W1_HardFault_IRQn 0 */

```

```

93 }
94 }
95
96 /**
97 * @brief This function handles System service call via SWI instruction.
98 */
99 void SVC_Handler(void)
100 {
101 /* USER CODE BEGIN SVC_IRQn 0 */
102
103 /* USER CODE END SVC_IRQn 0 */
104 /* USER CODE BEGIN SVC_IRQn 1 */
105
106 /* USER CODE END SVC_IRQn 1 */
107 }
108
109 /**
110 * @brief This function handles Pendable request for system service.
111 */
112 void PendSV_Handler(void)
113 {
114 /* USER CODE BEGIN PendSV_IRQn 0 */
115
116 /* USER CODE END PendSV_IRQn 0 */
117 /* USER CODE BEGIN PendSV_IRQn 1 */
118
119 /* USER CODE END PendSV_IRQn 1 */
120 }
121
122 /**
123 * @brief This function handles System tick timer.
124 */
125 void SysTick_Handler(void)
126 {
127 /* USER CODE BEGIN SysTick_IRQn 0 */
128
129 /* USER CODE END SysTick_IRQn 0 */
130 HAL_IncTick();
131 /* USER CODE BEGIN SysTick_IRQn 1 */
132
133 /* USER CODE END SysTick_IRQn 1 */
134 }
135
136 /*****
137  * STM32L0xx Peripheral Interrupt Handlers */
138  * Add here the Interrupt Handlers for the used peripherals. */
139  * For the available peripheral interrupt handler names, */
140  * please refer to the startup file (startup_stm32l0xx.s). */
141 *****/
142
143 /* USER CODE BEGIN 1 */
144
145 void EXTI4_15_IRQHandler(void)
146 {
147 HAL_GPIO_EXTI_IRQHandler(LORA_DI00_Pin);
148 }
149
150 /* USER CODE END 1 */
151

```

Receiver - Sources → [sx1278.c](#)

Path: receiver/Core/Src/sx1278.c | Lines: 255

```

1 #include <sx1278.h>;
2
3 extern SPI_HandleTypeDef hspi1;
4
5 volatile bool sx1278_tx_done = false;
6 volatile bool sx1278_rx_done = false;
7
8 static void SX1278_CS_LOW(void)
9 {
10 HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_RESET);
11 }
12

```

```

13 static void SX1278_CS_HIGH(void)
14 {
15     HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
16 }
17
18 static void spi_wait_ready(void)
19 {
20     while (HAL_SPI_GetState(&hspi1) != HAL_SPI_STATE_READY);
21 }
22
23 static uint8_t SX1278_ReadReg(uint8_t reg)
24 {
25     uint8_t addr = reg & 0x7F;
26     uint8_t val;
27     SX1278_CS_LOW();
28     HAL_SPI_Transmit(&hspi1, &addr, 1, 100);
29     spi_wait_ready();
30     HAL_SPI_Receive(&hspi1, &val, 1, 100);
31     spi_wait_ready();
32     SX1278_CS_HIGH();
33     return val;
34 }
35
36 static void SX1278_WriteReg(uint8_t reg, uint8_t val)
37 {
38     uint8_t buf[2];
39     buf[0] = reg | 0x80;
40     buf[1] = val;
41     SX1278_CS_LOW();
42     HAL_SPI_Transmit(&hspi1, buf, 2, 100);
43     spi_wait_ready();
44     SX1278_CS_HIGH();
45 }
46
47 static void SX1278_WriteBurst(uint8_t reg, uint8_t *data, uint8_t len)
48 {
49     uint8_t addr = reg | 0x80;
50     SX1278_CS_LOW();
51     HAL_SPI_Transmit(&hspi1, &addr, 1, 100);
52     spi_wait_ready();
53     HAL_SPI_Transmit(&hspi1, data, len, 100);
54     spi_wait_ready();
55     SX1278_CS_HIGH();
56 }
57
58 void SX1278_HardReset(void)
59 {
60     HAL_GPIO_WritePin(LORA_RST_GPIO_Port, LORA_RST_Pin, GPIO_PIN_RESET);
61     HAL_Delay(1);
62     HAL_GPIO_WritePin(LORA_RST_GPIO_Port, LORA_RST_Pin, GPIO_PIN_SET);
63     HAL_Delay(100);
64 }
65
66 static bool SX1278_CheckVersion(void)
67 {
68     uint8_t ver = SX1278_ReadReg(SX1278_REG_VERSION);
69     return (ver == 0x12);
70 }
71
72 static bool SX1278_WaitMode(uint8_t expected_mode)
73 {
74     uint32_t timeout = HAL_GetTick() + 100;
75     while (1) {
76         uint8_t current = SX1278_ReadReg(SX1278_REG_OP_MODE);
77         if ((current & 0x07) == expected_mode) return true;
78         if (HAL_GetTick() >= timeout) return false;
79     }
80 }
81
82 bool SX1278_Init(void)
83 {
84     SX1278_HardReset();
85
86     bool ok = SX1278_CheckVersion();
87     if (!ok) return false;

```

```

88
89 SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_SLEEP);
90 HAL_Delay(100);
91
92 SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_STDBY);
93 SX1278_WaitMode(SX1278_MODE_STDBY);
94
95 uint64_t frf = ((uint64_t)433000000 << 19) / SX1278_FXOSC;
96 SX1278_WriteReg(SX1278_REG_FRF_MSB, (frf >> 16) & 0xFF);
97 SX1278_WriteReg(SX1278_REG_FRF_MID, (frf >> 8) & 0xFF);
98 SX1278_WriteReg(SX1278_REG_FRF_LSB, frf & 0xFF);
99
100 SX1278_WriteReg(SX1278_REG_PA_CONFIG, 0x8F);
101 SX1278_WriteReg(SX1278_REG_PA_RAMP, 0x09);
102 SX1278_WriteReg(SX1278_REG_OCP, 0x0B);
103 SX1278_WriteReg(SX1278_REG_LNA, 0x23);
104 SX1278_WriteReg(SX1278_REG_PA_DAC, 0x84);
105
106 SX1278_WriteReg(SX1278_REG_MODEM_CONFIG_1, 0x72);
107 SX1278_WriteReg(SX1278_REG_MODEM_CONFIG_2, 0x94);
108 SX1278_WriteReg(SX1278_REG_MODEM_CONFIG_3, 0x04);
109
110 SX1278_WriteReg(SX1278_REG_SYMB_TIMEOUT_LSB, 0xFF);
111 SX1278_WriteReg(SX1278_REG_PREAMBLE_MSB, 0x00);
112 SX1278_WriteReg(SX1278_REG_PREAMBLE_LSB, 0x08);
113 SX1278_WriteReg(SX1278_REG_SYNC_WORD, 0x12);
114
115 SX1278_WriteReg(SX1278_REG_FIFO_TX_BASE_ADDR, 0x00);
116 SX1278_WriteReg(SX1278_REG_FIFO_RX_BASE_ADDR, 0x00);
117
118 SX1278_WriteReg(SX1278_REG_DIO_MAPPING_1, SX1278_DIO0_RX_DONE);
119
120 SX1278_WriteReg(SX1278_REG_IRQ_FLAGS_MASK, 0xF7);
121
122 return true;
123 }
124
125 bool SX1278_SendPacket(uint8_t *data, uint8_t len)
126 {
127     if (len > SX1278_MAX_PACKET_SIZE) return false;
128
129     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_STDBY);
130     SX1278_WaitMode(SX1278_MODE_STDBY);
131
132     SX1278_WriteReg(SX1278_REG_FIFO_ADDR_PTR, 0x00);
133     SX1278_WriteReg(SX1278_REG_PAYLOAD_LENGTH, len);
134     SX1278_WriteBurst(SX1278_REG_FIFO, data, len);
135
136     sx1278_tx_done = false;
137
138     SX1278_WriteReg(SX1278_REG_IRQ_FLAGS, 0xFF);
139     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_TX);
140
141     uint32_t timeout = HAL_GetTick() + 5000;
142     while (!sx1278_tx_done)
143     {
144         if (HAL_GetTick() > timeout)
145         {
146             SX1278_HardReset();
147             SX1278_Init();
148             return false;
149         }
150     }
151
152     SX1278_WriteReg(SX1278_REG_IRQ_FLAGS, 0xFF);
153     return true;
154 }
155
156 void SX1278_EnterSleep(void)
157 {
158     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_SLEEP);
159 }
160
161 void SX1278_SetStandby(void)
162 {

```

```

163 SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_STDBY);
164 }
165
166 void SX1278_StartRX(void)
167 {
168     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_STDBY);
169     SX1278_WaitMode(SX1278_MODE_STDBY);
170
171     SX1278_WriteReg(SX1278_REG_DIO_MAPPING_1, SX1278_DIO0_RX_DONE);
172
173     SX1278_WriteReg(SX1278_REG_IRQ_FLAGS, 0xFF);
174     SX1278_WriteReg(SX1278_REG_FIFO_RX_BASE_ADDR, 0x00);
175
176     sx1278_rx_done = false;
177
178     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_RX_CONTINUOUS);
179 }
180
181 bool SX1278_ReadPacket(uint8_t *data, uint8_t *len)
182 {
183     uint8_t irq = SX1278_ReadReg(SX1278_REG_IRQ_FLAGS);
184
185     if (!(irq & SX1278_IRQ_RX_DONE_MASK))
186     {
187         return false;
188     }
189
190     SX1278_WriteReg(SX1278_REG_IRQ_FLAGS, 0xFF);
191
192     if (irq & SX1278_IRQ_PAYLOAD_CRC_ERROR_MASK)
193     {
194         return false;
195     }
196
197     uint8_t nb_bytes = SX1278_ReadReg(SX1278_REG_RX_NB_BYTES);
198     if (nb_bytes == 0 || nb_bytes > SX1278_MAX_PACKET_SIZE)
199     {
200         return false;
201     }
202
203     SX1278_WriteReg(SX1278_REG_FIFO_ADDR_PTR,
204     SX1278_ReadReg(SX1278_REG_FIFO_RX_CURRENT_ADDR));
205
206     uint8_t addr = SX1278_REG_FIFO;
207     SX1278_CS_LOW();
208     HAL_SPI_Transmit(&hspi1, &addr, 1, 100);
209     spi_wait_ready();
210     HAL_SPI_Receive(&hspi1, data, nb_bytes, 100);
211     spi_wait_ready();
212     SX1278_CS_HIGH();
213
214     *len = nb_bytes;
215     return true;
216 }
217
218 int SX1278_GetLastRSSI(void)
219 {
220     int rssi = (int)SX1278_ReadReg(SX1278_REG_PKT_RSSI_VALUE);
221     return rssi - 157;
222 }
223
224 int SX1278_GetLastSNR(void)
225 {
226     int snr = (int_t)SX1278_ReadReg(SX1278_REG_PKT_SNR_VALUE);
227     return snr / 4;
228 }
229
230 uint8_t SX1278_ReadIrqFlags(void)
231 {
232     return SX1278_ReadReg(SX1278_REG_IRQ_FLAGS);
233 }
234
235 void SX1278_ClearIRQ(void)
236 {
237     SX1278_WriteReg(SX1278_REG_IRQ_FLAGS, 0xFF);

```

```

238 }
239
240 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
241 {
242     if (GPIO_Pin == LORA_DIO0_Pin)
243     {
244         uint8_t irq = SX1278_ReadReg(SX1278_REG_IRQ_FLAGS);
245         if (irq & SX1278_IRQ_TX_DONE_MASK)
246         {
247             sx1278_tx_done = true;
248         }
249         if (irq & SX1278_IRQ_RX_DONE_MASK)
250         {
251             sx1278_rx_done = true;
252         }
253     }
254 }
255

```

Receiver - Sources → [syscalls.c](#)

Path: receiver/Core/Src/syscalls.c | Lines: 245

```

1  /**
2  * *****
3  * @file syscalls.c
4  * @author Auto-generated by STM32CubeMX
5  * @brief Minimal System calls file
6  *
7  * For more information about which c-functions
8  * need which of these lowlevel functions
9  * please consult the Newlib or Picolibc libc-manual
10 * *****
11 * @attention
12 *
13 * Copyright (c) 2020-2025 STMicroelectronics.
14 * All rights reserved.
15 *
16 * This software is licensed under terms that can be found in the LICENSE file
17 * in the root directory of this software component.
18 * If no LICENSE file comes with this software, it is provided AS-IS.
19 *
20 * *****
21 */
22
23 /* Includes */
24 #include <sys/stat.h>;
25 #include <stdlib.h>;
26 #include <errno.h>;
27 #include <stdio.h>;
28 #include <signal.h>;
29 #include <time.h>;
30 #include <sys/time.h>;
31 #include <sys/times.h>;
32
33
34 /* Variables */
35 extern int __io_putchar(int ch) __attribute__((weak));
36 extern int __io_getchar(void) __attribute__((weak));
37
38
39 char *__env[1] = { 0 };
40 char **environ = __env;
41
42
43 /* Functions */
44 void initialise_monitor_handles()
45 {
46 }
47
48 int _getpid(void)
49 {
50     return 1;
51 }
52
53 int _kill(int pid, int sig)

```

```

54 {
55 (void)pid;
56 (void)sig;
57 errno = EINVAL;
58 return -1;
59 }
60
61 void _exit (int status)
62 {
63 _kill(status, -1);
64 while (1) {} /* Make sure we hang here */
65 }
66
67 __attribute__((weak)) int _read(int file, char *ptr, int len)
68 {
69 (void)file;
70 int DataIdx;
71
72 for (DataIdx = 0; DataIdx < len; DataIdx++)
73 {
74 *ptr++ = __io_getchar();
75 }
76
77 return len;
78 }
79
80 __attribute__((weak)) int _write(int file, char *ptr, int len)
81 {
82 (void)file;
83 int DataIdx;
84
85 for (DataIdx = 0; DataIdx < len; DataIdx++)
86 {
87 __io_putchar(*ptr++);
88 }
89 return len;
90 }
91
92 int _close(int file)
93 {
94 (void)file;
95 return -1;
96 }
97
98
99 int _fstat(int file, struct stat *st)
100 {
101 (void)file;
102 st->st_mode = S_IFCHR;
103 return 0;
104 }
105
106 int _isatty(int file)
107 {
108 (void)file;
109 return 1;
110 }
111
112 int _lseek(int file, int ptr, int dir)
113 {
114 (void)file;
115 (void)ptr;
116 (void)dir;
117 return 0;
118 }
119
120 int _open(char *path, int flags, ...)
121 {
122 (void)path;
123 (void)flags;
124 /* Pretend like we always fail */
125 return -1;
126 }
127
128 int _wait(int *status)

```

```

129 {
130 (void)status;
131 errno = ECHILD;
132 return -1;
133 }
134
135 int _unlink(char *name)
136 {
137 (void)name;
138 errno = ENOENT;
139 return -1;
140 }
141
142 clock_t _times(struct tms *buf)
143 {
144 (void)buf;
145 return -1;
146 }
147
148 int _stat(const char *file, struct stat *st)
149 {
150 (void)file;
151 st->st_mode = S_IFCHR;
152 return 0;
153 }
154
155 int _link(char *old, char *new)
156 {
157 (void)old;
158 (void)new;
159 errno = EMLINK;
160 return -1;
161 }
162
163 int _fork(void)
164 {
165 errno = EAGAIN;
166 return -1;
167 }
168
169 int _execve(char *name, char **argv, char **env)
170 {
171 (void)name;
172 (void)argv;
173 (void)env;
174 errno = ENOMEM;
175 return -1;
176 }
177
178 // --- Picolibc Specific Section ---
179 #if defined(__PICOLIBC__)
180
181 /**
182 * @brief Picolibc helper function to output a character to a FILE stream.
183 * This redirects the output to the low-level __io_putchar function.
184 * @param c Character to write.
185 * @param file FILE stream pointer (ignored).
186 * @retval int The character written.
187 */
188 static int starm_putc(char c, FILE *file)
189 {
190 (void) file;
191 __io_putchar(c);
192 return c;
193 }
194
195 /**
196 * @brief Picolibc helper function to input a character from a FILE stream.
197 * This redirects the input from the low-level __io_getchar function.
198 * @param file FILE stream pointer (ignored).
199 * @retval int The character read, cast to an unsigned char then int.
200 */
201 static int starm_getc(FILE *file)
202 {
203 unsigned char c;

```

```

204 (void) file;
205 c = __io_getchar();
206 return c;
207 }
208
209 // Define and initialize the standard I/O streams for Picolibc.
210 // FDEV_SETUP_STREAM connects the starm_putc and starm_getc helper functions to a FILE structure.
211 // _FDEV_SETUP_RW indicates the stream is for reading and writing.
212 static FILE __stdio = FDEV_SETUP_STREAM(starm_putc,
213 starm_getc,
214 NULL,
215 _FDEV_SETUP_RW);
216
217 // Assign the standard stream pointers (stdin, stdout, stderr) to the initialized stream.
218 // Picolibc uses these pointers for standard I/O operations (printf, scanf, etc.).
219 FILE *const stdin = &__stdio;
220 __strong_reference(stdin, stdout);
221 __strong_reference(stdin, stderr);
222
223 // Create strong aliases mapping standard C library function names (without underscore)
224 // to the implemented system call stubs (with underscore). Picolibc uses these
225 // standard names internally, so this linking is required.
226 __strong_reference(_read, read);
227 __strong_reference(_write, write);
228 __strong_reference(_times, times);
229 __strong_reference(_execve, execve);
230 __strong_reference(_fork, fork);
231 __strong_reference(_link, link);
232 __strong_reference(_unlink, unlink);
233 __strong_reference(_stat, stat);
234 __strong_reference(_wait, wait);
235 __strong_reference(_open, open);
236 __strong_reference(_close, close);
237 __strong_reference(_lseek, lseek);
238 __strong_reference(_isatty, isatty);
239 __strong_reference(_fstat, fstat);
240 __strong_reference(_exit, exit);
241 __strong_reference(_kill, kill);
242 __strong_reference(_getpid, getpid);
243
244 #endif // __PICOLIBC__
245

```

Receiver - Sources → [systemem.c](#)

Path: receiver/Core/Src/systemem.c | Lines: 88

```

1 /**
2 *****
3 * @file systemem.c
4 * @author Generated by STM32CubeMX
5 * @brief System Memory calls file
6 *
7 * For more information about which C functions
8 * need which of these lowlevel functions
9 * please consult the Newlib or Picolibc libc manual
10 *****
11 * @attention
12 *
13 * Copyright (c) 2025 STMicroelectronics.
14 * All rights reserved.
15 *
16 * This software is licensed under terms that can be found in the LICENSE file
17 * in the root directory of this software component.
18 * If no LICENSE file comes with this software, it is provided AS-IS.
19 *
20 *****
21 */
22
23 /* Includes */
24 #include <errno.h>;
25 #include <stdint.h>;
26 #include <stddef.h>;
27
28 /**
29 * Pointer to the current high watermark of the heap usage

```

```

30 */
31 static uint8_t *__sbrk_heap_end = NULL;
32
33 /**
34 * @brief _sbrk() allocates memory to the newlib heap and is used by malloc
35 * and others from the C library
36 *
37 * @verbatim
38 * #####
39 * # .data # .bss # newlib heap # MSP stack #
40 * # # # # Reserved by _Min_Stack_Size #
41 * #####
42 * ^-- RAM start ^-- _end _estack, RAM end --^
43 * @endverbatim
44 *
45 * This implementation starts allocating at the '_end' linker symbol
46 * The '_Min_Stack_Size' linker symbol reserves a memory for the MSP stack
47 * The implementation considers '_estack' linker symbol to be RAM end
48 * NOTE: If the MSP stack, at any point during execution, grows larger than the
49 * reserved size, please increase the '_Min_Stack_Size'.
50 *
51 * @param incr Memory size
52 * @return Pointer to allocated memory
53 */
54 void *_sbrk(ptrdiff_t incr)
55 {
56 extern uint8_t _end; /* Symbol defined in the linker script */
57 extern uint8_t _estack; /* Symbol defined in the linker script */
58 extern uint32_t _Min_Stack_Size; /* Symbol defined in the linker script */
59 const uint32_t stack_limit = (uint32_t)&_estack - (uint32_t)&_Min_Stack_Size;
60 const uint8_t *max_heap = (uint8_t *)stack_limit;
61 uint8_t *prev_heap_end;
62
63 /* Initialize heap end at first call */
64 if (NULL == __sbrk_heap_end)
65 {
66 __sbrk_heap_end = &_end;
67 }
68
69 /* Protect heap from growing into the reserved MSP stack */
70 if (__sbrk_heap_end + incr > max_heap)
71 {
72 errno = ENOMEM;
73 return (void *)-1;
74 }
75
76 prev_heap_end = __sbrk_heap_end;
77 __sbrk_heap_end += incr;
78
79 return (void *)prev_heap_end;
80 }
81
82 #if defined(__PICOLIBC__)
83 // Picolibc expects syscalls without the leading underscore.
84 // This creates a strong alias so that
85 // calls to `sbrk()` are resolved to our `_sbrk()` implementation.
86 __strong_reference(_sbrk, sbrk);
87 #endif
88

```

Receiver - Sources → [system_stm32l0xx.c](#)

Path: receiver/Core/Src/system_stm32l0xx.c | Lines: 274

```

1 /**
2 *****
3 * @file system_stm32l0xx.c
4 * @author MCD Application Team
5 * @brief CMSIS Cortex-M0+ Device Peripheral Access Layer System Source File.
6 *
7 * This file provides two functions and one global variable to be called from
8 * user application:
9 * - SystemInit(): This function is called at startup just after reset and
10 * before branch to main program. This call is made inside
11 * the "startup_stm32l0xx.s" file.
12 *

```

```

13 * - SystemCoreClock variable: Contains the core clock (HCLK), it can be used
14 * by the user application to setup the SysTick
15 * timer or configure other parameters.
16 *
17 * - SystemCoreClockUpdate(): Updates the variable SystemCoreClock and must
18 * be called whenever the core clock is changed
19 * during program execution.
20 *
21 *
22 *****
23 * @attention
24 *
25 * Copyright (c) 2016 STMicroelectronics.
26 * All rights reserved.
27 *
28 * This software is licensed under terms that can be found in the LICENSE file
29 * in the root directory of this software component.
30 * If no LICENSE file comes with this software, it is provided AS-IS.
31 *
32 *****
33 */
34
35 /** @addtogroup CMSIS
36 * @{
37 */
38
39 /** @addtogroup stm32l0xx_system
40 * @{
41 */
42
43 /** @addtogroup STM32L0xx_System_Private_Includes
44 * @{
45 */
46
47 #include <stm32l0xx.h>
48
49 #if !defined (HSE_VALUE)
50 #define HSE_VALUE ((uint32_t)8000000U) /*!< Value of the External oscillator in Hz */
51 #endif /* HSE_VALUE */
52
53 #if !defined (MSI_VALUE)
54 #define MSI_VALUE ((uint32_t)2097152U) /*!< Value of the Internal oscillator in Hz*/
55 #endif /* MSI_VALUE */
56
57 #if !defined (HSI_VALUE)
58 #define HSI_VALUE ((uint32_t)16000000U) /*!< Value of the Internal oscillator in Hz*/
59 #endif /* HSI_VALUE */
60
61
62 /**
63 * @}
64 */
65
66 /** @addtogroup STM32L0xx_System_Private_TypeDefinitions
67 * @{
68 */
69
70 /**
71 * @}
72 */
73
74 /** @addtogroup STM32L0xx_System_Private_Defines
75 * @{
76 */
77 /***** Miscellaneous Configuration *****/
78
79 /* Note: Following vector table addresses must be defined in line with linker
80 configuration. */
81 /*!< Uncomment the following line if you need to relocate the vector table
82 anywhere in Flash or Sram, else the vector table is kept at the automatic
83 remap of boot address selected */
84 /* #define USER_VECT_TAB_ADDRESS */
85
86 #if defined(USER_VECT_TAB_ADDRESS)
87 /*!< Uncomment the following line if you need to relocate your vector Table

```

```

88 in Sram else user remap will be done in Flash. */
89 /* #define VECT_TAB_SRAM */
90 #if defined(VECT_TAB_SRAM)
91 #define VECT_TAB_BASE_ADDRESS SRAM_BASE /*!& Vector Table base address field.
92 This value must be a multiple of 0x200. */
93 #else
94 #define VECT_TAB_BASE_ADDRESS FLASH_BASE /*!& Vector Table base address field.
95 This value must be a multiple of 0x200. */
96 #endif /* VECT_TAB_SRAM */
97 #if !defined(VECT_TAB_OFFSET)
98 #define VECT_TAB_OFFSET 0x00000000U /*!& Vector Table offset field.
99 This value must be a multiple of 0x200. */
100 #endif /* VECT_TAB_OFFSET */
101 #endif /* USER_VECT_TAB_ADDRESS */
102
103 /*****
104 **
105 * @}
106 */
107
108 /** @addtogroup STM32L0xx_System_Private_Macros
109 * @{
110 */
111
112 /**
113 * @}
114 */
115
116 /** @addtogroup STM32L0xx_System_Private_Variables
117 * @{
118 */
119 /* This variable is updated in three ways:
120 1) by calling CMSIS function SystemCoreClockUpdate()
121 2) by calling HAL API function HAL_RCC_GetHCLKFreq()
122 3) each time HAL_RCC_ClockConfig() is called to configure the system clock frequency
123 Note: If you use this function to configure the system clock; then there
124 is no need to call the 2 first functions listed above, since SystemCoreClock
125 variable is updated automatically.
126 */
127 uint32_t SystemCoreClock = 2097152U; /* 32.768 kHz * 2^6 */
128 const uint8_t AHBPrescTable[16] = {0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U, 6U, 7U, 8U, 9U};
129 const uint8_t APBPrescTable[8] = {0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U};
130 const uint8_t PLLMulTable[9] = {3U, 4U, 6U, 8U, 12U, 16U, 24U, 32U, 48U};
131
132 /**
133 * @}
134 */
135
136 /** @addtogroup STM32L0xx_System_Private_FunctionPrototypes
137 * @{
138 */
139
140 /**
141 * @}
142 */
143
144 /** @addtogroup STM32L0xx_System_Private_Functions
145 * @{
146 */
147
148 /**
149 * @brief Setup the microcontroller system.
150 * @param None
151 * @retval None
152 */
153 void SystemInit (void)
154 {
155 /* Configure the Vector Table location add offset address -----*/
156 #if defined (USER_VECT_TAB_ADDRESS)
157 SCB->VTOR = VECT_TAB_BASE_ADDRESS | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal SRAM */
158 #endif /* USER_VECT_TAB_ADDRESS */
159 }
160
161 /**
162 * @brief Update SystemCoreClock variable according to Clock Register Values.

```

```

163 * The SystemCoreClock variable contains the core clock (HCLK), it can
164 * be used by the user application to setup the SysTick timer or configure
165 * other parameters.
166 *
167 * @note Each time the core clock (HCLK) changes, this function must be called
168 * to update SystemCoreClock variable value. Otherwise, any configuration
169 * based on this variable will be incorrect.
170 *
171 * @note - The system frequency computed by this function is not the real
172 * frequency in the chip. It is calculated based on the predefined
173 * constant and the selected clock source:
174 *
175 * - If SYSCLK source is MSI, SystemCoreClock will contain the MSI
176 * value as defined by the MSI range.
177 *
178 * - If SYSCLK source is HSI, SystemCoreClock will contain the HSI_VALUE(*)
179 *
180 * - If SYSCLK source is HSE, SystemCoreClock will contain the HSE_VALUE(**)
181 *
182 * - If SYSCLK source is PLL, SystemCoreClock will contain the HSE_VALUE(**)
183 * or HSI_VALUE(*) multiplied/divided by the PLL factors.
184 *
185 * (*) HSI_VALUE is a constant defined in stm32l0xx_hal.h file (default value
186 * 16 MHz) but the real value may vary depending on the variations
187 * in voltage and temperature.
188 *
189 * (**) HSE_VALUE is a constant defined in stm32l0xx_hal.h file (default value
190 * 8 MHz), user has to ensure that HSE_VALUE is same as the real
191 * frequency of the crystal used. Otherwise, this function may
192 * have wrong result.
193 *
194 * - The result of this function could be not correct when using fractional
195 * value for HSE crystal.
196 * @param None
197 * @retval None
198 */
199 void SystemCoreClockUpdate (void)
200 {
201     uint32_t tmp = 0U, pllmul = 0U, plldiv = 0U, pllsource = 0U, msirange = 0U;
202
203     /* Get SYSCLK source -----*/
204     tmp = RCC->CFGR & RCC_CFGR_SWS;
205
206     switch (tmp)
207     {
208         case 0x00U: /* MSI used as system clock */
209             msirange = (RCC->ICSCR & RCC_ICSCR_MSIRANGE) >> RCC_ICSCR_MSIRANGE_Pos;
210             SystemCoreClock = (32768U * (1U << (msirange + 1U)));
211             break;
212         case 0x04U: /* HSI used as system clock */
213             if ((RCC->CR & RCC_CR_HSIDIVF) != 0U)
214             {
215                 SystemCoreClock = HSI_VALUE / 4U;
216             }
217             else
218             {
219                 SystemCoreClock = HSI_VALUE;
220             }
221             break;
222         case 0x08U: /* HSE used as system clock */
223             SystemCoreClock = HSE_VALUE;
224             break;
225         default: /* PLL used as system clock */
226             /* Get PLL clock source and multiplication factor -----*/
227             pllmul = RCC->CFGR & RCC_CFGR_PLLMUL;
228             plldiv = RCC->CFGR & RCC_CFGR_PLLDIV;
229             pllmul = PLLMulTable[(pllmul >> RCC_CFGR_PLLMUL_Pos)];
230             plldiv = (plldiv >> RCC_CFGR_PLLDIV_Pos) + 1U;
231
232             pllsource = RCC->CFGR & RCC_CFGR_PLLSRC;
233
234             if (pllsource == 0x00U)
235             {
236                 /* HSI oscillator clock selected as PLL clock entry */
237                 if ((RCC->CR & RCC_CR_HSIDIVF) != 0U)

```

```

238 {
239 SystemCoreClock = (((HSI_VALUE / 4U) * pllmul) / plldiv);
240 }
241 else
242 {
243 SystemCoreClock = (((HSI_VALUE) * pllmul) / plldiv);
244 }
245 }
246 else
247 {
248 /* HSE selected as PLL clock entry */
249 SystemCoreClock = (((HSE_VALUE) * pllmul) / plldiv);
250 }
251 break;
252 }
253 /* Compute HCLK clock frequency ----- */
254 /* Get HCLK prescaler */
255 tmp = AHBPrescTable[((RCC->CFGR & RCC_CFGR_HPRE) >> RCC_CFGR_HPRE_Pos)];
256 /* HCLK clock frequency */
257 SystemCoreClock >= tmp;
258 }
259
260
261
262 /**
263 * @}
264 */
265
266 /**
267 * @}
268 */
269
270 /**
271 * @}
272 */
273
274

```

Sensor - Headers → [bme280.h](#)

Path: sensor/Core/Inc/bme280.h | Lines: 87

```

1 #ifndef __BME280_H
2 #define __BME280_H
3
4 #include <main.h>;
5 #include <stdint.h>;
6 #include <stdbool.h>;
7
8 #define BME280_I2C_ADDR_PRIM 0x76
9 #define BME280_I2C_ADDR_SEC 0x77
10
11 #define BME280_REG_CHIP_ID 0xD0
12 #define BME280_REG_RESET 0xE0
13 #define BME280_REG_CTRL_HUM 0xF2
14 #define BME280_REG_STATUS 0xF3
15 #define BME280_REG_CTRL_MEAS 0xF4
16 #define BME280_REG_CONFIG 0xF5
17 #define BME280_REG_PRESS_MSB 0xF7
18 #define BME280_REG_TEMP_MSB 0xFA
19 #define BME280_REG_HUM_MSB 0xFD
20
21 #define BME280_CHIP_ID 0x60
22
23 #define BME280_RESET_CMD 0xB6
24
25 #define BME280_OVERSAMPLING_SKIP 0
26 #define BME280_OVERSAMPLING_1 1
27 #define BME280_OVERSAMPLING_2 2
28 #define BME280_OVERSAMPLING_4 3
29 #define BME280_OVERSAMPLING_8 4
30 #define BME280_OVERSAMPLING_16 5
31
32 #define BME280_MODE_SLEEP 0
33 #define BME280_MODE_FORCED 1
34 #define BME280_MODE_NORMAL 3

```

```

35
36 #define BME280_STANDBY_0_5 0
37 #define BME280_STANDBY_62_5 1
38 #define BME280_STANDBY_125 2
39 #define BME280_STANDBY_250 3
40 #define BME280_STANDBY_500 4
41 #define BME280_STANDBY_1000 5
42 #define BME280_STANDBY_10 6
43 #define BME280_STANDBY_20 7
44
45 #define BME280_FILTER_OFF 0
46 #define BME280_FILTER_2 1
47 #define BME280_FILTER_4 2
48 #define BME280_FILTER_8 3
49 #define BME280_FILTER_16 4
50
51 typedef struct {
52     uint8_t addr;
53     I2C_HandleTypeDef *hi2c;
54
55     uint16_t dig_T1;
56     int16_t dig_T2;
57     int16_t dig_T3;
58     uint16_t dig_P1;
59     int16_t dig_P2;
60     int16_t dig_P3;
61     int16_t dig_P4;
62     int16_t dig_P5;
63     int16_t dig_P6;
64     int16_t dig_P7;
65     int16_t dig_P8;
66     int16_t dig_P9;
67     uint8_t dig_H1;
68     int16_t dig_H2;
69     uint8_t dig_H3;
70     int16_t dig_H4;
71     int16_t dig_H5;
72     int8_t dig_H6;
73
74     int32_t t_fine;
75 } BME280_HandleTypeDef;
76
77 typedef struct {
78     float temperature;
79     float humidity;
80     float pressure;
81 } BME280_Data;
82
83 bool BME280_Init(BME280_HandleTypeDef *bme, I2C_HandleTypeDef *hi2c, uint8_t addr);
84 bool BME280_ReadAll(BME280_HandleTypeDef *bme, BME280_Data *data);
85
86 #endif
87

```

Sensor - Headers → [cmd.h](#)

Path: sensor/Core/Inc/cmd.h | Lines: 20

```

1 #ifndef __CMD_H
2 #define __CMD_H
3
4 #include <main.h>;
5 #include <sx1278.h>;
6 #include <stdint.h>;
7
8 extern SX1278_Config g_lora_cfg;
9 extern uint32_t g_interval_ms;
10 extern float g_offset_temp;
11 extern float g_offset_hum;
12 extern float g_offset_pres;
13 extern bool g_config_dirty;
14
15 void CMD_Init(UART_HandleTypeDef *huart);
16 void CMD_Poll(void);
17 bool CMD_ConsumeDirty(void);
18

```

```

19 #endif
20

```

Sensor - Headers → [eink_display.h](#)

Path: sensor/Core/Inc/eink_display.h | Lines: 31

```

1 #ifndef __EINK_DISPLAY_H
2 #define __EINK_DISPLAY_H
3
4 #include <main.h>;
5 #include <stdint.h>;
6 #include <stdbool.h>;
7
8 #define EINK_WIDTH 200
9 #define EINK_HEIGHT 200
10
11 #define EINK_COLOR_WHITE 0xFF
12 #define EINK_COLOR_BLACK 0x00
13
14 #define EINK_W_BUFF_SIZE ((EINK_WIDTH % 8 == 0) ? (EINK_WIDTH / 8) : (EINK_WIDTH / 8 + 1))
15 #define EINK_BUFF_SIZE (EINK_W_BUFF_SIZE * EINK_HEIGHT)
16
17 typedef struct {
18     uint16_t width;
19     uint16_t height;
20 } EINK_Config;
21
22 bool EINK_Init(EINK_Config *cfg);
23 void EINK_Clear(void);
24 void EINK_Update(void);
25 void EINK_Sleep(void);
26 void EINK_DrawString(int x, int y, const char *str, uint8_t size);
27 void EINK_DrawStringScaled(int x, int y, const char *str, uint8_t scale);
28 void EINK_LoadImage(const uint8_t *image);
29
30 #endif
31

```

Sensor - Headers → [main.h](#)

Path: sensor/Core/Inc/main.h | Lines: 84

```

1 /* USER CODE BEGIN Header */
2 /**
3  *
4  * @file : main.h
5  * @brief : Header for main.c file.
6  * This file contains the common defines of the application.
7  *
8  * @attention
9  *
10 * Copyright (c) 2026 STMicroelectronics.
11 * All rights reserved.
12 *
13 * This software is licensed under terms that can be found in the LICENSE file
14 * in the root directory of this software component.
15 * If no LICENSE file comes with this software, it is provided AS-IS.
16 *
17 *****
18 */
19 /* USER CODE END Header */
20
21 /* Define to prevent recursive inclusion -----*/
22 #ifndef __MAIN_H
23 #define __MAIN_H
24
25 #ifdef __cplusplus
26 extern "C" {
27 #endif
28
29 /* Includes -----*/
30 #include <stm32l0xx_hal.h>;
31
32 /* Private includes -----*/
33 /* USER CODE BEGIN Includes */
34

```

```

35 /* USER CODE END Includes */
36
37 /* Exported types ----- */
38 /* USER CODE BEGIN ET */
39
40 /* USER CODE END ET */
41
42 /* Exported constants ----- */
43 /* USER CODE BEGIN EC */
44
45 /* USER CODE END EC */
46
47 /* Exported macro ----- */
48 /* USER CODE BEGIN EM */
49
50 /* USER CODE END EM */
51
52 /* Exported functions prototypes ----- */
53 void Error_Handler(void);
54
55 /* USER CODE BEGIN EFP */
56
57 /* USER CODE END EFP */
58
59 /* Private defines ----- */
60 #define LORA_NSS_Pin GPIO_PIN_4
61 #define LORA_NSS_GPIO_Port GPIOA
62 #define LORA_DIO0_Pin GPIO_PIN_4
63 #define LORA_DIO0_GPIO_Port GPIOC
64 #define LORA_RST_Pin GPIO_PIN_5
65 #define LORA_RST_GPIO_Port GPIOC
66 #define EINK_D_C_Pin GPIO_PIN_12
67 #define EINK_D_C_GPIO_Port GPIOB
68 #define EINK_BUSY_Pin GPIO_PIN_14
69 #define EINK_BUSY_GPIO_Port GPIOB
70 #define EINK_CS_Pin GPIO_PIN_6
71 #define EINK_CS_GPIO_Port GPIOC
72 #define EINK_RES_Pin GPIO_PIN_7
73 #define EINK_RES_GPIO_Port GPIOC
74
75 /* USER CODE BEGIN Private defines */
76
77 /* USER CODE END Private defines */
78
79 #ifdef __cplusplus
80 }
81 #endif
82
83 #endif /* __MAIN_H */
84

```

Sensor - Headers → [power_manager.h](#)

Path: sensor/Core/Inc/power_manager.h | Lines: 13

```

1 #ifndef __POWER_MANAGER_H
2 #define __POWER_MANAGER_H
3
4 #include <main.h>;
5 #include <stdint.h>;
6
7 #define WAKEUP_INTERVAL_MINUTES 10
8
9 void PM_Init(void);
10 void PM_EnterStop(uint32_t seconds);
11
12 #endif
13

```

Sensor - Headers → [stm32l0xx_hal_conf.h](#)

Path: sensor/Core/Inc/stm32l0xx_hal_conf.h | Lines: 330

```

1 /* USER CODE BEGIN Header */
2 /**
3 ****
4 * @file stm32l0xx_hal_conf.h

```

```

5 * @author MCD Application Team
6 * @brief HAL configuration template file.
7 * This file should be copied to the application folder and renamed
8 * to stm32l0xx_hal_conf.h.
9 *****
10 * @attention
11 *
12 * Copyright (c) 2016 STMicroelectronics.
13 * All rights reserved.
14 *
15 * This software is licensed under terms that can be found in the LICENSE file
16 * in the root directory of this software component.
17 * If no LICENSE file comes with this software, it is provided AS-IS.
18 *
19 *****
20 */
21 /* USER CODE END Header */
22
23 /* Define to prevent recursive inclusion -----*/
24 #ifndef __STM32L0xx_HAL_CONF_H
25 #define __STM32L0xx_HAL_CONF_H
26
27 #ifdef __cplusplus
28 extern "C" {
29 #endif
30
31 /* Exported types -----*/
32 /* Exported constants -----*/
33
34 /* ##### Module Selection ##### */
35 /**
36 * @brief This is the list of modules to be used in the HAL driver
37 */
38
39 #define HAL_MODULE_ENABLED
40 /*#define HAL_ADC_MODULE_ENABLED */
41 /*#define HAL_CRYP_MODULE_ENABLED */
42 /*#define HAL_COMP_MODULE_ENABLED */
43 #define HAL_I2C_MODULE_ENABLED
44 /*#define HAL_CRC_MODULE_ENABLED */
45 /*#define HAL_CRYP_MODULE_ENABLED */
46 /*#define HAL_DAC_MODULE_ENABLED */
47 /*#define HAL_FIREWALL_MODULE_ENABLED */
48 /*#define HAL_I2S_MODULE_ENABLED */
49 /*#define HAL_IWDG_MODULE_ENABLED */
50 /*#define HAL_LCD_MODULE_ENABLED */
51 /*#define HAL_LPTIM_MODULE_ENABLED */
52 /*#define HAL_RNG_MODULE_ENABLED */
53 #define HAL_RTC_MODULE_ENABLED
54 #define HAL_SPI_MODULE_ENABLED
55 /*#define HAL_TIM_MODULE_ENABLED */
56 /*#define HAL_TSC_MODULE_ENABLED */
57 #define HAL_UART_MODULE_ENABLED
58 /*#define HAL_USART_MODULE_ENABLED */
59 /*#define HAL_IRDA_MODULE_ENABLED */
60 /*#define HAL_SMARTCARD_MODULE_ENABLED */
61 /*#define HAL_SMBUS_MODULE_ENABLED */
62 /*#define HAL_WWDG_MODULE_ENABLED */
63 /*#define HAL_PCD_MODULE_ENABLED */
64 #define HAL_GPIO_MODULE_ENABLED
65 #define HAL_EXTI_MODULE_ENABLED
66 #define HAL_DMA_MODULE_ENABLED
67 #define HAL_RCC_MODULE_ENABLED
68 #define HAL_FLASH_MODULE_ENABLED
69 #define HAL_PWR_MODULE_ENABLED
70 #define HAL_CORTEX_MODULE_ENABLED
71
72 /* ##### Oscillator Values adaptation ##### */
73 /**
74 * @brief Adjust the value of External High Speed oscillator (HSE) used in your application.
75 * This value is used by the RCC HAL module to compute the system frequency
76 * (when HSE is used as system clock source, directly or through the PLL).
77 */
78 #if !defined(HSE_VALUE)
79 #define HSE_VALUE ((uint32_t)8000000U) /*< Value of the External oscillator in Hz */

```

```

80 #endif /* HSE_VALUE */
81
82 #if !defined (HSE_STARTUP_TIMEOUT)
83 #define HSE_STARTUP_TIMEOUT ((uint32_t)100U) /*!&lt; Time out for HSE start up, in ms */
84 #endif /* HSE_STARTUP_TIMEOUT */
85
86 /**
87 * @brief Internal Multiple Speed oscillator (MSI) default value.
88 * This value is the default MSI range value after Reset.
89 */
90 #if !defined (MSI_VALUE)
91 #define MSI_VALUE ((uint32_t)2097000U) /*!&lt; Value of the Internal oscillator in Hz*/
92 #endif /* MSI_VALUE */
93
94 /**
95 * @brief Internal High Speed oscillator (HSI) value.
96 * This value is used by the RCC HAL module to compute the system frequency
97 * (when HSI is used as system clock source, directly or through the PLL).
98 */
99 #if !defined (HSI_VALUE)
100 #define HSI_VALUE ((uint32_t)16000000U) /*!&lt; Value of the Internal oscillator in Hz*/
101 #endif /* HSI_VALUE */
102
103 /**
104 * @brief Internal High Speed oscillator for USB (HSI48) value.
105 */
106 #if !defined (HSI48_VALUE)
107 #define HSI48_VALUE ((uint32_t)4800000U) /*!&lt; Value of the Internal High Speed oscillator for USB in Hz.
108 The real value may vary depending on the variations
109 in voltage and temperature. */
110 #endif /* HSI48_VALUE */
111
112 /**
113 * @brief Internal Low Speed oscillator (LSI) value.
114 */
115 #if !defined (LSI_VALUE)
116 #define LSI_VALUE ((uint32_t)37000U) /*!&lt; LSI Typical Value in Hz*/
117 #endif /* LSI_VALUE */ /*!&lt; Value of the Internal Low Speed oscillator in Hz
118 The real value may vary depending on the variations
119 in voltage and temperature.*/
120 /**
121 * @brief External Low Speed oscillator (LSE) value.
122 * This value is used by the UART, RTC HAL module to compute the system frequency
123 */
124 #if !defined (LSE_VALUE)
125 #define LSE_VALUE ((uint32_t)32768U) /*!&lt; Value of the External oscillator in Hz*/
126 #endif /* LSE_VALUE */
127
128 #if !defined (LSE_STARTUP_TIMEOUT)
129 #define LSE_STARTUP_TIMEOUT ((uint32_t)5000U) /*!&lt; Time out for LSE start up, in ms */
130 #endif /* LSE_STARTUP_TIMEOUT */
131
132 /* Tip: To avoid modifying this file each time you need to use different HSE,
133 == you can define the HSE value in your toolchain compiler preprocessor. */
134
135 /* ##### System Configuration ##### */
136 /**
137 * @brief This is the HAL system configuration section
138 */
139 #define VDD_VALUE ((uint32_t)3300U) /*!&lt; Value of VDD in mv */
140 #define TICK_INT_PRIORITY ((uint32_t)3U) /*!&lt; tick interrupt priority */
141 #define USE_RTOS 0U
142 #define PREFETCH_ENABLE 0U
143 #define PREREAD_ENABLE 1U
144 #define BUFFER_CACHE_DISABLE 0U
145
146 /* ##### Assert Selection ##### */
147 /**
148 * @brief Uncomment the line below to expanse the "assert_param" macro in the
149 * HAL drivers code
150 */
151 /* #define USE_FULL_ASSERT 1U */
152
153 /* ##### Register callback feature configuration ##### */
154 /**

```

```

155 * @brief Set below the peripheral configuration to "1U" to add the support
156 * of HAL callback registration/deregistration feature for the HAL
157 * driver(s). This allows user application to provide specific callback
158 * functions thanks to HAL_PPP_RegisterCallback() rather than overwriting
159 * the default weak callback functions (see each stm32l0xx_hal_ppp.h file
160 * for possible callback identifiers defined in HAL_PPP_CallbackIDTypeDef
161 * for each PPP peripheral).
162 */
163 #define USE_HAL_ADC_REGISTER_CALLBACKS 0U
164 #define USE_HAL_COMP_REGISTER_CALLBACKS 0U
165 #define USE_HAL_DAC_REGISTER_CALLBACKS 0U
166 #define USE_HAL_I2C_REGISTER_CALLBACKS 0U
167 #define USE_HAL_I2S_REGISTER_CALLBACKS 0U
168 #define USE_HAL_IRDA_REGISTER_CALLBACKS 0U
169 #define USE_HAL_LPTIM_REGISTER_CALLBACKS 0U
170 #define USE_HAL_PCD_REGISTER_CALLBACKS 0U
171 #define USE_HAL_RNG_REGISTER_CALLBACKS 0U
172 #define USE_HAL_RTC_REGISTER_CALLBACKS 0U
173 #define USE_HAL_SMARTCARD_REGISTER_CALLBACKS 0U
174 #define USE_HAL_SMBUS_REGISTER_CALLBACKS 0U
175 #define USE_HAL_SPI_REGISTER_CALLBACKS 0U
176 #define USE_HAL_TIM_REGISTER_CALLBACKS 0U
177 #define USE_HAL_TSC_REGISTER_CALLBACKS 0U
178 #define USE_HAL_UART_REGISTER_CALLBACKS 0U
179 #define USE_HAL_USART_REGISTER_CALLBACKS 0U
180 #define USE_HAL_WWDG_REGISTER_CALLBACKS 0U
181
182 /* Includes ----- */
183 /**
184 * @brief Include module's header file
185 */
186
187 #ifdef HAL_RCC_MODULE_ENABLED
188 #include <stm32l0xx_hal_rcc.h>;
189 #endif /* HAL_RCC_MODULE_ENABLED */
190
191 #ifdef HAL_EXTI_MODULE_ENABLED
192 #include <stm32l0xx_hal_exti.h>;
193 #endif /* HAL_EXTI_MODULE_ENABLED */
194
195 #ifdef HAL_GPIO_MODULE_ENABLED
196 #include <stm32l0xx_hal_gpio.h>;
197 #endif /* HAL_GPIO_MODULE_ENABLED */
198
199 #ifdef HAL_DMA_MODULE_ENABLED
200 #include <stm32l0xx_hal_dma.h>;
201 #endif /* HAL_DMA_MODULE_ENABLED */
202
203 #ifdef HAL_CORTEX_MODULE_ENABLED
204 #include <stm32l0xx_hal_cortex.h>;
205 #endif /* HAL_CORTEX_MODULE_ENABLED */
206
207 #ifdef HAL_ADC_MODULE_ENABLED
208 #include <stm32l0xx_hal_adc.h>;
209 #endif /* HAL_ADC_MODULE_ENABLED */
210
211 #ifdef HAL_COMP_MODULE_ENABLED
212 #include <stm32l0xx_hal_comp.h>;
213 #endif /* HAL_COMP_MODULE_ENABLED */
214
215 #ifdef HAL_CRC_MODULE_ENABLED
216 #include <stm32l0xx_hal_crc.h>;
217 #endif /* HAL_CRC_MODULE_ENABLED */
218
219 #ifdef HAL_Cryp_MODULE_ENABLED
220 #include <stm32l0xx_hal_cryp.h>;
221 #endif /* HAL_Cryp_MODULE_ENABLED */
222
223 #ifdef HAL_DAC_MODULE_ENABLED
224 #include <stm32l0xx_hal_dac.h>;
225 #endif /* HAL_DAC_MODULE_ENABLED */
226
227 #ifdef HAL_FIREWALL_MODULE_ENABLED
228 #include <stm32l0xx_hal_firewall.h>;
229 #endif /* HAL_FIREWALL_MODULE_ENABLED */

```

```

230
231 #ifndef HAL_FLASH_MODULE_ENABLED
232 #include <stm32l0xx_hal_flash.h>;
233 #endif /* HAL_FLASH_MODULE_ENABLED */
234
235 #ifndef HAL_I2C_MODULE_ENABLED
236 #include <stm32l0xx_hal_i2c.h>;
237 #endif /* HAL_I2C_MODULE_ENABLED */
238
239 #ifndef HAL_I2S_MODULE_ENABLED
240 #include <stm32l0xx_hal_i2s.h>;
241 #endif /* HAL_I2S_MODULE_ENABLED */
242
243 #ifndef HAL_IWDG_MODULE_ENABLED
244 #include <stm32l0xx_hal_iwdg.h>;
245 #endif /* HAL_IWDG_MODULE_ENABLED */
246
247 #ifndef HAL_LCD_MODULE_ENABLED
248 #include <stm32l0xx_hal_lcd.h>;
249 #endif /* HAL_LCD_MODULE_ENABLED */
250
251 #ifndef HAL_LPTIM_MODULE_ENABLED
252 #include <stm32l0xx_hal_lptim.h>;
253 #endif /* HAL_LPTIM_MODULE_ENABLED */
254
255 #ifndef HAL_PWR_MODULE_ENABLED
256 #include <stm32l0xx_hal_pwr.h>;
257 #endif /* HAL_PWR_MODULE_ENABLED */
258
259 #ifndef HAL_RNG_MODULE_ENABLED
260 #include <stm32l0xx_hal_rng.h>;
261 #endif /* HAL_RNG_MODULE_ENABLED */
262
263 #ifndef HAL_RTC_MODULE_ENABLED
264 #include <stm32l0xx_hal_rtc.h>;
265
266 #endif /* HAL_RTC_MODULE_ENABLED */
267
268 #ifndef HAL_SPI_MODULE_ENABLED
269 #include <stm32l0xx_hal_spi.h>;
270 #endif /* HAL_SPI_MODULE_ENABLED */
271
272 #ifndef HAL_TIM_MODULE_ENABLED
273 #include <stm32l0xx_hal_tim.h>;
274 #endif /* HAL_TIM_MODULE_ENABLED */
275
276 #ifndef HAL_TSC_MODULE_ENABLED
277 #include <stm32l0xx_hal_tsc.h>;
278 #endif /* HAL_TSC_MODULE_ENABLED */
279
280 #ifndef HAL_UART_MODULE_ENABLED
281 #include <stm32l0xx_hal_uart.h>;
282 #endif /* HAL_UART_MODULE_ENABLED */
283
284 #ifndef HAL_USART_MODULE_ENABLED
285 #include <stm32l0xx_hal_usart.h>;
286 #endif /* HAL_USART_MODULE_ENABLED */
287
288 #ifndef HAL_IRDA_MODULE_ENABLED
289 #include <stm32l0xx_hal_irda.h>;
290 #endif /* HAL_IRDA_MODULE_ENABLED */
291
292 #ifndef HAL_SMARTCARD_MODULE_ENABLED
293 #include <stm32l0xx_hal_smartcard.h>;
294 #endif /* HAL_SMARTCARD_MODULE_ENABLED */
295
296 #ifndef HAL_SMBUS_MODULE_ENABLED
297 #include <stm32l0xx_hal_smbus.h>;
298 #endif /* HAL_SMBUS_MODULE_ENABLED */
299
300 #ifndef HAL_WWDG_MODULE_ENABLED
301 #include <stm32l0xx_hal_wwdg.h>;
302 #endif /* HAL_WWDG_MODULE_ENABLED */
303
304 #ifndef HAL_PCD_MODULE_ENABLED

```

```

305 #include <stm32l0xx_hal_pcd.h>;
306 #endif /* HAL_PCD_MODULE_ENABLED */
307
308 /* Exported macro ----- */
309 #ifdef USE_FULL_ASSERT
310 /**
311 * @brief The assert_param macro is used for function's parameters check.
312 * @param expr: If expr is false, it calls assert_failed function
313 * which reports the name of the source file and the source
314 * line number of the call that failed.
315 * If expr is true, it returns no value.
316 * @retval None
317 */
318 #define assert_param(expr) ((expr) ? (void)0U : assert_failed((uint8_t *)__FILE__, __LINE__))
319 /* Exported functions ----- */
320 void assert_failed(uint8_t* file, uint32_t line);
321 #else
322 #define assert_param(expr) ((void)0U)
323 #endif /* USE_FULL_ASSERT */
324
325 #ifdef __cplusplus
326 }
327 #endif
328
329 #endif /* __STM32L0xx_HAL_CONF_H */
330

```

Sensor - Headers → [stm32l0xx_it.h](#)

Path: sensor/Core/Inc/stm32l0xx_it.h | Lines: 63

```

1 /* USER CODE BEGIN Header */
2 /**
3 *****
4 * @file stm32l0xx_it.h
5 * @brief This file contains the headers of the interrupt handlers.
6 *****
7 * @attention
8 *
9 * Copyright (c) 2026 STMicroelectronics.
10 * All rights reserved.
11 *
12 * This software is licensed under terms that can be found in the LICENSE file
13 * in the root directory of this software component.
14 * If no LICENSE file comes with this software, it is provided AS-IS.
15 *
16 *****
17 */
18 /* USER CODE END Header */
19
20 /* Define to prevent recursive inclusion ----- */
21 #ifndef __STM32L0xx_IT_H
22 #define __STM32L0xx_IT_H
23
24 #ifdef __cplusplus
25 extern "C" {
26 #endif
27
28 /* Private includes ----- */
29 /* USER CODE BEGIN Includes */
30
31 /* USER CODE END Includes */
32
33 /* Exported types ----- */
34 /* USER CODE BEGIN ET */
35
36 /* USER CODE END ET */
37
38 /* Exported constants ----- */
39 /* USER CODE BEGIN EC */
40
41 /* USER CODE END EC */
42
43 /* Exported macro ----- */
44 /* USER CODE BEGIN EM */
45

```

```

46 /* USER CODE END EM */
47
48 /* Exported functions prototypes -----*/
49 void NMI_Handler(void);
50 void HardFault_Handler(void);
51 void SVC_Handler(void);
52 void PendSV_Handler(void);
53 void SysTick_Handler(void);
54 /* USER CODE BEGIN EFP */
55
56 /* USER CODE END EFP */
57
58 #ifdef __cplusplus
59 }
60 #endif
61
62 #endif /* __STM32L0xx_IT_H */
63

```

Sensor - Headers → [sx1278.h](#)

Path: sensor/Core/Inc/sx1278.h | Lines: 74

```

1 #ifndef __SX1278_H
2 #define __SX1278_H
3
4 #include <main.h>;
5 #include <stdint.h>;
6 #include <stdbool.h>;
7
8 #define SX1278_REG_FIFO 0x00
9 #define SX1278_REG_OP_MODE 0x01
10 #define SX1278_REG_FRF_MSB 0x06
11 #define SX1278_REG_FRF_MID 0x07
12 #define SX1278_REG_FRF_LSB 0x08
13 #define SX1278_REG_PA_CONFIG 0x09
14 #define SX1278_REG_PA_RAMP 0x0A
15 #define SX1278_REG_OCP 0x0B
16 #define SX1278_REG_LNA 0x0C
17 #define SX1278_REG_FIFO_ADDR_PTR 0x0D
18 #define SX1278_REG_FIFO_TX_BASE_ADDR 0x0E
19 #define SX1278_REG_FIFO_RX_BASE_ADDR 0x0F
20 #define SX1278_REG_FIFO_RX_CURRENT_ADDR 0x10
21 #define SX1278_REG_IRQ_FLAGS_MASK 0x11
22 #define SX1278_REG_IRQ_FLAGS 0x12
23 #define SX1278_REG_RX_NB_BYTES 0x13
24 #define SX1278_REG_PKT_SNR_VALUE 0x19
25 #define SX1278_REG_PKT_RSSI_VALUE 0x1A
26 #define SX1278_REG_MODEM_CONFIG_1 0x1D
27 #define SX1278_REG_MODEM_CONFIG_2 0x1E
28 #define SX1278_REG_MODEM_CONFIG_3 0x26
29 #define SX1278_REG_PAYLOAD_LENGTH 0x22
30 #define SX1278_REG_FIFO_RX_BYTE_ADDR 0x25
31 #define SX1278_REG_DIO_MAPPING_1 0x40
32 #define SX1278_REG_DIO_MAPPING_2 0x41
33 #define SX1278_REG_VERSION 0x42
34 #define SX1278_REG_PA_DAC 0x4D
35
36 #define SX1278_MODE_LONG_RANGE 0x80
37 #define SX1278_MODE_SLEEP 0x00
38 #define SX1278_MODE_STDBY 0x01
39 #define SX1278_MODE_TX 0x03
40 #define SX1278_MODE_RX_CONTINUOUS 0x05
41 #define SX1278_MODE_RX_SINGLE 0x06
42
43 #define SX1278_FXOSC 32000000ULL
44
45 #define SX1278_IRQ_TX_DONE_MASK 0x08
46 #define SX1278_IRQ_RX_DONE_MASK 0x40
47 #define SX1278_IRQ_RX_TIMEOUT_MASK 0x80
48
49 #define SX1278_DIO0_TX_DONE 0x01
50 #define SX1278_DIO0_RX_DONE 0x00
51
52 #define SX1278_MAX_PACKET_SIZE 255
53

```

```

54 typedef enum {
55     SX1278_CFG_POWER = 0,
56     SX1278_CFG_BALANCED,
57     SX1278_CFG_PERFORMANCE,
58     SX1278_CFG_COUNT
59 } SX1278_Config;
60
61 extern volatile bool sx1278_tx_done;
62
63 bool SX1278_Init(SX1278_Config cfg);
64 bool SX1278_SendPacket(uint8_t *data, uint8_t len, SX1278_Config cfg);
65 void SX1278_HardReset(void);
66 void SX1278_EnterSleep(void);
67 void SX1278_SetStandby(void);
68 uint8_t SX1278_ReadVersion(void);
69 uint8_t SX1278_ReadIrqFlags(void);
70 uint8_t SX1278_ReadReg(uint8_t reg);
71 void SX1278_WriteReg(uint8_t reg, uint8_t val);
72
73 #endif
74

```

Sensor - Sources → [bme280.c](#)

Path: sensor/Core/Src/bme280.c | Lines: 181

```

1  #include <bme280.h>;
2
3  static HAL_StatusTypeDef i2c_status;
4
5  static uint8_t BME280_ReadReg(BME280_HandleTypeDef *bme, uint8_t reg)
6  {
7      uint8_t val = 0;
8      i2c_status = HAL_I2C_Mem_Read(bme->hi2c, bme->addr << 1, reg,
9      I2C_MEMADD_SIZE_8BIT, &val, 1, 100);
10     return val;
11 }
12
13 static HAL_StatusTypeDef BME280_WriteReg(BME280_HandleTypeDef *bme, uint8_t reg, uint8_t val)
14 {
15     i2c_status = HAL_I2C_Mem_Write(bme->hi2c, bme->addr << 1, reg,
16     I2C_MEMADD_SIZE_8BIT, &val, 1, 100);
17     return i2c_status;
18 }
19
20 static void BME280_ReadBurst(BME280_HandleTypeDef *bme, uint8_t reg,
21 uint8_t *buf, uint8_t len)
22 {
23     i2c_status = HAL_I2C_Mem_Read(bme->hi2c, bme->addr << 1, reg,
24     I2C_MEMADD_SIZE_8BIT, buf, len, 100);
25 }
26
27 static void BME280_ReadCalib(BME280_HandleTypeDef *bme)
28 {
29     uint8_t calib[26];
30     BME280_ReadBurst(bme, 0x88, calib, 24);
31
32     bme->dig_T1 = (uint16_t)(calib[1] << 8 | calib[0]);
33     bme->dig_T2 = (int16_t)(calib[3] << 8 | calib[2]);
34     bme->dig_T3 = (int16_t)(calib[5] << 8 | calib[4]);
35     bme->dig_P1 = (uint16_t)(calib[7] << 8 | calib[6]);
36     bme->dig_P2 = (int16_t)(calib[9] << 8 | calib[8]);
37     bme->dig_P3 = (int16_t)(calib[11] << 8 | calib[10]);
38     bme->dig_P4 = (int16_t)(calib[13] << 8 | calib[12]);
39     bme->dig_P5 = (int16_t)(calib[15] << 8 | calib[14]);
40     bme->dig_P6 = (int16_t)(calib[17] << 8 | calib[16]);
41     bme->dig_P7 = (int16_t)(calib[19] << 8 | calib[18]);
42     bme->dig_P8 = (int16_t)(calib[21] << 8 | calib[20]);
43     bme->dig_P9 = (int16_t)(calib[23] << 8 | calib[22]);
44
45     bme->dig_H1 = BME280_ReadReg(bme, 0xA1);
46
47     BME280_ReadBurst(bme, 0xE1, calib, 8);
48     bme->dig_H2 = (int16_t)((uint16_t)calib[1] << 8 | calib[0]);
49     bme->dig_H3 = calib[2];
50     bme->dig_H4 = (int16_t)(((uint16_t)calib[3] << 4) | (calib[4] & 0x0F));

```

```

51 bme->dig_H5 = (int16_t)((((uint16_t)calib[5] << 4) | (calib[4] >> 4)));
52 bme->dig_H6 = (int8_t)calib[6];
53 }
54
55 bool BME280_Init(BME280_HandleTypeDef *bme, I2C_HandleTypeDef *hi2c, uint8_t addr)
56 {
57     bme->hi2c = hi2c;
58     bme->addr = addr;
59     bme->t_fine = 0;
60
61     uint8_t id = BME280_ReadReg(bme, BME280_REG_CHIP_ID);
62     if (id != BME280_CHIP_ID) return false;
63
64     BME280_WriteReg(bme, BME280_REG_RESET, BME280_RESET_CMD);
65     HAL_Delay(2);
66 {
67     uint8_t status;
68     int tries = 5;
69     do {
70         HAL_Delay(2);
71         BME280_ReadBurst(bme, BME280_REG_STATUS, &status, 1);
72     } while (--tries && (status & 0x01));
73 }
74
75     BME280_ReadCalib(bme);
76
77     BME280_WriteReg(bme, BME280_REG_CTRL_HUM, BME280_OVERSAMPLING_1);
78     BME280_WriteReg(bme, BME280_REG_CONFIG,
79 (BME280_STANDBY_0_5 << 5) | (BME280_FILTER_OFF << 2));
80     BME280_WriteReg(bme, BME280_REG_CTRL_MEAS,
81 (BME280_OVERSAMPLING_1 << 5) |
82 (BME280_OVERSAMPLING_1 << 2) |
83 BME280_MODE_SLEEP);
84
85     return true;
86 }
87
88 static int32_t BME280_Compensate_T(BME280_HandleTypeDef *bme, int32_t adc_T)
89 {
90     int32_t var1 = (((adc_T >> 3) - ((int32_t)bme->dig_T1 << 1)))
91 * ((int32_t)bme->dig_T2) >> 11;
92     int32_t var2 = (((adc_T >> 4) - ((int32_t)bme->dig_T1))
93 * ((adc_T >> 4) - ((int32_t)bme->dig_T1))) >> 12;
94 * ((int32_t)bme->dig_T3) >> 14;
95     bme->t_fine = var1 + var2;
96     return (bme->t_fine * 5 + 128) >> 8;
97 }
98
99 static uint32_t BME280_Compensate_P(BME280_HandleTypeDef *bme, int32_t adc_P)
100 {
101     int64_t var1 = ((int64_t)bme->t_fine) - 128000;
102     int64_t var2 = var1 * var1 * (int64_t)bme->dig_P6;
103     var2 = var2 + ((var1 * (int64_t)bme->dig_P5) << 17);
104     var2 = var2 + (((int64_t)bme->dig_P4) << 35);
105     var1 = ((var1 * var1 * (int64_t)bme->dig_P3) >> 8) +
106 ((var1 * (int64_t)bme->dig_P2) << 12);
107     var1 = (((((int64_t)1) << 47) + var1)) * ((int64_t)bme->dig_P1) >> 33;
108     if (var1 == 0) return 0;
109     int64_t p = 1048576 - adc_P;
110     p = ((p << 31) - var2) * 3125 / var1;
111     var1 = (((int64_t)bme->dig_P9) * (p >> 13) * (p >> 13)) >> 25;
112     var2 = (((int64_t)bme->dig_P8) * p) >> 19;
113     p = ((p + var1 + var2) >> 8) + (((int64_t)bme->dig_P7) << 4);
114     return (uint32_t)p;
115 }
116
117 static uint32_t BME280_Compensate_H(BME280_HandleTypeDef *bme, int32_t adc_H)
118 {
119     int32_t v_x1_u32r = bme->t_fine - ((int32_t)76800);
120     v_x1_u32r = (((adc_H << 14) - ((int32_t)bme->dig_H4) << 20)
121 - (((int32_t)bme->dig_H5) * v_x1_u32r))
122 + ((int32_t)16384) >> 15;
123 * ((((((v_x1_u32r * ((int32_t)bme->dig_H6)) >> 10)
124 * ((v_x1_u32r * ((int32_t)bme->dig_H3)) >> 11)
125 + ((int32_t)32768))) >> 10) + ((int32_t)2097152))

```

```

126 * ((int32_t)bme->dig_H2) + 8192) >> 14));
127 v_x1_u32r = v_x1_u32r - (((((v_x1_u32r >> 15) * (v_x1_u32r >> 15)) >> 7)
128 * ((int32_t)bme->dig_H1)) >> 4);
129 v_x1_u32r = (v_x1_u32r < 0) ? 0 : v_x1_u32r;
130 v_x1_u32r = (v_x1_u32r > 419430400) ? 419430400 : v_x1_u32r;
131 return (uint32_t)(v_x1_u32r >> 12);
132 }
133
134 bool BME280_ReadAll(BME280_HandleTypeDef *bme, BME280_Data *data)
135 {
136 BME280_WriteReg(bme, BME280_REG_CTRL_HUM, BME280_OVERSAMPLING_1);
137 if (i2c_status != HAL_OK) goto err;
138
139 BME280_WriteReg(bme, BME280_REG_CTRL_MEAS,
140 (BME280_OVERSAMPLING_1 << 5) |
141 (BME280_OVERSAMPLING_1 << 2) |
142 BME280_MODE_FORCED);
143 if (i2c_status != HAL_OK) goto err;
144
145 HAL_Delay(10);
146
147 {
148 uint8_t press_msb = BME280_ReadReg(bme, 0xF7);
149 uint8_t press_lsb = BME280_ReadReg(bme, 0xF8);
150 uint8_t press_xlsb = BME280_ReadReg(bme, 0xF9);
151 uint8_t temp_msb = BME280_ReadReg(bme, 0xFA);
152 uint8_t temp_lsb = BME280_ReadReg(bme, 0xFB);
153 uint8_t temp_xlsb = BME280_ReadReg(bme, 0xFC);
154 uint8_t hum_msb = BME280_ReadReg(bme, 0xFD);
155 uint8_t hum_lsb = BME280_ReadReg(bme, 0xFE);
156 if (i2c_status != HAL_OK) goto err;
157
158 int32_t adc_p = ((int32_t)press_msb << 12) | ((int32_t)press_lsb << 4) | (press_xlsb >> 4);
159 int32_t adc_t = ((int32_t)temp_msb << 12) | ((int32_t)temp_lsb << 4) | (temp_xlsb >> 4);
160 int32_t adc_h = ((int32_t)hum_msb << 8) | hum_lsb;
161
162 int32_t t_comp = BME280_Compensate_T(bme, adc_t);
163 data->temperature = t_comp / 100.0f;
164
165 uint32_t p_var4 = BME280_Compensate_P(bme, adc_p);
166 data->pressure = (float)p_var4 / 256.0f;
167
168 uint32_t h_comp = BME280_Compensate_H(bme, adc_h);
169 data->humidity = h_comp / 1024.0f;
170 }
171 return true;
172
173 err:
174 data->temperature = -(float)i2c_status;
175 data->humidity = 0.0f;
176 data->pressure = 0.0f;
177 HAL_I2C_DeInit(bme->hi2c);
178 HAL_I2C_Init(bme->hi2c);
179 return false;
180 }
181

```

Sensor - Sources → [cmd.c](#)

Path: sensor/Core/Src/cmd.c | Lines: 210

```

1 #include <cmd.h>;
2 #include <sx1278.h>;
3 #include <stdio.h>;
4 #include <string.h>;
5 #include <stdlib.h>;
6 #include <ctype.h>;
7
8 #define CONFIG_FLASH_ADDR 0x0801FC00UL
9 #define CONFIG_MAGIC 0xBEEFCAFEUL
10
11 static UART_HandleTypeDef *cmd_huart;
12 static char rx_buf[64];
13 static uint8_t rx_idx;
14
15 SX1278_Config g_lora_cfg = SX1278_CFG_PERFORMANCE;

```

```

16 uint32_t g_interval_ms = 5000;
17 float g_offset_temp = 0.0f;
18 float g_offset_hum = 0.0f;
19 float g_offset_pres = 0.0f;
20 bool g_config_dirty = false;
21
22 static void config_save(void)
23 {
24 HAL_FLASH_Unlock();
25 FLASH_EraseInitTypeDef erase = {0};
26 erase.TypeErase = FLASH_TYPEERASE_PAGES;
27 erase.PageAddress = CONFIG_FLASH_ADDR;
28 erase.NbPages = 1;
29 uint32_t page_err;
30 HAL_FLASHEx_Erase(&erase, &page_err);
31 uint32_t data[6];
32 data[0] = CONFIG_MAGIC;
33 data[1] = (uint32_t)g_lora_cfg;
34 data[2] = g_interval_ms;
35 memcpy(&data[3], &g_offset_temp, 4);
36 memcpy(&data[4], &g_offset_hum, 4);
37 memcpy(&data[5], &g_offset_pres, 4);
38 for (int i = 0; i < 6; i++) {
39 HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, CONFIG_FLASH_ADDR + i * 4, data[i]);
40 }
41 HAL_FLASH_Lock();
42 }
43
44 static bool config_load(void)
45 {
46 uint32_t *p = (uint32_t *)CONFIG_FLASH_ADDR;
47 if (p[0] != CONFIG_MAGIC) return false;
48 uint32_t cfg = p[1];
49 uint32_t interval = p[2];
50 float ot, oh, op;
51 memcpy(&ot, &p[3], 4);
52 memcpy(&oh, &p[4], 4);
53 memcpy(&op, &p[5], 4);
54 if (cfg >= SX1278_CFG_COUNT) return false;
55 if (interval < 5000 || interval > 3600000) return false;
56 if (ot < -100 || ot > 100) return false;
57 if (oh < -100 || oh > 100) return false;
58 if (op < -1000000 || op > 1000000) return false;
59 g_lora_cfg = (SX1278_Config)cfg;
60 g_interval_ms = interval;
61 g_offset_temp = ot;
62 g_offset_hum = oh;
63 g_offset_pres = op;
64 return true;
65 }
66
67 static const char *cfg_name(SX1278_Config cfg)
68 {
69 switch (cfg) {
70 case SX1278_CFG_POWER: return "POWER";
71 case SX1278_CFG_BALANCED: return "BALANCED";
72 case SX1278_CFG_PERFORMANCE: return "PERFORMANCE";
73 default: return "?";
74 }
75 }
76
77 static void respond(const char *msg)
78 {
79 HAL_UART_Transmit(cmd_huart, (uint8_t *)msg, strlen(msg), 1000);
80 }
81
82 static int streq_ncase(const char *a, const char *b)
83 {
84 while (*a && *b) {
85 if (tolower((unsigned char)*a) != tolower((unsigned char)*b))
86 return 0;
87 a++; b++;
88 }
89 return *a == *b;
90 }

```

```

91
92 static void cmd_help(void)
93 {
94     respond("Commands:\r\n"
95     " LORA POWER|BALANCED|PERF <-- switch mode\r\n"
96     " RATE <sec> <-- sensor interval (5-3600)\r\n"
97     " OFFSET T|H|P <val> <-- cal offset\r\n"
98     " STATUS <-- show config\r\n"
99     " PING <-- PONG\r\n"
100    " HELP <-- this\r\n");
101 }
102
103 static void cmd_status(void)
104 {
105     char buf[128];
106     snprintf(buf, sizeof(buf),
107     "Mode:%s Rate:%lus Off T:%.1f H:%.1f P:%.0f\r\n",
108     cfg_name(g_lora_cfg), (unsigned long)(g_interval_ms / 1000),
109     (double)g_offset_temp, (double)g_offset_hum,
110     (double)g_offset_pres);
111     respond(buf);
112 }
113
114 static void cmd_parse(const char *line)
115 {
116     char cmd[16] = {0};
117     char a1[16] = {0};
118     float val = 0.0f;
119
120     int n = sscanf(line, "%15s %15s %f", cmd, a1, &val);
121
122     if (n < 1) return;
123
124     if (streq_ncase(cmd, "PING")) {
125         respond("PONG\r\n");
126     } else if (streq_ncase(cmd, "HELP")) {
127         cmd_help();
128     } else if (streq_ncase(cmd, "STATUS")) {
129         cmd_status();
130     } else if (streq_ncase(cmd, "LORA")) {
131         if (n < 2) { respond("ERR: usage LORA POWER|BALANCED|PERF\r\n"); return; }
132         SX1278_Config new_cfg = g_lora_cfg;
133         if (streq_ncase(a1, "POWER")) new_cfg = SX1278_CFG_POWER;
134         else if (streq_ncase(a1, "BALANCED")) new_cfg = SX1278_CFG_BALANCED;
135         else if (streq_ncase(a1, "PERF") ||
136         streq_ncase(a1, "PERFORMANCE")) new_cfg = SX1278_CFG_PERFORMANCE;
137         else { respond("ERR: unknown mode\r\n"); return; }
138         if (new_cfg != g_lora_cfg) {
139             g_lora_cfg = new_cfg;
140             SX1278_Init(g_lora_cfg);
141         }
142         g_config_dirty = true;
143         config_save();
144         char r[64];
145         snprintf(r, sizeof(r), "OK LoRa=%s\r\n", cfg_name(g_lora_cfg));
146         respond(r);
147     } else if (streq_ncase(cmd, "RATE")) {
148         int sec;
149         if (sscanf(line, "%*s %d", &sec) != 1) {
150             respond("ERR: usage RATE <seconds>\r\n"); return;
151         }
152         if (sec < 5 || sec > 3600) { respond("ERR: rate 5-3600s\r\n"); return; }
153         g_interval_ms = (uint32_t)sec * 1000;
154         g_config_dirty = true;
155         config_save();
156         char r[64];
157         snprintf(r, sizeof(r), "OK rate=%lus\r\n", (unsigned long)(g_interval_ms / 1000));
158         respond(r);
159     } else if (streq_ncase(cmd, "OFFSET")) {
160         if (sscanf(line, "%*s %15s %f", a1, &val) != 2) {
161             respond("ERR: usage OFFSET T|H|P <val>\r\n"); return;
162         }
163         if (streq_ncase(a1, "T")) { g_offset_temp = val; }
164         else if (streq_ncase(a1, "H")) { g_offset_hum = val; }
165         else if (streq_ncase(a1, "P")) { g_offset_pres = val; }

```

```

166 else { respond("ERR: use T, H, or P\r\n"); return; }
167 g_config_dirty = true;
168 config_save();
169 respond("OK\r\n");
170 } else {
171 respond("ERR: unknown command. Try HELP\r\n");
172 }
173 }
174
175 void CMD_Init(UART_HandleTypeDef *huart)
176 {
177 cmd_huart = huart;
178 rx_idx = 0;
179 memset(rx_buf, 0, sizeof(rx_buf));
180 if (config_load()) {
181 respond("Loaded saved config\r\n");
182 } else {
183 respond("Defaults loaded\r\n");
184 }
185 respond("Ready. Type HELP for commands\r\n");
186 }
187
188 bool CMD_ConsumeDirty(void)
189 {
190 bool was = g_config_dirty;
191 g_config_dirty = false;
192 return was;
193 }
194
195 void CMD_Poll(void)
196 {
197 uint8_t ch;
198 while (HAL_UART_Receive(cmd_huart, &ch, 1, 1) == HAL_OK) {
199 if (ch == '\r' || ch == '\n') {
200 if (rx_idx > 0) {
201 rx_buf[rx_idx] = '\0';
202 cmd_parse(rx_buf);
203 rx_idx = 0;
204 }
205 } else if (rx_idx < (int)sizeof(rx_buf) - 1) {
206 rx_buf[rx_idx++] = (char)ch;
207 }
208 }
209 }
210

```

Sensor - Sources → [eink_display.c](#)

Path: sensor/Core/Src/eink_display.c | Lines: 364

```

1 #include "eink_display.h"
2 #include <string.h>
3
4 extern SPI_HandleTypeDef hspi2;
5
6 static uint8_t fb[EINK_BUFF_SIZE];
7 static uint16_t e_w = EINK_WIDTH;
8 static uint16_t e_h = EINK_HEIGHT;
9 static bool sleeping = false;
10 static void EINK_WakeInternal(void);
11
12 static const uint8_t font_8x6[96][6] = {
13 {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, /* */
14 {0x00, 0x00, 0x4F, 0x00, 0x00, 0x00}, /* ! */
15 {0x00, 0x07, 0x00, 0x07, 0x00, 0x00}, /* " */
16 {0x14, 0x7F, 0x14, 0x7F, 0x14, 0x00}, /* # */
17 {0x24, 0x2A, 0x7F, 0x2A, 0x12, 0x00}, /* $ */
18 {0x23, 0x13, 0x08, 0x64, 0x62, 0x00}, /* % */
19 {0x36, 0x49, 0x55, 0x22, 0x50, 0x00}, /* & */
20 {0x00, 0x05, 0x03, 0x00, 0x00, 0x00}, /* ' */
21 {0x00, 0x1C, 0x22, 0x41, 0x00, 0x00}, /* ( */
22 {0x00, 0x41, 0x22, 0x1C, 0x00, 0x00}, /* ) */
23 {0x08, 0x2A, 0x1C, 0x2A, 0x08, 0x00}, /* * */
24 {0x08, 0x08, 0x3E, 0x08, 0x08, 0x00}, /* + */
25 {0x00, 0x50, 0x30, 0x00, 0x00, 0x00}, /* , */
26 {0x08, 0x08, 0x08, 0x08, 0x08, 0x00}, /* - */

```

```

27 {0x00, 0x60, 0x60, 0x00, 0x00, 0x00}, /* . */
28 {0x20, 0x10, 0x08, 0x04, 0x02, 0x00}, /* / */
29 {0x3E, 0x51, 0x49, 0x45, 0x3E, 0x00}, /* 0 */
30 {0x00, 0x42, 0x7F, 0x40, 0x00, 0x00}, /* 1 */
31 {0x42, 0x61, 0x51, 0x49, 0x46, 0x00}, /* 2 */
32 {0x21, 0x41, 0x45, 0x4B, 0x31, 0x00}, /* 3 */
33 {0x18, 0x14, 0x12, 0x7F, 0x10, 0x00}, /* 4 */
34 {0x27, 0x45, 0x45, 0x45, 0x39, 0x00}, /* 5 */
35 {0x3C, 0x4A, 0x49, 0x49, 0x30, 0x00}, /* 6 */
36 {0x01, 0x71, 0x09, 0x05, 0x03, 0x00}, /* 7 */
37 {0x36, 0x49, 0x49, 0x49, 0x36, 0x00}, /* 8 */
38 {0x06, 0x49, 0x49, 0x29, 0x1E, 0x00}, /* 9 */
39 {0x00, 0x36, 0x36, 0x00, 0x00, 0x00}, /* : */
40 {0x00, 0x56, 0x36, 0x00, 0x00, 0x00}, /* ; */
41 {0x00, 0x08, 0x14, 0x22, 0x41, 0x00}, /* < */
42 {0x14, 0x14, 0x14, 0x14, 0x14, 0x00}, /* = */
43 {0x41, 0x22, 0x14, 0x08, 0x00, 0x00}, /* > */
44 {0x02, 0x01, 0x51, 0x09, 0x06, 0x00}, /* ? */
45 {0x32, 0x49, 0x79, 0x41, 0x3E, 0x00}, /* @ */
46 {0x7E, 0x11, 0x11, 0x11, 0x7E, 0x00}, /* A */
47 {0x7F, 0x49, 0x49, 0x49, 0x36, 0x00}, /* B */
48 {0x3E, 0x41, 0x41, 0x41, 0x22, 0x00}, /* C */
49 {0x7F, 0x41, 0x41, 0x22, 0x1C, 0x00}, /* D */
50 {0x7F, 0x49, 0x49, 0x49, 0x41, 0x00}, /* E */
51 {0x7F, 0x09, 0x09, 0x01, 0x01, 0x00}, /* F */
52 {0x3E, 0x41, 0x41, 0x51, 0x32, 0x00}, /* G */
53 {0x7F, 0x08, 0x08, 0x08, 0x7F, 0x00}, /* H */
54 {0x00, 0x41, 0x7F, 0x41, 0x00, 0x00}, /* I */
55 {0x20, 0x40, 0x41, 0x3F, 0x01, 0x00}, /* J */
56 {0x7F, 0x08, 0x14, 0x22, 0x41, 0x00}, /* K */
57 {0x7F, 0x40, 0x40, 0x40, 0x40, 0x00}, /* L */
58 {0x7F, 0x02, 0x04, 0x02, 0x7F, 0x00}, /* M */
59 {0x7F, 0x04, 0x08, 0x10, 0x7F, 0x00}, /* N */
60 {0x3E, 0x41, 0x41, 0x41, 0x3E, 0x00}, /* O */
61 {0x7F, 0x09, 0x09, 0x09, 0x06, 0x00}, /* P */
62 {0x3E, 0x41, 0x51, 0x21, 0x5E, 0x00}, /* Q */
63 {0x7F, 0x09, 0x19, 0x29, 0x46, 0x00}, /* R */
64 {0x46, 0x49, 0x49, 0x49, 0x31, 0x00}, /* S */
65 {0x01, 0x01, 0x7F, 0x01, 0x01, 0x00}, /* T */
66 {0x3F, 0x40, 0x40, 0x40, 0x3F, 0x00}, /* U */
67 {0x1F, 0x20, 0x40, 0x20, 0x1F, 0x00}, /* V */
68 {0x7F, 0x20, 0x18, 0x20, 0x7F, 0x00}, /* W */
69 {0x63, 0x14, 0x08, 0x14, 0x63, 0x00}, /* X */
70 {0x03, 0x04, 0x78, 0x04, 0x03, 0x00}, /* Y */
71 {0x61, 0x51, 0x49, 0x45, 0x43, 0x00}, /* Z */
72 {0x00, 0x00, 0x7F, 0x41, 0x41, 0x00}, /* [ */
73 {0x02, 0x04, 0x08, 0x10, 0x20, 0x00}, /* \ */
74 {0x41, 0x41, 0x7F, 0x00, 0x00, 0x00}, /* ] */
75 {0x04, 0x02, 0x01, 0x02, 0x04, 0x00}, /* ^ */
76 {0x40, 0x40, 0x40, 0x40, 0x40, 0x00}, /* _ */
77 {0x00, 0x01, 0x02, 0x04, 0x00, 0x00}, /* ` */
78 {0x20, 0x54, 0x54, 0x54, 0x78, 0x00}, /* a */
79 {0x7F, 0x48, 0x44, 0x44, 0x38, 0x00}, /* b */
80 {0x38, 0x44, 0x44, 0x44, 0x20, 0x00}, /* c */
81 {0x38, 0x44, 0x44, 0x48, 0x7F, 0x00}, /* d */
82 {0x38, 0x54, 0x54, 0x54, 0x18, 0x00}, /* e */
83 {0x08, 0x7E, 0x09, 0x01, 0x02, 0x00}, /* f */
84 {0x08, 0x14, 0x54, 0x54, 0x3C, 0x00}, /* g */
85 {0x7F, 0x08, 0x04, 0x04, 0x78, 0x00}, /* h */
86 {0x00, 0x44, 0x7D, 0x40, 0x00, 0x00}, /* i */
87 {0x20, 0x40, 0x44, 0x3D, 0x00, 0x00}, /* j */
88 {0x00, 0x7F, 0x10, 0x28, 0x44, 0x00}, /* k */
89 {0x00, 0x41, 0x7F, 0x40, 0x00, 0x00}, /* l */
90 {0x7C, 0x04, 0x18, 0x04, 0x78, 0x00}, /* m */
91 {0x7C, 0x08, 0x04, 0x04, 0x78, 0x00}, /* n */
92 {0x38, 0x44, 0x44, 0x44, 0x38, 0x00}, /* o */
93 {0x7C, 0x14, 0x14, 0x14, 0x08, 0x00}, /* p */
94 {0x08, 0x14, 0x14, 0x18, 0x7C, 0x00}, /* q */
95 {0x7C, 0x08, 0x04, 0x04, 0x08, 0x00}, /* r */
96 {0x48, 0x54, 0x54, 0x54, 0x20, 0x00}, /* s */
97 {0x04, 0x3F, 0x44, 0x40, 0x20, 0x00}, /* t */
98 {0x3C, 0x40, 0x40, 0x20, 0x7C, 0x00}, /* u */
99 {0x1C, 0x20, 0x40, 0x20, 0x1C, 0x00}, /* v */
100 {0x3C, 0x40, 0x30, 0x40, 0x3C, 0x00}, /* w */
101 {0x44, 0x28, 0x10, 0x28, 0x44, 0x00}, /* x */

```

```

102 {0x0C, 0x50, 0x50, 0x50, 0x3C, 0x00}, /* y */
103 {0x44, 0x64, 0x54, 0x4C, 0x44, 0x00}, /* z */
104 };
105
106 static void EINK_DC_CMD(void)
107 {
108 HAL_GPIO_WritePin(EINK_D_C_GPIO_Port, EINK_D_C_Pin, GPIO_PIN_RESET);
109 }
110
111 static void EINK_DC_DATA(void)
112 {
113 HAL_GPIO_WritePin(EINK_D_C_GPIO_Port, EINK_D_C_Pin, GPIO_PIN_SET);
114 }
115
116 static void EINK_CS_LOW(void)
117 {
118 HAL_GPIO_WritePin(EINK_CS_GPIO_Port, EINK_CS_Pin, GPIO_PIN_RESET);
119 }
120
121 static void EINK_CS_HIGH(void)
122 {
123 HAL_GPIO_WritePin(EINK_CS_GPIO_Port, EINK_CS_Pin, GPIO_PIN_SET);
124 }
125
126 static void EINK_SendCmd(uint8_t cmd)
127 {
128 EINK_DC_CMD();
129 EINK_CS_LOW();
130 HAL_SPI_Transmit(&hspi2, &cmd, 1, 10);
131 EINK_CS_HIGH();
132 EINK_DC_DATA();
133 }
134
135 static void EINK_SendData(uint8_t data)
136 {
137 EINK_CS_LOW();
138 HAL_SPI_Transmit(&hspi2, &data, 1, 10);
139 EINK_CS_HIGH();
140 }
141
142 static void EINK_SendDataBurst(uint8_t *data, uint16_t len)
143 {
144 EINK_CS_LOW();
145 HAL_SPI_Transmit(&hspi2, data, len, 1000);
146 EINK_CS_HIGH();
147 }
148
149 static bool EINK_Busy(void)
150 {
151 return (HAL_GPIO_ReadPin(EINK_BUSY_GPIO_Port, EINK_BUSY_Pin) == GPIO_PIN_SET);
152 }
153
154 static void EINK_WaitBusy(void)
155 {
156 uint32_t timeout = HAL_GetTick() + 30000;
157 while (EINK_Busy())
158 {
159 if (HAL_GetTick() > timeout) break;
160 HAL_Delay(1);
161 }
162 }
163
164 static void EINK_Reset(void)
165 {
166 HAL_GPIO_WritePin(EINK_RES_GPIO_Port, EINK_RES_Pin, GPIO_PIN_RESET);
167 HAL_Delay(10);
168 HAL_GPIO_WritePin(EINK_RES_GPIO_Port, EINK_RES_Pin, GPIO_PIN_SET);
169 HAL_Delay(10);
170 }
171
172 static void EINK_SetRamArea(uint16_t x, uint16_t y, uint16_t w, uint16_t h)
173 {
174 EINK_SendCmd(0x11);
175 EINK_SendData(0x03);
176

```

```

177 EINK_SendCmd(0x44);
178 EINK_SendData(x / 8);
179 EINK_SendData((x + w - 1) / 8);
180
181 EINK_SendCmd(0x45);
182 EINK_SendData(y & 0xFF);
183 EINK_SendData((y >> 8) & 0xFF);
184 EINK_SendData((y + h - 1) & 0xFF);
185 EINK_SendData(((y + h - 1) >> 8) & 0xFF);
186
187 EINK_SendCmd(0x4E);
188 EINK_SendData(x / 8);
189
190 EINK_SendCmd(0x4F);
191 EINK_SendData(y & 0xFF);
192 EINK_SendData((y >> 8) & 0xFF);
193 }
194
195 static void EINK_WriteFB(uint8_t cmd)
196 {
197     EINK_SetRamArea(0, 0, e_w, e_h);
198     EINK_SendCmd(cmd);
199     EINK_SendDataBurst(fb, EINK_BUFF_SIZE);
200 }
201
202 bool EINK_Init(EINK_Config *cfg)
203 {
204     e_w = cfg->width;
205     e_h = cfg->height;
206
207     EINK_Reset();
208     HAL_Delay(10);
209
210     EINK_SendCmd(0x12);
211     HAL_Delay(10);
212     EINK_WaitBusy();
213
214     EINK_SendCmd(0x01);
215     EINK_SendData(0xC7);
216     EINK_SendData(0x00);
217     EINK_SendData(0x00);
218
219     EINK_SendCmd(0x3C);
220     EINK_SendData(0x05);
221
222     EINK_SendCmd(0x18);
223     EINK_SendData(0x80);
224
225     EINK_SetRamArea(0, 0, e_w, e_h);
226
227     memset(fb, 0xFF, sizeof(fb));
228
229     return true;
230 }
231
232 void EINK_Clear(void)
233 {
234     memset(fb, 0xFF, sizeof(fb));
235 }
236
237 void EINK_Update(void)
238 {
239     EINK_WakeInternal();
240
241     EINK_WriteFB(0x26);
242     EINK_WriteFB(0x24);
243
244     EINK_SendCmd(0x22);
245     EINK_SendData(0xF7);
246     EINK_SendCmd(0x20);
247     EINK_WaitBusy();
248 }
249
250 void EINK_Sleep(void)
251 {

```

```

252 EINK_SendCmd(0x22);
253 EINK_SendData(0x83);
254 EINK_SendCmd(0x20);
255 EINK_WaitBusy();
256
257 EINK_SendCmd(0x10);
258 EINK_SendData(0x01);
259 sleeping = true;
260 }
261
262 static void EINK_WakeInternal(void)
263 {
264     if (!sleeping) return;
265     EINK_Reset();
266     HAL_Delay(10);
267     EINK_SendCmd(0x12);
268     HAL_Delay(10);
269     EINK_WaitBusy();
270     EINK_SendCmd(0x01);
271     EINK_SendData(0xC7);
272     EINK_SendData(0x00);
273     EINK_SendData(0x00);
274     EINK_SendCmd(0x3C);
275     EINK_SendData(0x05);
276     EINK_SendCmd(0x18);
277     EINK_SendData(0x80);
278     EINK_SetRamArea(0, 0, e_w, e_h);
279     sleeping = false;
280 }
281
282 void EINK_DrawString(int x, int y, const char *str, uint8_t size)
283 {
284     while (*str)
285     {
286         if (size == 8 && x + 6 < e_w && y + 8 < e_h)
287         {
288             uint8_t c = (uint8_t)*str;
289             if (c < ' ' || c > '~') c = ' ';
290             c -= ' ';
291             for (uint8_t i = 0; i < 6; i++)
292             {
293                 uint8_t line = font_8x6[c][i];
294                 for (uint8_t m = 0; m < 8; m++)
295                 {
296                     if (line & 0x01)
297                     {
298                         uint16_t px = x + i;
299                         uint16_t py = y + m;
300                         if (px < e_w && py < e_h)
301                         {
302                             uint32_t addr = (px / 8) + py * EINK_W_BUFF_SIZE;
303                             fb[addr] &= ~(0x80 >> (px % 8));
304                         }
305                     }
306                     line >>= 1;
307                 }
308             }
309             x += 6;
310         }
311         else
312         {
313             x += 6;
314         }
315         str++;
316     }
317 }
318
319 void EINK_DrawStringScaled(int x, int y, const char *str, uint8_t scale)
320 {
321     if (scale == 0) return;
322
323     while (*str)
324     {
325         uint8_t c = (uint8_t)*str;
326         if (c < ' ' || c > '~') c = ' ';

```

```

327 c -= ' ';
328
329 for (uint8_t i = 0; i < 6; i++)
330 {
331     uint8_t col = font_8x6[c][i];
332     for (uint8_t m = 0; m < 8; m++)
333     {
334         if (col & 0x01)
335         {
336             uint16_t bx = (uint16_t)(x + i * scale);
337             uint16_t by = (uint16_t)(y + m * scale);
338             for (uint8_t sy = 0; sy < scale; sy++)
339             {
340                 for (uint8_t sx = 0; sx < scale; sx++)
341                 {
342                     uint16_t px = bx + sx;
343                     uint16_t py = by + sy;
344                     if (px < e_w && py < e_h)
345                     {
346                         uint32_t addr = (px / 8) + py * EINK_W_BUFF_SIZE;
347                         fb[addr] &= ~(0x80 >> (px % 8));
348                     }
349                 }
350             }
351         }
352         col >>= 1;
353     }
354 }
355 x += 6 * scale;
356 str++;
357 }
358 }
359
360 void EINK_LoadImage(const uint8_t *image)
361 {
362     memcpy(fb, image, EINK_BUFF_SIZE);
363 }
364

```

Sensor - Sources → [main.c](#)

Path: sensor/Core/Src/main.c | Lines: 603

```

1  /* USER CODE BEGIN Header */
2  /**
3   * *****
4   * @file : main.c
5   * @brief : Main program body
6   * *****
7   * @attention
8   *
9   * Copyright (c) 2026 STMicroelectronics.
10  * All rights reserved.
11  *
12  * This software is licensed under terms that can be found in the LICENSE file
13  * in the root directory of this software component.
14  * If no LICENSE file comes with this software, it is provided AS-IS.
15  *
16  * *****
17  */
18  /* USER CODE END Header */
19  /* Includes -----*/
20  #include <main.h>;
21
22  /* Private includes -----*/
23  /* USER CODE BEGIN Includes */
24  #include <bme280.h>;
25  #include <sx1278.h>;
26  #include <eink_display.h>;
27  #include <power_manager.h>;
28  #include <cmd.h>;
29  #include <stdio.h>;
30  #include <string.h>;
31  /* USER CODE END Includes */
32
33  /* Private typedef -----*/

```

```

34 /* USER CODE BEGIN PTD */
35
36 /* USER CODE END PTD */
37
38 /* Private define -----*/
39 /* USER CODE BEGIN PD */
40
41 /* USER CODE END PD */
42
43 /* Private macro -----*/
44 /* USER CODE BEGIN PM */
45
46 /* USER CODE END PM */
47
48 /* Private variables -----*/
49 I2C_HandleTypeDef hi2c3;
50
51 RTC_HandleTypeDef hrtc;
52
53 SPI_HandleTypeDef hspi1;
54 SPI_HandleTypeDef hspi2;
55
56 UART_HandleTypeDef huart2;
57
58 /* USER CODE BEGIN PV */
59
60 #define STATION_ID 0x01
61
62 static const char *lora_mode_label(SX1278_Config cfg)
63 {
64     switch (cfg) {
65     case SX1278_CFG_POWER: return "LoRa:Pwr";
66     case SX1278_CFG_BALANCED: return "LoRa:Bal";
67     case SX1278_CFG_PERFORMANCE: return "LoRa:Perf";
68     default: return "LoRa: ?";
69     }
70 }
71
72 BME280_HandleTypeDef bme280;
73 BME280_Data sensor_data;
74 char print_buf[128];
75
76 /* USER CODE END PV */
77
78 /* Private function prototypes -----*/
79 void SystemClock_Config(void);
80
81 static void MX_GPIO_Init(void);
82
83 static void MX_I2C3_Init(void);
84
85 static void MX_SPI1_Init(void);
86
87 static void MX_SPI2_Init(void);
88
89 static void MX_USART2_UART_Init(void);
90
91 static void MX_RTC_Init(void);
92
93 /* USER CODE BEGIN PFP */
94
95 /* USER CODE END PFP */
96
97 /* Private user code -----*/
98 /* USER CODE BEGIN 0 */
99
100 /* USER CODE END 0 */
101
102 /**
103  * @brief The application entry point.
104  * @retval int
105  */
106 int main(void)
107 {
108     /* USER CODE BEGIN 1 */

```

```

109
110 /* USER CODE END 1 */
111
112 /* MCU Configuration-----*/
113
114 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
115 HAL_Init();
116
117 /* USER CODE BEGIN Init */
118
119 /* USER CODE END Init */
120
121 /* Configure the system clock */
122 SystemClock_Config();
123
124 /* USER CODE BEGIN SysInit */
125
126 /* USER CODE END SysInit */
127
128 /* Initialize all configured peripherals */
129 MX_GPIO_Init();
130 MX_I2C3_Init();
131 MX_SPI1_Init();
132 MX_SPI2_Init();
133 MX_USART2_UART_Init();
134 MX_RTC_Init();
135 CMD_Init(&huart2);
136
137 HAL_UART_Transmit(&huart2, (uint8_t *) "cmd ok\r\n", 8, 1000);
138
139 PM_Init();
140 HAL_UART_Transmit(&huart2, (uint8_t *) "pm ok\r\n", 7, 1000);
141
142 if (!BME280_Init(&bme280, &hi2c3, BME280_I2C_ADDR_PRIM))
143 {
144     Error_Handler();
145 }
146 HAL_UART_Transmit(&huart2, (uint8_t *) "bme ok\r\n", 8, 1000);
147
148 if (!SX1278_Init(g_lora_cfg))
149 {
150     Error_Handler();
151 }
152
153 {
154     const char *mode = lora_mode_label(g_lora_cfg);
155     snprintf(print_buf, sizeof(print_buf), "lora:%s\r\n", mode);
156     HAL_UART_Transmit(&huart2, (uint8_t *) print_buf, strlen(print_buf), 1000);
157 }
158
159 {
160     EINK_Config eink_cfg;
161     eink_cfg.width = EINK_WIDTH;
162     eink_cfg.height = EINK_HEIGHT;
163     EINK_Init(&eink_cfg);
164 }
165 EINK_Clear();
166 EINK_DrawStringScaled((200 - 12*12)/2, 86, "Booting...", 2);
167 EINK_Update();
168 EINK_Sleep();
169 HAL_UART_Transmit(&huart2, (uint8_t *) "rdy\r\n", 5, 1000);
170
171 /* USER CODE END 2 */
172
173 /* Infinite loop */
174 /* USER CODE BEGIN WHILE */
175 while (1)
176 {
177     /* USER CODE END WHILE */
178
179     /* USER CODE BEGIN 3 */
180
181     BME280_ReadAll(&bme280, &sensor_data);
182     sensor_data.temperature += g_offset_temp;
183     sensor_data.humidity += g_offset_hum;

```

```

184 sensor_data.pressure += g_offset_pres;
185
186 const int t = (int) sensor_data.temperature;
187 const int t_d = (int) ((sensor_data.temperature - (float) t) * 10.0f);
188 const int h = (int) sensor_data.humidity;
189 int p_kpa = (int)(sensor_data.pressure / 1000.0f);
190 int p_dec = ((int)(sensor_data.pressure / 100.0f) % 10);
191
192 snprintf(print_buf, sizeof(print_buf),
193 "%s Temp:%d.%dC Hum:%d%% Pres:%d.%dkPa\r\n",
194 lora_mode_label(g_lora_cfg),
195 t, t_d < 0 ? -t_d : t_d, h, p_kpa, p_dec);
196 HAL_UART_Transmit(&huart2, (uint8_t *)print_buf, strlen(print_buf), 1000);
197
198 uint8_t payload[10];
199 {
200 int16_t te = (int16_t)(sensor_data.temperature * 10.0f);
201 int16_t he = (int16_t)(sensor_data.humidity * 10.0f);
202 uint16_t pe = (uint16_t)(sensor_data.pressure / 10.0f);
203
204 payload[0] = STATION_ID;
205 payload[1] = (te >> 8) & 0xFF;
206 payload[2] = te & 0xFF;
207 payload[3] = (he >> 8) & 0xFF;
208 payload[4] = he & 0xFF;
209 payload[5] = (pe >> 8) & 0xFF;
210 payload[6] = pe & 0xFF;
211 payload[7] = 0;
212 for (int i = 0; i < 7; i++) payload[7] ^= payload[i];
213 payload[8] = (uint8_t)(HAL_GetTick() & 0xFF);
214 payload[9] = 0xAA;
215 }
216
217 uint8_t ver = SX1278_ReadVersion();
218 uint8_t mode = SX1278_ReadReg(SX1278_REG_OP_MODE);
219 bool tx_ok = SX1278_SendPacket(payload, 10, g_lora_cfg);
220 snprintf(print_buf, sizeof(print_buf), "Ver:0x%02X Mode:0x%02X Tx:%s\r\n",
221 ver, mode, tx_ok ? "OK" : "FAIL");
222 HAL_UART_Transmit(&huart2, (uint8_t *)print_buf, strlen(print_buf), 1000);
223
224 {
225 int cx;
226 EINK_Clear();
227 snprintf(print_buf, sizeof(print_buf), "Station #01");
228 cx = (EINK_WIDTH - (int)strlen(print_buf) * 12) / 2;
229 EINK_DrawStringScaled(cx > 0 ? cx : 0, 8, print_buf, 2);
230 snprintf(print_buf, sizeof(print_buf), "Temp: %d.%d C",
231 t, t_d < 0 ? -t_d : t_d);
232 cx = (EINK_WIDTH - (int)strlen(print_buf) * 12) / 2;
233 EINK_DrawStringScaled(cx > 0 ? cx : 0, 36, print_buf, 2);
234 snprintf(print_buf, sizeof(print_buf), "Humidity: %d %%", h);
235 cx = (EINK_WIDTH - (int)strlen(print_buf) * 12) / 2;
236 EINK_DrawStringScaled(cx > 0 ? cx : 0, 62, print_buf, 2);
237 snprintf(print_buf, sizeof(print_buf), "Pres: %d.%d kPa", p_kpa, p_dec);
238 cx = (EINK_WIDTH - (int)strlen(print_buf) * 12) / 2;
239 EINK_DrawStringScaled(cx > 0 ? cx : 0, 88, print_buf, 2);
240 snprintf(print_buf, sizeof(print_buf), "%s", lora_mode_label(g_lora_cfg));
241 cx = (EINK_WIDTH - (int)strlen(print_buf) * 12) / 2;
242 EINK_DrawStringScaled(cx > 0 ? cx : 0, 114, print_buf, 2);
243 }
244 EINK_Update();
245 EINK_Sleep();
246
247 CMD_Poll();
248
249 {
250 uint32_t wait_end = HAL_GetTick() + g_interval_ms;
251 while (HAL_GetTick() < wait_end) {
252 CMD_Poll();
253 if (CMD_ConsumeDirty()) break;
254 HAL_Delay(100);
255 }
256 }
257 }
258 /* USER CODE END 3 */

```

```

259 }
260
261 /**
262  * @brief System Clock Configuration
263  * @retval None
264  */
265 void SystemClock_Config(void)
266 {
267     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
268     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
269     RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};
270
271     /** Configure the main internal regulator output voltage
272     */
273     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
274
275     /** Initializes the RCC Oscillators according to the specified parameters
276     * in the RCC_OscInitTypeDef structure.
277     */
278     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSI | RCC_OSCILLATORTYPE_MSI;
279     RCC_OscInitStruct.LSIState = RCC_LSI_ON;
280     RCC_OscInitStruct.MSIState = RCC_MSI_ON;
281     RCC_OscInitStruct.MSICalibrationValue = 0;
282     RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_5;
283     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
284     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
285     {
286         Error_Handler();
287     }
288
289     /** Initializes the CPU, AHB and APB buses clocks
290     */
291     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
292     | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
293     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
294     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
295     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
296     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
297
298     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
299     {
300         Error_Handler();
301     }
302     PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_USART2 | RCC_PERIPHCLK_I2C3
303     | RCC_PERIPHCLK_RTC;
304     PeriphClkInit.Usart2ClockSelection = RCC_USART2CLKSOURCE_PCLK1;
305     PeriphClkInit.I2c3ClockSelection = RCC_I2C3CLKSOURCE_PCLK1;
306     PeriphClkInit.RTCClockSelection = RCC_RTCCLKSOURCE_LSI;
307     if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
308     {
309         Error_Handler();
310     }
311 }
312
313 /**
314  * @brief I2C3 Initialization Function
315  * @param None
316  * @retval None
317  */
318 static void MX_I2C3_Init(void)
319 {
320     /* USER CODE BEGIN I2C3_Init 0 */
321
322     /* USER CODE END I2C3_Init 0 */
323
324     /* USER CODE BEGIN I2C3_Init 1 */
325
326     /* USER CODE END I2C3_Init 1 */
327     hi2c3.Instance = I2C3;
328     hi2c3.Init.Timing = 0x00000608;
329     hi2c3.Init.OwnAddress1 = 0;
330     hi2c3.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
331     hi2c3.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
332     hi2c3.Init.OwnAddress2 = 0;
333     hi2c3.Init.OwnAddress2Masks = I2C_OA2_NOMASK;

```

```

334 hi2c3.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
335 hi2c3.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
336 if (HAL_I2C_Init(&hi2c3) != HAL_OK)
337 {
338     Error_Handler();
339 }
340
341 /** Configure Analogue filter
342 */
343 if (HAL_I2CEx_ConfigAnalogFilter(&hi2c3, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
344 {
345     Error_Handler();
346 }
347
348 /** Configure Digital filter
349 */
350 if (HAL_I2CEx_ConfigDigitalFilter(&hi2c3, 0) != HAL_OK)
351 {
352     Error_Handler();
353 }
354 /* USER CODE BEGIN I2C3_Init 2 */
355
356 /* USER CODE END I2C3_Init 2 */
357 }
358
359 /**
360  * @brief RTC Initialization Function
361  * @param None
362  * @retval None
363  */
364 static void MX_RTC_Init(void)
365 {
366     /* USER CODE BEGIN RTC_Init 0 */
367
368     /* USER CODE END RTC_Init 0 */
369
370     /* USER CODE BEGIN RTC_Init 1 */
371
372     /* USER CODE END RTC_Init 1 */
373
374     /** Initialize RTC Only
375     */
376     hrtc.Instance = RTC;
377     hrtc.Init.HourFormat = RTC_HOURFORMAT_24;
378     hrtc.Init.AsynchPrediv = 127;
379     hrtc.Init.SynchPrediv = 255;
380     hrtc.Init.OutPut = RTC_OUTPUT_DISABLE;
381     hrtc.Init.OutPutRemap = RTC_OUTPUT_REMAP_NONE;
382     hrtc.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
383     hrtc.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;
384     if (HAL_RTC_Init(&hrtc) != HAL_OK)
385     {
386         Error_Handler();
387     }
388
389     /** Enable the WakeUp
390     */
391     if (HAL_RTCEx_SetWakeUpTimer(&hrtc, 0, RTC_WAKEUPCLOCK_RTCCLK_DIV16) != HAL_OK)
392     {
393         Error_Handler();
394     }
395     /* USER CODE BEGIN RTC_Init 2 */
396
397     /* USER CODE END RTC_Init 2 */
398 }
399
400 /**
401  * @brief SPI1 Initialization Function
402  * @param None
403  * @retval None
404  */
405 static void MX_SPI1_Init(void)
406 {
407     /* USER CODE BEGIN SPI1_Init 0 */
408

```

```

409 /* USER CODE END SPI1_Init 0 */
410
411 /* USER CODE BEGIN SPI1_Init 1 */
412
413 /* USER CODE END SPI1_Init 1 */
414 /* SPI1 parameter configuration*/
415 hspi1.Instance = SPI1;
416 hspi1.Init.Mode = SPI_MODE_MASTER;
417 hspi1.Init.Direction = SPI_DIRECTION_2LINES;
418 hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
419 hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
420 hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;
421 hspi1.Init.NSS = SPI_NSS_SOFT;
422 hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_2;
423 hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
424 hspi1.Init.TIMode = SPI_TIMODE_DISABLE;
425 hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
426 hspi1.Init.CRCPolynomial = 7;
427 if (HAL_SPI_Init(&hspi1) != HAL_OK)
428 {
429     Error_Handler();
430 }
431 /* USER CODE BEGIN SPI1_Init 2 */
432
433 /* USER CODE END SPI1_Init 2 */
434 }
435
436 /**
437  * @brief SPI2 Initialization Function
438  * @param None
439  * @retval None
440  */
441 static void MX_SPI2_Init(void)
442 {
443     /* USER CODE BEGIN SPI2_Init 0 */
444
445     /* USER CODE END SPI2_Init 0 */
446
447     /* USER CODE BEGIN SPI2_Init 1 */
448
449     /* USER CODE END SPI2_Init 1 */
450     /* SPI2 parameter configuration*/
451     hspi2.Instance = SPI2;
452     hspi2.Init.Mode = SPI_MODE_MASTER;
453     hspi2.Init.Direction = SPI_DIRECTION_2LINES;
454     hspi2.Init.DataSize = SPI_DATASIZE_8BIT;
455     hspi2.Init.CLKPolarity = SPI_POLARITY_LOW;
456     hspi2.Init.CLKPhase = SPI_PHASE_1EDGE;
457     hspi2.Init.NSS = SPI_NSS_SOFT;
458     hspi2.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_2;
459     hspi2.Init.FirstBit = SPI_FIRSTBIT_MSB;
460     hspi2.Init.TIMode = SPI_TIMODE_DISABLE;
461     hspi2.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
462     hspi2.Init.CRCPolynomial = 7;
463     if (HAL_SPI_Init(&hspi2) != HAL_OK)
464     {
465         Error_Handler();
466     }
467     /* USER CODE BEGIN SPI2_Init 2 */
468
469     /* USER CODE END SPI2_Init 2 */
470 }
471
472 /**
473  * @brief USART2 Initialization Function
474  * @param None
475  * @retval None
476  */
477 static void MX_USART2_UART_Init(void)
478 {
479     /* USER CODE BEGIN USART2_Init 0 */
480
481     /* USER CODE END USART2_Init 0 */
482
483     /* USER CODE BEGIN USART2_Init 1 */

```

```

484
485 /* USER CODE END USART2_Init 1 */
486 huart2.Instance = USART2;
487 huart2.Init.BaudRate = 9600;
488 huart2.Init.WordLength = UART_WORDLENGTH_8B;
489 huart2.Init.StopBits = UART_STOPBITS_1;
490 huart2.Init.Parity = UART_PARITY_NONE;
491 huart2.Init.Mode = UART_MODE_TX_RX;
492 huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
493 huart2.Init.OverSampling = UART_OVERSAMPLING_16;
494 huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
495 huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
496 if (HAL_UART_Init(&huart2) != HAL_OK)
497 {
498     Error_Handler();
499 }
500 /* USER CODE BEGIN USART2_Init 2 */
501
502 /* USER CODE END USART2_Init 2 */
503 }
504
505 /**
506  * @brief GPIO Initialization Function
507  * @param None
508  * @retval None
509  */
510 static void MX_GPIO_Init(void)
511 {
512     GPIO_InitTypeDef GPIO_InitStruct = {0};
513     /* USER CODE BEGIN MX_GPIO_Init_1 */
514
515     /* USER CODE END MX_GPIO_Init_1 */
516
517     /* GPIO Ports Clock Enable */
518     __HAL_RCC_GPIOC_CLK_ENABLE();
519     __HAL_RCC_GPIOA_CLK_ENABLE();
520     __HAL_RCC_GPIOB_CLK_ENABLE();
521
522     /*Configure GPIO pin Output Level */
523     HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
524
525     /*Configure GPIO pin Output Level */
526     HAL_GPIO_WritePin(GPIOC, LORA_RST_Pin | EINK_CS_Pin | EINK_RES_Pin, GPIO_PIN_SET);
527
528     /*Configure GPIO pin Output Level */
529     HAL_GPIO_WritePin(EINK_D_C_GPIO_Port, EINK_D_C_Pin, GPIO_PIN_SET);
530
531     /*Configure GPIO pin : LORA_NSS_Pin */
532     GPIO_InitStruct.Pin = LORA_NSS_Pin;
533     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
534     GPIO_InitStruct.Pull = GPIO_PULLUP;
535     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
536     HAL_GPIO_Init(LORA_NSS_GPIO_Port, &GPIO_InitStruct);
537
538     /*Configure GPIO pin : LORA_DIO0_Pin */
539     GPIO_InitStruct.Pin = LORA_DIO0_Pin;
540     GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
541     GPIO_InitStruct.Pull = GPIO_PULLDOWN;
542     HAL_GPIO_Init(LORA_DIO0_GPIO_Port, &GPIO_InitStruct);
543
544     /*Configure GPIO pins : LORA_RST_Pin EINK_CS_Pin EINK_RES_Pin */
545     GPIO_InitStruct.Pin = LORA_RST_Pin | EINK_CS_Pin | EINK_RES_Pin;
546     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
547     GPIO_InitStruct.Pull = GPIO_PULLUP;
548     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
549     HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
550
551     /*Configure GPIO pin : EINK_D_C_Pin */
552     GPIO_InitStruct.Pin = EINK_D_C_Pin;
553     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
554     GPIO_InitStruct.Pull = GPIO_PULLUP;
555     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
556     HAL_GPIO_Init(EINK_D_C_GPIO_Port, &GPIO_InitStruct);
557
558     /*Configure GPIO pin : EINK_BUSY_Pin */

```

```

559 GPIO_InitStruct.Pin = EINK_BUSY_Pin;
560 GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
561 GPIO_InitStruct.Pull = GPIO_NOPULL;
562 HAL_GPIO_Init(EINK_BUSY_GPIO_Port, &GPIO_InitStruct);
563
564 /* USER CODE BEGIN MX_GPIO_Init_2 */
565
566 /* USER CODE END MX_GPIO_Init_2 */
567 }
568
569 /* USER CODE BEGIN 4 */
570
571 /* USER CODE END 4 */
572
573 /**
574 * @brief This function is executed in case of error occurrence.
575 * @retval None
576 */
577 void Error_Handler(void)
578 {
579 /* USER CODE BEGIN Error_Handler_Debug */
580 /* User can add his own implementation to report the HAL error return state */
581 __disable_irq();
582 while (1)
583 {
584 }
585 /* USER CODE END Error_Handler_Debug */
586 }
587 #ifdef USE_FULL_ASSERT
588 /**
589 * @brief Reports the name of the source file and the source line number
590 * where the assert_param error has occurred.
591 * @param file: pointer to the source file name
592 * @param line: assert_param error line source number
593 * @retval None
594 */
595 void assert_failed(uint8_t *file, uint32_t line)
596 {
597 /* USER CODE BEGIN 6 */
598 /* User can add his own implementation to report the file name and line number,
599 ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
600 /* USER CODE END 6 */
601 }
602 #endif /* USE_FULL_ASSERT */
603

```

Sensor - Sources → [power_manager.c](#)

Path: sensor/Core/Src/power_manager.c | Lines: 51

```

1 #include <power_manager.h>;
2
3 extern RTC_HandleTypeDef hrtc;
4 extern void SystemClock_Config(void);
5
6 void PM_Init(void)
7 {
8 }
9
10 static void PM_ConfigureGPIOForSleep(void)
11 {
12 GPIO_InitTypeDef GPIO_InitStruct = {0};
13
14 __HAL_RCC_GPIOA_CLK_ENABLE();
15 __HAL_RCC_GPIOB_CLK_ENABLE();
16 __HAL_RCC_GPIOC_CLK_ENABLE();
17
18 GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
19 GPIO_InitStruct.Pull = GPIO_NOPULL;
20
21 GPIO_InitStruct.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_8 | GPIO_PIN_9 |
22 GPIO_PIN_10 | GPIO_PIN_11 | GPIO_PIN_15;
23 HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
24
25 GPIO_InitStruct.Pin = 0xFFFF & ~(EINK_D_C_Pin | EINK_BUSY_Pin);
26 HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

```

```

27
28 GPIO_InitStruct.Pin = 0xFFFF & ~(LORA_DI00_Pin | LORA_RST_Pin | EINK_CS_Pin | EINK_RES_Pin);
29 HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
30
31 HAL_SuspendTick();
32 }
33
34 void PM_EnterStop(uint32_t seconds)
35 {
36 PM_ConfigureGPIOForSleep();
37
38 HAL_RTCEX_DeactivateWakeUpTimer(&hrtc);
39
40 HAL_RTCEX_SetWakeUpTimer(&hrtc, (uint16_t)seconds,
41 RTC_WAKEUPCLOCK_CK_SPRE_16BITS);
42
43 __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
44 HAL_PWREx_EnableUltraLowPower();
45 HAL_PWREx_EnableFastWakeUp();
46 HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
47
48 HAL_ResumeTick();
49 SystemClock_Config();
50 }
51

```

Sensor - Sources → [stm32l0xx_hal_msp.c](#)

Path: sensor/Core/Src/stm32l0xx_hal_msp.c | Lines: 370

```

1  /* USER CODE BEGIN Header */
2  /**
3  ****
4  * @file stm32l0xx_hal_msp.c
5  * @brief This file provides code for the MSP Initialization
6  * and de-Initialization codes.
7  ****
8  * @attention
9  *
10 * Copyright (c) 2026 STMicroelectronics.
11 * All rights reserved.
12 *
13 * This software is licensed under terms that can be found in the LICENSE file
14 * in the root directory of this software component.
15 * If no LICENSE file comes with this software, it is provided AS-IS.
16 *
17 ****
18 */
19 /* USER CODE END Header */
20 /* Includes -----*/
21 #include "main.h";
22 /* USER CODE BEGIN Includes */
23
24 /* USER CODE END Includes */
25
26 /* Private typedef -----*/
27 /* USER CODE BEGIN TD */
28
29 /* USER CODE END TD */
30
31 /* Private define -----*/
32 /* USER CODE BEGIN Define */
33
34 /* USER CODE END Define */
35
36 /* Private macro -----*/
37 /* USER CODE BEGIN Macro */
38
39 /* USER CODE END Macro */
40
41 /* Private variables -----*/
42 /* USER CODE BEGIN PV */
43
44 /* USER CODE END PV */
45
46 /* Private function prototypes -----*/

```

```

47 /* USER CODE BEGIN PFP */
48
49 /* USER CODE END PFP */
50
51 /* External functions -----*/
52 /* USER CODE BEGIN ExternalFunctions */
53
54 /* USER CODE END ExternalFunctions */
55
56 /* USER CODE BEGIN 0 */
57
58 /* USER CODE END 0 */
59 /**
60 * Initializes the Global MSP.
61 */
62 void HAL_MspInit(void)
63 {
64
65 /* USER CODE BEGIN MspInit 0 */
66
67 /* USER CODE END MspInit 0 */
68
69 __HAL_RCC_SYSCFG_CLK_ENABLE();
70 __HAL_RCC_PWR_CLK_ENABLE();
71
72 /* System interrupt init*/
73
74 /* USER CODE BEGIN MspInit 1 */
75
76 /* USER CODE END MspInit 1 */
77 }
78
79 /**
80 * @brief I2C MSP Initialization
81 * This function configures the hardware resources used in this example
82 * @param hi2c: I2C handle pointer
83 * @retval None
84 */
85 void HAL_I2C_MspInit(I2C_HandleTypeDef* hi2c)
86 {
87 GPIO_InitTypeDef GPIO_InitStruct = {0};
88 if(hi2c->Instance==I2C3)
89 {
90 /* USER CODE BEGIN I2C3_MspInit 0 */
91
92 /* USER CODE END I2C3_MspInit 0 */
93
94 __HAL_RCC_GPIOC_CLK_ENABLE();
95 /**I2C3 GPIO Configuration
96 PC0 -----> I2C3_SCL
97 PC1 -----> I2C3_SDA
98 */
99 GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
100 GPIO_InitStruct.Mode = GPIO_MODE_AF_OD;
101 GPIO_InitStruct.Pull = GPIO_NOPULL;
102 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
103 GPIO_InitStruct.Alternate = GPIO_AF7_I2C3;
104 HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
105
106 /* Peripheral clock enable */
107 __HAL_RCC_I2C3_CLK_ENABLE();
108 /* USER CODE BEGIN I2C3_MspInit 1 */
109
110 /* USER CODE END I2C3_MspInit 1 */
111
112 }
113
114 }
115
116 /**
117 * @brief I2C MSP De-Initialization
118 * This function freeze the hardware resources used in this example
119 * @param hi2c: I2C handle pointer
120 * @retval None

```

```

121 */
122 void HAL_I2C_MspDeInit(I2C_HandleTypeDef* hi2c)
123 {
124     if(hi2c->Instance==I2C3)
125     {
126         /* USER CODE BEGIN I2C3_MspDeInit 0 */
127
128         /* USER CODE END I2C3_MspDeInit 0 */
129         /* Peripheral clock disable */
130         __HAL_RCC_I2C3_CLK_DISABLE();
131
132         /**I2C3 GPIO Configuration
133         PC0 -----> I2C3_SCL
134         PC1 -----> I2C3_SDA
135         */
136         HAL_GPIO_DeInit(GPIOC, GPIO_PIN_0);
137
138         HAL_GPIO_DeInit(GPIOC, GPIO_PIN_1);
139
140         /* USER CODE BEGIN I2C3_MspDeInit 1 */
141
142         /* USER CODE END I2C3_MspDeInit 1 */
143     }
144 }
145 }
146
147 /**
148  * @brief RTC MSP Initialization
149  * This function configures the hardware resources used in this example
150  * @param hrtc: RTC handle pointer
151  * @retval None
152  */
153 void HAL_RTC_MspInit(RTC_HandleTypeDef* hrtc)
154 {
155     if(hrtc->Instance==RTC)
156     {
157         /* USER CODE BEGIN RTC_MspInit 0 */
158
159         /* USER CODE END RTC_MspInit 0 */
160         /* Peripheral clock enable */
161         __HAL_RCC_RTC_ENABLE();
162         /* USER CODE BEGIN RTC_MspInit 1 */
163
164         /* USER CODE END RTC_MspInit 1 */
165
166     }
167 }
168 }
169
170 /**
171  * @brief RTC MSP De-Initialization
172  * This function freeze the hardware resources used in this example
173  * @param hrtc: RTC handle pointer
174  * @retval None
175  */
176 void HAL_RTC_MspDeInit(RTC_HandleTypeDef* hrtc)
177 {
178     if(hrtc->Instance==RTC)
179     {
180         /* USER CODE BEGIN RTC_MspDeInit 0 */
181
182         /* USER CODE END RTC_MspDeInit 0 */
183         /* Peripheral clock disable */
184         __HAL_RCC_RTC_DISABLE();
185         /* USER CODE BEGIN RTC_MspDeInit 1 */
186
187         /* USER CODE END RTC_MspDeInit 1 */
188     }
189 }
190 }
191
192 /**
193  * @brief SPI MSP Initialization
194  * This function configures the hardware resources used in this example
195  * @param hspi: SPI handle pointer
196  * @retval None

```

```

197 */
198 void HAL_SPI_MspInit(SPI_HandleTypeDef* hspi)
199 {
200   GPIO_InitTypeDef GPIO_InitStruct = {0};
201   if(hspi->Instance==SPI1)
202   {
203     /* USER CODE BEGIN SPI1_MspInit 0 */
204
205     /* USER CODE END SPI1_MspInit 0 */
206     /* Peripheral clock enable */
207     __HAL_RCC_SPI1_CLK_ENABLE();
208
209     __HAL_RCC_GPIOA_CLK_ENABLE();
210     /**SPI1 GPIO Configuration
211     PA5 -----> SPI1_SCK
212     PA6 -----> SPI1_MISO
213     PA7 -----> SPI1_MOSI
214     */
215     GPIO_InitStruct.Pin = GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
216     GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
217     GPIO_InitStruct.Pull = GPIO_NOPULL;
218     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
219     GPIO_InitStruct.Alternate = GPIO_AF0_SPI1;
220     HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
221
222     /* USER CODE BEGIN SPI1_MspInit 1 */
223
224     /* USER CODE END SPI1_MspInit 1 */
225   }
226   else if(hspi->Instance==SPI2)
227   {
228     /* USER CODE BEGIN SPI2_MspInit 0 */
229
230     /* USER CODE END SPI2_MspInit 0 */
231     /* Peripheral clock enable */
232     __HAL_RCC_SPI2_CLK_ENABLE();
233
234     __HAL_RCC_GPIOB_CLK_ENABLE();
235     /**SPI2 GPIO Configuration
236     PB13 -----> SPI2_SCK
237     PB15 -----> SPI2_MOSI
238     */
239     GPIO_InitStruct.Pin = GPIO_PIN_13|GPIO_PIN_15;
240     GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
241     GPIO_InitStruct.Pull = GPIO_NOPULL;
242     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
243     GPIO_InitStruct.Alternate = GPIO_AF0_SPI2;
244     HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
245
246     /* USER CODE BEGIN SPI2_MspInit 1 */
247
248     /* USER CODE END SPI2_MspInit 1 */
249   }
250 }
251 }
252
253 /**
254  * @brief SPI MSP De-Initialization
255  * This function freeze the hardware resources used in this example
256  * @param hspi: SPI handle pointer
257  * @retval None
258  */
259 void HAL_SPI_MspDeInit(SPI_HandleTypeDef* hspi)
260 {
261   if(hspi->Instance==SPI1)
262   {
263     /* USER CODE BEGIN SPI1_MspDeInit 0 */
264
265     /* USER CODE END SPI1_MspDeInit 0 */
266     /* Peripheral clock disable */
267     __HAL_RCC_SPI1_CLK_DISABLE();
268
269     /**SPI1 GPIO Configuration
270     PA5 -----> SPI1_SCK
271     PA6 -----> SPI1_MISO

```

```

272 PA7 -----> SPI1_MOSI
273 */
274 HAL_GPIO_DeInit(GPIOA, GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7);
275
276 /* USER CODE BEGIN SPI1_MspDeInit 1 */
277
278 /* USER CODE END SPI1_MspDeInit 1 */
279 }
280 else if(hspi->Instance==SPI2)
281 {
282 /* USER CODE BEGIN SPI2_MspDeInit 0 */
283
284 /* USER CODE END SPI2_MspDeInit 0 */
285 /* Peripheral clock disable */
286 __HAL_RCC_SPI2_CLK_DISABLE();
287
288 /**SPI2 GPIO Configuration
289 PB13 -----> SPI2_SCK
290 PB15 -----> SPI2_MOSI
291 */
292 HAL_GPIO_DeInit(GPIOB, GPIO_PIN_13|GPIO_PIN_15);
293
294 /* USER CODE BEGIN SPI2_MspDeInit 1 */
295
296 /* USER CODE END SPI2_MspDeInit 1 */
297 }
298
299 }
300
301 /**
302 * @brief UART MSP Initialization
303 * This function configures the hardware resources used in this example
304 * @param huart: UART handle pointer
305 * @retval None
306 */
307 void HAL_UART_MspInit(UART_HandleTypeDef* huart)
308 {
309 GPIO_InitTypeDef GPIO_InitStruct = {0};
310 if(huart->Instance==USART2)
311 {
312 /* USER CODE BEGIN USART2_MspInit 0 */
313
314 /* USER CODE END USART2_MspInit 0 */
315 /* Peripheral clock enable */
316 __HAL_RCC_USART2_CLK_ENABLE();
317
318 __HAL_RCC_GPIOA_CLK_ENABLE();
319 /**USART2 GPIO Configuration
320 PA2 -----> USART2_TX
321 PA3 -----> USART2_RX
322 */
323 GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3;
324 GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
325 GPIO_InitStruct.Pull = GPIO_NOPULL;
326 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
327 GPIO_InitStruct.Alternate = GPIO_AF4_USART2;
328 HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
329
330 /* USER CODE BEGIN USART2_MspInit 1 */
331
332 /* USER CODE END USART2_MspInit 1 */
333
334 }
335
336 }
337
338 /**
339 * @brief UART MSP De-Initialization
340 * This function freeze the hardware resources used in this example
341 * @param huart: UART handle pointer
342 * @retval None
343 */
344 void HAL_UART_MspDeInit(UART_HandleTypeDef* huart)
345 {
346 if(huart->Instance==USART2)

```

```

347 {
348 /* USER CODE BEGIN USART2_MspDeInit 0 */
349
350 /* USER CODE END USART2_MspDeInit 0 */
351 /* Peripheral clock disable */
352 __HAL_RCC_USART2_CLK_DISABLE();
353
354 /**USART2 GPIO Configuration
355 PA2 -----> USART2_TX
356 PA3 -----> USART2_RX
357 */
358 HAL_GPIO_DeInit(GPIOA, GPIO_PIN_2|GPIO_PIN_3);
359
360 /* USER CODE BEGIN USART2_MspDeInit 1 */
361
362 /* USER CODE END USART2_MspDeInit 1 */
363 }
364
365 }
366
367 /* USER CODE BEGIN 1 */
368
369 /* USER CODE END 1 */
370

```

Sensor - Sources → [stm32l0xx_it.c](#)

Path: sensor/Core/Src/stm32l0xx_it.c | Lines: 151

```

1 /* USER CODE BEGIN Header */
2 /**
3 *****
4 * @file stm32l0xx_it.c
5 * @brief Interrupt Service Routines.
6 *****
7 * @attention
8 *
9 * Copyright (c) 2026 STMicroelectronics.
10 * All rights reserved.
11 *
12 * This software is licensed under terms that can be found in the LICENSE file
13 * in the root directory of this software component.
14 * If no LICENSE file comes with this software, it is provided AS-IS.
15 *
16 *****
17 */
18 /* USER CODE END Header */
19
20 /* Includes -----*/
21 #include "main.h"
22 #include "stm32l0xx_it.h"
23 /* Private includes -----*/
24 /* USER CODE BEGIN Includes */
25 /* USER CODE END Includes */
26
27 /* Private typedef -----*/
28 /* USER CODE BEGIN TD */
29
30 /* USER CODE END TD */
31
32 /* Private define -----*/
33 /* USER CODE BEGIN PD */
34
35 /* USER CODE END PD */
36
37 /* Private macro -----*/
38 /* USER CODE BEGIN PM */
39
40 /* USER CODE END PM */
41
42 /* Private variables -----*/
43 /* USER CODE BEGIN PV */
44
45 /* USER CODE END PV */
46
47 /* Private function prototypes -----*/

```

```

48 /* USER CODE BEGIN PFP */
49
50 /* USER CODE END PFP */
51
52 /* Private user code -----*/
53 /* USER CODE BEGIN 0 */
54
55 /* USER CODE END 0 */
56
57 /* External variables -----*/
58
59 /* USER CODE BEGIN EV */
60
61 /* USER CODE END EV */
62
63 /*****
64  * Cortex-M0+ Processor Interruption and Exception Handlers */
65  *****/
66 /**
67  * @brief This function handles Non maskable Interrupt.
68  */
69 void NMI_Handler(void)
70 {
71 /* USER CODE BEGIN NonMaskableInt_IRQn 0 */
72
73 /* USER CODE END NonMaskableInt_IRQn 0 */
74 /* USER CODE BEGIN NonMaskableInt_IRQn 1 */
75 while (1)
76 {
77 }
78 /* USER CODE END NonMaskableInt_IRQn 1 */
79 }
80
81 /**
82  * @brief This function handles Hard fault interrupt.
83  */
84 void HardFault_Handler(void)
85 {
86 /* USER CODE BEGIN HardFault_IRQn 0 */
87
88 /* USER CODE END HardFault_IRQn 0 */
89 while (1)
90 {
91 /* USER CODE BEGIN W1_HardFault_IRQn 0 */
92 /* USER CODE END W1_HardFault_IRQn 0 */
93 }
94 }
95
96 /**
97  * @brief This function handles System service call via SWI instruction.
98  */
99 void SVC_Handler(void)
100 {
101 /* USER CODE BEGIN SVC_IRQn 0 */
102
103 /* USER CODE END SVC_IRQn 0 */
104 /* USER CODE BEGIN SVC_IRQn 1 */
105
106 /* USER CODE END SVC_IRQn 1 */
107 }
108
109 /**
110  * @brief This function handles Pendable request for system service.
111  */
112 void PendSV_Handler(void)
113 {
114 /* USER CODE BEGIN PendSV_IRQn 0 */
115
116 /* USER CODE END PendSV_IRQn 0 */
117 /* USER CODE BEGIN PendSV_IRQn 1 */
118
119 /* USER CODE END PendSV_IRQn 1 */
120 }
121
122 /**

```

```

123 * @brief This function handles System tick timer.
124 */
125 void SysTick_Handler(void)
126 {
127 /* USER CODE BEGIN SysTick_IRQn 0 */
128
129 /* USER CODE END SysTick_IRQn 0 */
130 HAL_IncTick();
131 /* USER CODE BEGIN SysTick_IRQn 1 */
132
133 /* USER CODE END SysTick_IRQn 1 */
134 }
135
136 /*****
137  * STM32L0xx Peripheral Interrupt Handlers */
138  * Add here the Interrupt Handlers for the used peripherals. */
139  * For the available peripheral interrupt handler names, */
140  * please refer to the startup file (startup_stm32l0xx.s). */
141  *****/
142
143 /* USER CODE BEGIN 1 */
144
145 void EXTI4_15_IRQHandler(void)
146 {
147 HAL_GPIO_EXTI_IRQHandler(LORA_DI00_Pin);
148 }
149
150 /* USER CODE END 1 */
151

```

Sensor - Sources → [sx1278.c](#)

Path: sensor/Core/Src/sx1278.c | Lines: 183

```

1 #include <sx1278.h>;
2
3 extern SPI_HandleTypeDef hspi1;
4
5 volatile bool sx1278_tx_done = false;
6
7 static void SX1278_CS_LOW(void)
8 {
9 HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_RESET);
10 }
11
12 static void SX1278_CS_HIGH(void)
13 {
14 HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
15 }
16
17 uint8_t SX1278_ReadReg(uint8_t reg)
18 {
19 uint8_t addr = reg & 0x7F;
20 uint8_t val;
21 SX1278_CS_LOW();
22 HAL_SPI_Transmit(&hspi1, &addr, 1, 100);
23 HAL_SPI_Receive(&hspi1, &val, 1, 100);
24 SX1278_CS_HIGH();
25 return val;
26 }
27
28 void SX1278_WriteReg(uint8_t reg, uint8_t val)
29 {
30 uint8_t buf[2];
31 buf[0] = reg | 0x80;
32 buf[1] = val;
33 SX1278_CS_LOW();
34 HAL_SPI_Transmit(&hspi1, buf, 2, 100);
35 SX1278_CS_HIGH();
36 }
37
38 static void SX1278_WriteBurst(uint8_t reg, uint8_t *data, uint8_t len)
39 {
40 uint8_t addr = reg | 0x80;
41 SX1278_CS_LOW();
42 HAL_SPI_Transmit(&hspi1, &addr, 1, 100);

```

```

43 HAL_SPI_Transmit(&hspi1, data, len, 100);
44 SX1278_CS_HIGH();
45 }
46
47 void SX1278_HardReset(void)
48 {
49 HAL_GPIO_WritePin(LORA_RST_GPIO_Port, LORA_RST_Pin, GPIO_PIN_RESET);
50 HAL_Delay(10);
51 HAL_GPIO_WritePin(LORA_RST_GPIO_Port, LORA_RST_Pin, GPIO_PIN_SET);
52 HAL_Delay(10);
53 }
54
55 static bool SX1278_CheckVersion(void)
56 {
57 uint8_t ver = SX1278_ReadReg(SX1278_REG_VERSION);
58 return (ver == 0x12);
59 }
60
61 typedef struct {
62 uint8_t pa_config;
63 uint8_t pa_dac;
64 uint8_t ocp;
65 uint8_t lna;
66 uint8_t modem_cfg1;
67 uint8_t modem_cfg2;
68 uint8_t modem_cfg3;
69 } SX1278_ConfigTable;
70
71 static const SX1278_ConfigTable cfg_table[SX1278_CFG_COUNT] = {
72 [SX1278_CFG_POWER] = { 0x8C, 0x00, 0x0B, 0x23, 0x72, 0x74, 0x04 },
73 [SX1278_CFG_BALANCED] = { 0xCF, 0x00, 0x0B, 0x23, 0x72, 0x94, 0x04 },
74 [SX1278_CFG_PERFORMANCE] = { 0xFF, 0x87, 0x1F, 0x23, 0x82, 0xC4, 0x04 },
75 };
76
77 bool SX1278_Init(SX1278_Config cfg)
78 {
79 if (cfg >= SX1278_CFG_COUNT) return false;
80
81 SX1278_HardReset();
82
83 bool ok = SX1278_CheckVersion();
84 if (!ok) return false;
85
86 SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_SLEEP);
87 HAL_Delay(10);
88
89 SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_STDBY);
90
91 uint64_t frf = ((uint64_t)433000000 << 19) / SX1278_FXOSC;
92 SX1278_WriteReg(SX1278_REG_FRF_MSB, (frf >> 16) & 0xFF);
93 SX1278_WriteReg(SX1278_REG_FRF_MID, (frf >> 8) & 0xFF);
94 SX1278_WriteReg(SX1278_REG_FRF_LSB, frf & 0xFF);
95
96 const SX1278_ConfigTable *c = &cfg_table[cfg];
97 SX1278_WriteReg(SX1278_REG_PA_CONFIG, c->pa_config);
98 SX1278_WriteReg(SX1278_REG_PA_RAMP, 0x09);
99 SX1278_WriteReg(SX1278_REG_OCP, c->ocp);
100 SX1278_WriteReg(SX1278_REG_LNA, c->lna);
101 if (c->pa_dac) {
102 SX1278_WriteReg(SX1278_REG_PA_DAC, c->pa_dac);
103 }
104 SX1278_WriteReg(SX1278_REG_MODEM_CONFIG_1, c->modem_cfg1);
105 SX1278_WriteReg(SX1278_REG_MODEM_CONFIG_2, c->modem_cfg2);
106 SX1278_WriteReg(SX1278_REG_MODEM_CONFIG_3, c->modem_cfg3);
107
108 SX1278_WriteReg(0x39, 0x12);
109
110 SX1278_WriteReg(SX1278_REG_FIFO_TX_BASE_ADDR, 0x00);
111 SX1278_WriteReg(SX1278_REG_FIFO_RX_BASE_ADDR, 0x00);
112
113 SX1278_WriteReg(SX1278_REG_DIO_MAPPING_1, (SX1278_DIO0_TX_DONE << 6));
114
115 SX1278_WriteReg(SX1278_REG_IRQ_FLAGS_MASK, 0xF7);
116 SX1278_WriteReg(SX1278_REG_IRQ_FLAGS, 0xFF);
117

```

```

118 return true;
119 }
120
121 bool SX1278_SendPacket(uint8_t *data, uint8_t len, SX1278_Config cfg)
122 {
123     (void)cfg;
124
125     if (len > SX1278_MAX_PACKET_SIZE) return false;
126
127     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_STDBY);
128
129     SX1278_WriteReg(SX1278_REG_FIFO_ADDR_PTR, 0x00);
130     SX1278_WriteReg(SX1278_REG_PAYLOAD_LENGTH, len);
131     SX1278_WriteBurst(SX1278_REG_FIFO, data, len);
132
133     sx1278_tx_done = false;
134
135     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_TX);
136
137     uint32_t timeout = HAL_GetTick() + 5000;
138     while (1)
139     {
140         uint8_t flags = SX1278_ReadReg(SX1278_REG_IRQ_FLAGS);
141         if (flags & 0x08)
142         {
143             sx1278_tx_done = true;
144             break;
145         }
146         if (HAL_GetTick() > timeout) break;
147     }
148
149     SX1278_WriteReg(SX1278_REG_IRQ_FLAGS, 0xFF);
150
151     if (!sx1278_tx_done) return false;
152
153     return true;
154 }
155
156 void SX1278_EnterSleep(void)
157 {
158     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_SLEEP);
159 }
160
161 void SX1278_SetStandby(void)
162 {
163     SX1278_WriteReg(SX1278_REG_OP_MODE, SX1278_MODE_LONG_RANGE | SX1278_MODE_STDBY);
164 }
165
166 uint8_t SX1278_ReadVersion(void)
167 {
168     return SX1278_ReadReg(SX1278_REG_VERSION);
169 }
170
171 uint8_t SX1278_ReadIrqFlags(void)
172 {
173     return SX1278_ReadReg(SX1278_REG_IRQ_FLAGS);
174 }
175
176 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
177 {
178     if (GPIO_Pin == LORA_DIO0_Pin)
179     {
180         sx1278_tx_done = true;
181     }
182 }
183

```

Sensor - Sources → [syscalls.c](#)

Path: sensor/Core/Src/syscalls.c | Lines: 245

```

1 /**
2  *
3  * @file syscalls.c
4  * @author Auto-generated by STM32CubeMX
5  * @brief Minimal System calls file

```

```

6 *
7 * For more information about which c-functions
8 * need which of these lowlevel functions
9 * please consult the Newlib or Picolibc libc-manual
10 *****
11 * @attention
12 *
13 * Copyright (c) 2020-2025 STMicroelectronics.
14 * All rights reserved.
15 *
16 * This software is licensed under terms that can be found in the LICENSE file
17 * in the root directory of this software component.
18 * If no LICENSE file comes with this software, it is provided AS-IS.
19 *
20 *****
21 */
22
23 /* Includes */
24 #include <sys/stat.h>;
25 #include <stdlib.h>;
26 #include <errno.h>;
27 #include <stdio.h>;
28 #include <signal.h>;
29 #include <time.h>;
30 #include <sys/time.h>;
31 #include <sys/times.h>;
32
33
34 /* Variables */
35 extern int __io_putchar(int ch) __attribute__((weak));
36 extern int __io_getchar(void) __attribute__((weak));
37
38
39 char *__env[1] = { 0 };
40 char **environ = __env;
41
42
43 /* Functions */
44 void initialise_monitor_handles()
45 {
46 }
47
48 int _getpid(void)
49 {
50     return 1;
51 }
52
53 int _kill(int pid, int sig)
54 {
55     (void)pid;
56     (void)sig;
57     errno = EINVAL;
58     return -1;
59 }
60
61 void _exit (int status)
62 {
63     _kill(status, -1);
64     while (1) {} /* Make sure we hang here */
65 }
66
67 __attribute__((weak)) int _read(int file, char *ptr, int len)
68 {
69     (void)file;
70     int DataIdx;
71
72     for (DataIdx = 0; DataIdx < len; DataIdx++)
73     {
74         *ptr++ = __io_getchar();
75     }
76
77     return len;
78 }
79
80 __attribute__((weak)) int _write(int file, char *ptr, int len)

```

```

81 {
82 (void)file;
83 int DataIdx;
84
85 for (DataIdx = 0; DataIdx < len; DataIdx++)
86 {
87 __io_putchar(*ptr++);
88 }
89 return len;
90 }
91
92 int _close(int file)
93 {
94 (void)file;
95 return -1;
96 }
97
98
99 int _fstat(int file, struct stat *st)
100 {
101 (void)file;
102 st->st_mode = S_IFCHR;
103 return 0;
104 }
105
106 int _isatty(int file)
107 {
108 (void)file;
109 return 1;
110 }
111
112 int _lseek(int file, int ptr, int dir)
113 {
114 (void)file;
115 (void)ptr;
116 (void)dir;
117 return 0;
118 }
119
120 int _open(char *path, int flags, ...)
121 {
122 (void)path;
123 (void)flags;
124 /* Pretend like we always fail */
125 return -1;
126 }
127
128 int _wait(int *status)
129 {
130 (void)status;
131 errno = ECHILD;
132 return -1;
133 }
134
135 int _unlink(char *name)
136 {
137 (void)name;
138 errno = ENOENT;
139 return -1;
140 }
141
142 clock_t _times(struct tms *buf)
143 {
144 (void)buf;
145 return -1;
146 }
147
148 int _stat(const char *file, struct stat *st)
149 {
150 (void)file;
151 st->st_mode = S_IFCHR;
152 return 0;
153 }
154
155 int _link(char *old, char *new)

```

```

156 {
157 (void)old;
158 (void)new;
159 errno = EMLINK;
160 return -1;
161 }
162
163 int _fork(void)
164 {
165 errno = EAGAIN;
166 return -1;
167 }
168
169 int _execve(char *name, char **argv, char **env)
170 {
171 (void)name;
172 (void)argv;
173 (void)env;
174 errno = ENOMEM;
175 return -1;
176 }
177
178 // --- Picolibc Specific Section ---
179 #if defined(__PICOLIBC__)
180
181 /**
182 * @brief Picolibc helper function to output a character to a FILE stream.
183 * This redirects the output to the low-level __io_putchar function.
184 * @param c Character to write.
185 * @param file FILE stream pointer (ignored).
186 * @retval int The character written.
187 */
188 static int starm_putc(char c, FILE *file)
189 {
190 (void) file;
191 __io_putchar(c);
192 return c;
193 }
194
195 /**
196 * @brief Picolibc helper function to input a character from a FILE stream.
197 * This redirects the input from the low-level __io_getchar function.
198 * @param file FILE stream pointer (ignored).
199 * @retval int The character read, cast to an unsigned char then int.
200 */
201 static int starm_getc(FILE *file)
202 {
203 unsigned char c;
204 (void) file;
205 c = __io_getchar();
206 return c;
207 }
208
209 // Define and initialize the standard I/O streams for Picolibc.
210 // FDEV_SETUP_STREAM connects the starm_putc and starm_getc helper functions to a FILE structure.
211 // _FDEV_SETUP_RW indicates the stream is for reading and writing.
212 static FILE __stdio = FDEV_SETUP_STREAM(starm_putc,
213 starm_getc,
214 NULL,
215 _FDEV_SETUP_RW);
216
217 // Assign the standard stream pointers (stdin, stdout, stderr) to the initialized stream.
218 // Picolibc uses these pointers for standard I/O operations (printf, scanf, etc.).
219 FILE *const stdin = &__stdio;
220 __strong_reference(stdin, stdout);
221 __strong_reference(stdin, stderr);
222
223 // Create strong aliases mapping standard C library function names (without underscore)
224 // to the implemented system call stubs (with underscore). Picolibc uses these
225 // standard names internally, so this linking is required.
226 __strong_reference(_read, read);
227 __strong_reference(_write, write);
228 __strong_reference(_times, times);
229 __strong_reference(_execve, execve);
230 __strong_reference(_fork, fork);

```

```

231 __strong_reference(_link, link);
232 __strong_reference(_unlink, unlink);
233 __strong_reference(_stat, stat);
234 __strong_reference(_wait, wait);
235 __strong_reference(_open, open);
236 __strong_reference(_close, close);
237 __strong_reference(_lseek, lseek);
238 __strong_reference(_isatty, isatty);
239 __strong_reference(_fstat, fstat);
240 __strong_reference(_exit, exit);
241 __strong_reference(_kill, kill);
242 __strong_reference(_getpid, getpid);
243
244 #endif // __PICOLIBC__
245

```

Sensor - Sources → [systemem.c](#)

Path: sensor/Core/Src/systemem.c | Lines: 88

```

1  /**
2  *
3  * @file systemem.c
4  * @author Generated by STM32CubeMX
5  * @brief System Memory calls file
6  *
7  * For more information about which C functions
8  * need which of these lowlevel functions
9  * please consult the Newlib or Picolibc libc manual
10 *
11 * @attention
12 *
13 * Copyright (c) 2025 STMicroelectronics.
14 * All rights reserved.
15 *
16 * This software is licensed under terms that can be found in the LICENSE file
17 * in the root directory of this software component.
18 * If no LICENSE file comes with this software, it is provided AS-IS.
19 *
20 *
21 */
22
23 /* Includes */
24 #include <errno.h>;
25 #include <stdint.h>;
26 #include <stddef.h>;
27
28 /**
29 * Pointer to the current high watermark of the heap usage
30 */
31 static uint8_t *__sbrk_heap_end = NULL;
32
33 /**
34 * @brief _sbrk() allocates memory to the newlib heap and is used by malloc
35 * and others from the C library
36 *
37 * @verbatim
38 * #####
39 * # .data # .bss # newlib heap # MSP stack #
40 * # # # # Reserved by _Min_Stack_Size #
41 * #####
42 * ^-- RAM start ^-- _end _estack, RAM end --^
43 * @endverbatim
44 *
45 * This implementation starts allocating at the '_end' linker symbol
46 * The '_Min_Stack_Size' linker symbol reserves a memory for the MSP stack
47 * The implementation considers '_estack' linker symbol to be RAM end
48 * NOTE: If the MSP stack, at any point during execution, grows larger than the
49 * reserved size, please increase the '_Min_Stack_Size'.
50 *
51 * @param incr Memory size
52 * @return Pointer to allocated memory
53 */
54 void *_sbrk(ptrdiff_t incr)
55 {
56 extern uint8_t _end; /* Symbol defined in the linker script */

```

```

57 extern uint8_t _estack; /* Symbol defined in the linker script */
58 extern uint32_t _Min_Stack_Size; /* Symbol defined in the linker script */
59 const uint32_t stack_limit = (uint32_t)&_estack - (uint32_t)&_Min_Stack_Size;
60 const uint8_t *max_heap = (uint8_t *)stack_limit;
61 uint8_t *prev_heap_end;
62
63 /* Initialize heap end at first call */
64 if (NULL == __sbrk_heap_end)
65 {
66     __sbrk_heap_end = &_end;
67 }
68
69 /* Protect heap from growing into the reserved MSP stack */
70 if (__sbrk_heap_end + incr > max_heap)
71 {
72     errno = ENOMEM;
73     return (void *)-1;
74 }
75
76 prev_heap_end = __sbrk_heap_end;
77 __sbrk_heap_end += incr;
78
79 return (void *)prev_heap_end;
80 }
81
82 #if defined(__PICOLIBC__)
83 // Picolibc expects syscalls without the leading underscore.
84 // This creates a strong alias so that
85 // calls to `sbrk()` are resolved to our `_sbrk()` implementation.
86 __strong_reference(_sbrk, sbrk);
87 #endif
88

```

Sensor - Sources → [system_stm32l0xx.c](#)

Path: sensor/Core/Src/system_stm32l0xx.c | Lines: 274

```

1 /**
2  ****
3  * @file system_stm32l0xx.c
4  * @author MCD Application Team
5  * @brief CMSIS Cortex-M0+ Device Peripheral Access Layer System Source File.
6  *
7  * This file provides two functions and one global variable to be called from
8  * user application:
9  * - SystemInit(): This function is called at startup just after reset and
10 * before branch to main program. This call is made inside
11 * the "startup_stm32l0xx.s" file.
12 *
13 * - SystemCoreClock variable: Contains the core clock (HCLK), it can be used
14 * by the user application to setup the SysTick
15 * timer or configure other parameters.
16 *
17 * - SystemCoreClockUpdate(): Updates the variable SystemCoreClock and must
18 * be called whenever the core clock is changed
19 * during program execution.
20 *
21 *
22  ****
23  * @attention
24  *
25  * Copyright (c) 2016 STMicroelectronics.
26  * All rights reserved.
27  *
28  * This software is licensed under terms that can be found in the LICENSE file
29  * in the root directory of this software component.
30  * If no LICENSE file comes with this software, it is provided AS-IS.
31  *
32  ****
33  */
34
35 /** @addtogroup CMSIS
36  * @{
37  */
38
39 /** @addtogroup stm32l0xx_system

```

```

40 * @{
41 */
42
43 /** @addtogroup STM32L0xx_System_Private_Includes
44 * @{
45 */
46
47 #include <stm32l0xx.h>;
48
49 #if !defined (HSE_VALUE)
50 #define HSE_VALUE ((uint32_t)8000000U) /*!&lt; Value of the External oscillator in Hz */
51 #endif /* HSE_VALUE */
52
53 #if !defined (MSI_VALUE)
54 #define MSI_VALUE ((uint32_t)2097152U) /*!&lt; Value of the Internal oscillator in Hz*/
55 #endif /* MSI_VALUE */
56
57 #if !defined (HSI_VALUE)
58 #define HSI_VALUE ((uint32_t)16000000U) /*!&lt; Value of the Internal oscillator in Hz*/
59 #endif /* HSI_VALUE */
60
61
62 /**
63 * @}
64 */
65
66 /** @addtogroup STM32L0xx_System_Private_TypesDefinitions
67 * @{
68 */
69
70 /**
71 * @}
72 */
73
74 /** @addtogroup STM32L0xx_System_Private_Defines
75 * @{
76 */
77 /***** Miscellaneous Configuration *****/
78
79 /* Note: Following vector table addresses must be defined in line with linker
80 configuration. */
81 /*!< Uncomment the following line if you need to relocate the vector table
82 anywhere in Flash or Sram, else the vector table is kept at the automatic
83 remap of boot address selected */
84 /* #define USER_VECT_TAB_ADDRESS */
85
86 #if defined(USER_VECT_TAB_ADDRESS)
87 /*!< Uncomment the following line if you need to relocate your vector Table
88 in Sram else user remap will be done in Flash. */
89 /* #define VECT_TAB_SRAM */
90 #if defined(VECT_TAB_SRAM)
91 #define VECT_TAB_BASE_ADDRESS SRAM_BASE /*!&lt; Vector Table base address field.
92 This value must be a multiple of 0x200. */
93 #else
94 #define VECT_TAB_BASE_ADDRESS FLASH_BASE /*!&lt; Vector Table base address field.
95 This value must be a multiple of 0x200. */
96 #endif /* VECT_TAB_SRAM */
97 #if !defined(VECT_TAB_OFFSET)
98 #define VECT_TAB_OFFSET 0x00000000U /*!&lt; Vector Table offset field.
99 This value must be a multiple of 0x200. */
100 #endif /* VECT_TAB_OFFSET */
101 #endif /* USER_VECT_TAB_ADDRESS */
102
103 /***** *****/
104 /**
105 * @}
106 */
107
108 /** @addtogroup STM32L0xx_System_Private_Macros
109 * @{
110 */
111
112 /**
113 * @}
114 */

```

```

115
116 /** @addtogroup STM32L0xx_System_Private_Variables
117 * @{
118 */
119 /* This variable is updated in three ways:
120 1) by calling CMSIS function SystemCoreClockUpdate()
121 2) by calling HAL API function HAL_RCC_GetHCLKFreq()
122 3) each time HAL_RCC_ClockConfig() is called to configure the system clock frequency
123 Note: If you use this function to configure the system clock; then there
124 is no need to call the 2 first functions listed above, since SystemCoreClock
125 variable is updated automatically.
126 */
127 uint32_t SystemCoreClock = 2097152U; /* 32.768 kHz * 2^6 */
128 const uint8_t AHBPrescTable[16] = {0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U, 6U, 7U, 8U, 9U};
129 const uint8_t APBPrescTable[8] = {0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U};
130 const uint8_t PLLMulTable[9] = {3U, 4U, 6U, 8U, 12U, 16U, 24U, 32U, 48U};
131
132 /**
133 * @}
134 */
135
136 /** @addtogroup STM32L0xx_System_Private_FunctionPrototypes
137 * @{
138 */
139
140 /**
141 * @}
142 */
143
144 /** @addtogroup STM32L0xx_System_Private_Functions
145 * @{
146 */
147
148 /**
149 * @brief Setup the microcontroller system.
150 * @param None
151 * @retval None
152 */
153 void SystemInit (void)
154 {
155 /* Configure the Vector Table location add offset address -----*/
156 #if defined (USER_VECT_TAB_ADDRESS)
157 SCB->VTOR = VECT_TAB_BASE_ADDRESS | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal SRAM */
158 #endif /* USER_VECT_TAB_ADDRESS */
159 }
160
161 /**
162 * @brief Update SystemCoreClock variable according to Clock Register Values.
163 * The SystemCoreClock variable contains the core clock (HCLK), it can
164 * be used by the user application to setup the SysTick timer or configure
165 * other parameters.
166 *
167 * @note Each time the core clock (HCLK) changes, this function must be called
168 * to update SystemCoreClock variable value. Otherwise, any configuration
169 * based on this variable will be incorrect.
170 *
171 * @note - The system frequency computed by this function is not the real
172 * frequency in the chip. It is calculated based on the predefined
173 * constant and the selected clock source:
174 *
175 * - If SYSCLK source is MSI, SystemCoreClock will contain the MSI
176 * value as defined by the MSI range.
177 *
178 * - If SYSCLK source is HSI, SystemCoreClock will contain the HSI_VALUE(*)
179 *
180 * - If SYSCLK source is HSE, SystemCoreClock will contain the HSE_VALUE(**)
181 *
182 * - If SYSCLK source is PLL, SystemCoreClock will contain the HSE_VALUE(**)
183 * or HSI_VALUE(*) multiplied/divided by the PLL factors.
184 *
185 * (*) HSI_VALUE is a constant defined in stm32l0xx_hal.h file (default value
186 * 16 MHz) but the real value may vary depending on the variations
187 * in voltage and temperature.
188 *
189 * (**) HSE_VALUE is a constant defined in stm32l0xx_hal.h file (default value

```

```

190 * 8 MHz), user has to ensure that HSE_VALUE is same as the real
191 * frequency of the crystal used. Otherwise, this function may
192 * have wrong result.
193 *
194 * - The result of this function could be not correct when using fractional
195 * value for HSE crystal.
196 * @param None
197 * @retval None
198 */
199 void SystemCoreClockUpdate (void)
200 {
201     uint32_t tmp = 0U, pllmul = 0U, plldiv = 0U, pllsource = 0U, msirange = 0U;
202
203     /* Get SYSCLK source -----*/
204     tmp = RCC->CFGR & RCC_CFGR_SWS;
205
206     switch (tmp)
207     {
208         case 0x00U: /* MSI used as system clock */
209             msirange = (RCC->ICSCR & RCC_ICSCR_MSIRANGE) >> RCC_ICSCR_MSIRANGE_Pos;
210             SystemCoreClock = (32768U * (1U << (msirange + 1U)));
211             break;
212         case 0x04U: /* HSI used as system clock */
213             if ((RCC->CR & RCC_CR_HSIDIVF) != 0U)
214             {
215                 SystemCoreClock = HSI_VALUE / 4U;
216             }
217             else
218             {
219                 SystemCoreClock = HSI_VALUE;
220             }
221             break;
222         case 0x08U: /* HSE used as system clock */
223             SystemCoreClock = HSE_VALUE;
224             break;
225         default: /* PLL used as system clock */
226             /* Get PLL clock source and multiplication factor -----*/
227             pllmul = RCC->CFGR & RCC_CFGR_PLLMUL;
228             plldiv = RCC->CFGR & RCC_CFGR_PLLDIV;
229             pllmul = PLLMulTable[(pllmul >> RCC_CFGR_PLLMUL_Pos)];
230             plldiv = (plldiv >> RCC_CFGR_PLLDIV_Pos) + 1U;
231
232             pllsource = RCC->CFGR & RCC_CFGR_PLLSRC;
233
234             if (pllsource == 0x00U)
235             {
236                 /* HSI oscillator clock selected as PLL clock entry */
237                 if ((RCC->CR & RCC_CR_HSIDIVF) != 0U)
238                 {
239                     SystemCoreClock = (((HSI_VALUE / 4U) * pllmul) / plldiv);
240                 }
241                 else
242                 {
243                     SystemCoreClock = (((HSI_VALUE) * pllmul) / plldiv);
244                 }
245             }
246             else
247             {
248                 /* HSE selected as PLL clock entry */
249                 SystemCoreClock = (((HSE_VALUE) * pllmul) / plldiv);
250             }
251             break;
252     }
253     /* Compute HCLK clock frequency -----*/
254     /* Get HCLK prescaler */
255     tmp = AHBPrescTable[((RCC->CFGR & RCC_CFGR_HPRE) >> RCC_CFGR_HPRE_Pos)];
256     /* HCLK clock frequency */
257     SystemCoreClock >>= tmp;
258 }
259
260
261
262 /**
263 * @}
264 */

```

```
265
266 /**
267  * @}
268 */
269
270 /**
271  * @}
272 */
273
274
```